

Context-aware Multi-agent Systems Using Deep Graph Learning and Deep Reinforcement Learning

Confirmation of Candidature (COC) Report

Vinh Hung Du
Student ID: 224119729

Supervisory Panel:

Dr Srikanth Thudumu (Principal Supervisor)
Prof. Kon Mouzakis (Associate Supervisor)
Prof. Rajesh Vasa (Associate Supervisor)



A²I²
APPLIED ARTIFICIAL
INTELLIGENCE INSTITUTE



31st October, 2024

Abstract

Recent research in Large Language Models (LLMs), deep multi-modal representation, Deep Graph Learning (DGL), and Deep Reinforcement Learning (DRL) highlight the potential for Artificial Intelligence (AI) agents to perform human-like tasks within a multi-agent framework. Multi-Agent Reinforcement Learning (MARL) is commonly employed to coordinate multiple agents for complex tasks, with strategies categorised into Centralised (Cen-MARL) and Decentralised (Dec-MARL) strategies. While Cen-MARL has been widely explored, its reliance on central coordination often leads to suboptimal task performance in real-world scenarios. These limitations are evident in domains such as search and rescue, autonomous vehicles, and military operations involving unmanned aerial vehicles (UAVs), where decentralisation introduces uncertainties, including unknown states, the presence of other agents, and novel conditions. To alleviate these challenges, it is necessary to develop strategies that dynamically synthesise interaction protocols among agents to enhance autonomous decision-making in Dec-MARL under uncertainty. In addition, context becomes crucial in such strategies. In Multi-Agent Systems (MAS), context can be communication among agents, knowledge relationships, and agent organisation structure. These data are modeled as graphs, and hence, can be represented by leveraging DGL. These representations can be incorporated into the latent space or reward functions in DRL to improve agent decision-making. This thesis aims to investigate and develop strategies that integrate DGL and DRL within Decentralized MAS (Dec-MAS) to enhance agent communication and coordination in dynamic, uncertain environments.

Contents

1	Introduction	1
1.1	Background	1
1.2	Research Hypotheses	6
1.3	Scope and Constraints	7
1.4	Outline	8
2	Research Methodology	9
2.1	Research Design	9
2.2	Data Collection Methods	10
2.3	Hypothesis Validation	11
3	Research Plan	15
3.1	Completed Research Work	15
3.2	Potential Publications	17
3.3	Target Conferences/Journals for Publications	17
3.4	Research Timeline	18
A	Literature Review	20
A.1	Front Matter	20
A.2	Introduction	21
A.3	Multi-Agent Systems	24
A.4	Context-Aware Systems	34
A.5	Context-Aware Multi-agent Systems	37
A.5.1	Awareness in MAS	39
A.5.2	The General Process of CA-MAS	40
A.6	Research Gaps and Future Directions	52
B	Goal-Aware Multi-Agent Reinforcement Learning for Decentralized Coordination	54
B.1	Front Matter	54
B.2	Introduction	55
B.3	Related Work	57

B.4	Problem Preliminary	58
B.5	Our Approach	59
B.5.1	Environment and Rewards	59
B.5.2	State, Action and Relay Buffer	61
B.5.3	Coordination Strategy	61
B.6	Experiments	62
B.6.1	Experiment Details	62
B.6.2	Results and Discussion	63
B.7	Conclusion and Future Work	64
C	Contextual Knowledge Sharing in Multi-Agent Reinforcement Learning with Decentralized Communication and Coordination	65
C.1	Front Matter	65
C.2	Introduction	66
C.3	Related Work	69
C.4	Problem Preliminary	71
C.5	Method	71
C.5.1	Representations and Value Approximation	71
C.5.2	Time Awareness and Intrinsic Rewards	73
C.5.3	Integration of Communication and Coordination	74
C.6	Experiments	78
C.6.1	Environments and Tasks	78
C.6.2	Implementation Details	78
C.6.3	Results and Discussion	80
C.7	Conclusion and Future Work	82

Chapter 1

Introduction

This chapter provides an introduction to the challenges and issues faced in multi-agent systems, which serve as the foundation for the research motivation, objectives, and questions explored in this thesis. We begin by highlighting problems within this field, setting the context for why this research is necessary. Following this, the scope of the research is outlined, including specific constraints and limitations that guide the presented work. Finally, we provide an overview of the structure of the thesis, giving the reader a clear road map of the content and progression of the remaining chapters.

1.1 Background

The rapidly evolving landscape of the Industrial Internet of Things (IIoT) and Artificial Intelligence (AI) introduces the high demand for AI applications on devices where each device can be an autonomous intelligent agent with human-like task performance. This leads to the more complex and dynamic environment where a vast number of agents are interconnected, communicate and coordinate in the centralized settings, the decentralized settings or both. Many existing AI systems or machine learning algorithms [1, 2] often struggle with such dynamism and scalability due to two main assumptions: (i) a single agent can be utilised as the whole system; and (ii) the environment is always static. To alleviate these challenges, the study of Multi-Agent Systems (MAS) becomes crucial. Specifically, MAS aim to enhance the robustness, generalisation and scalability of AI systems in dealing with the dynamic environment. Furthermore, existing surveys show that capabilities of MAS have been applied to tackle real-world problems in diverse domains such as computer networking, cyber security, social networks, robotics, smart cities and transportation, intelligent information system, public health, manufacturing, and education [3, 4]. Despite the wide range of applications, developing effective strategies to manage agent interactions in environ-

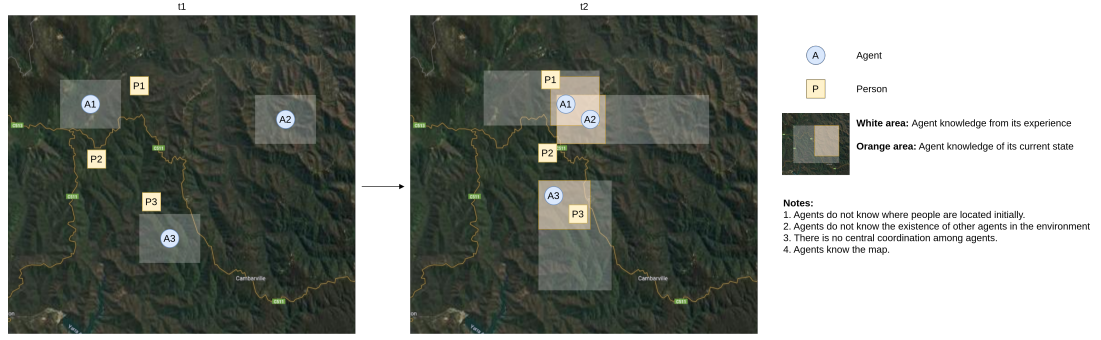


Figure 1.1: An illustration of search and rescue with multiple drones. The drone agents are individual agents without central coordination. In addition, they have limited observability and knowledge, and are required to find one or more people. To achieve their goals, these drone agents need to use peer-to-peer communication, decentralised coordination and knowledge sharing.

ments characterised by temporal heterogeneity remains a significant challenge. Tackling this challenge require an individual agent to have the following capabilities: (i) communication; (ii) coordination; (iii) organisational formation; (iv) competition; and (v) adaptation. Existing methodologies [3, 5–7] were proposed to endow agents with each of these capabilities. However, two key challenges in many MAS remain open: (i) establishing an effective interaction protocol among agents without central coordination; and (ii) managing each agent’s beliefs about the intentions of other agents to enhance the autonomous decision-making process. Examples of this are shown in Figures 1.1 and 1.2.

Motivation

Central coordination limits applications of many existing AI systems with autonomous agents in various real-world problem domains (e.g., search and rescue, autonomous driving vehicles, military operation with UAVs, and more). Without central coordination, there exists challenges of communication and coordination among decentralised agents in dynamic environments. This motivates the need for developing methodologies that dynamically synthesise interaction protocols among agents and improve autonomous decision-making under uncertainties.

An agent is an autonomous computational entity, which can be a software program, a hardware component, or a combination of both. It is trained to perform a specific task with a well-defined goal. In a multi-agent system (MAS), several agents interact with one another and their environment to achieve both individual and collective objectives. These interactions involve communication, coordination, and decision-making processes. The system may have multiple goals that need to be aligned across agents. Initially, each agent is assigned a subset

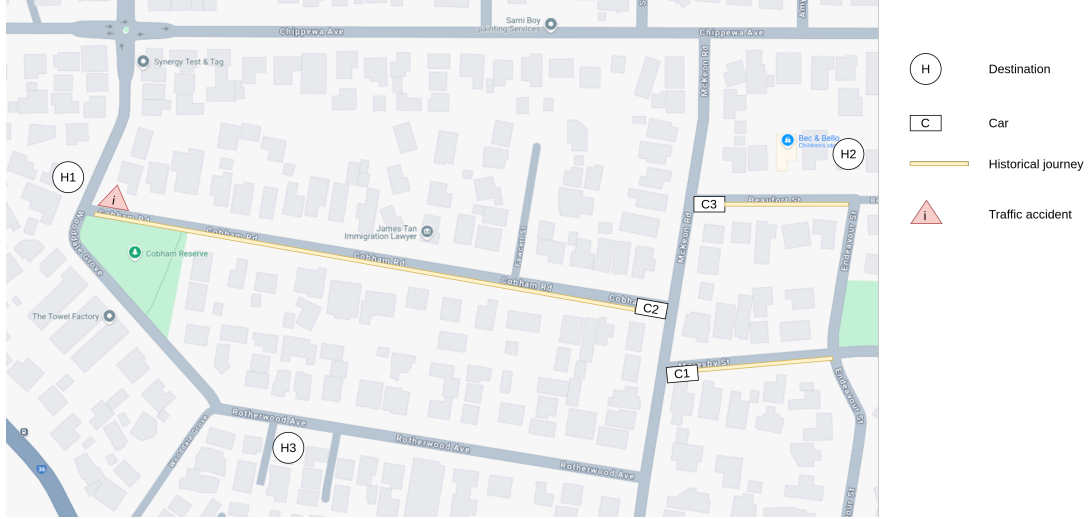


Figure 1.2: An illustration of cars finding optimal journey of reaching home. Car agents independently find their optimal paths to reach their respective homes (e.g., H1 for C1, H2 for C2, and H3 for C3) without central coordination. Each car has a history of previous journeys and must determine the best route to its destination. C1’s optimal route would normally involve using a middle lane that C2 has previously traveled. However, C2 observed a traffic accident in that lane and informs both C1 and C3. C1 receives the update and adjusts its route to avoid the blockage, while C3 ignores the information as it is not relevant to its goal.

of the system’s overall goals. Details of characteristics of MAS can be found in Section A.3. Furthermore, communication and coordination in MAS involves,

- An **interaction protocol** includes rules, methods, or conventions that are used by agents to establish communication.
- A **communication channel** provides the medium for information exchange among agents.
- The **coordination process** involves retrieving, sharing, and integrating information and knowledge that support agents to achieve their goals [8].

This is facilitated by centralised or decentralised strategies. Without central coordination, agents that are trained with existing centralised strategies [9–15] tend to perform tasks poorly and achieve goals suboptimally. This can be due to the following uncertainties introduced by decentralisation:

- **Environment** (Unknown states): Environment that the agent has not explored.
- **Presence** (of other agents): An agent does not know where other agents are in the environment.

- **History** (Novelty): Both agents and non-agents (e.g., objects and people) are keep moving, so historical knowledge may not be accurate.

Specifically, in spatial environments, decentralised multi-agent systems have to operate with the following constraints:

1. **Decentralised communication and coordination:** Without a central coordinating system, agents rely on peer-to-peer communication and autonomous coordination.
2. **Sensing:** Limited sensing and communication range (various factors such as energy, sensor depth etc.). As agents navigate the environment, they can recognise other agents as well as non-agents (static/dynamic objects in the environment).
3. **Dynamics:** Despite being structurally stable (e.g. terrain is stable), there is still dynamics in the environment (e.g., agents and non-agents are moving, dynamically appear, disappear with innate uncertainty).
4. **Temporal Communication:** Communication channels are transient (due to navigational dynamics, physics of environment), leading to challenges of sharing, integrating and utilising knowledge effectively.
5. **Dynamic Interaction Protocol:** Aspects of agent-agent interaction protocols must be synthesised dynamically since time window is limited, and value of information is changing in dynamic environment.

Moreover, the autonomous decision-making of an agent during navigation involves the following activities: (i) knowledge representation, (ii) knowledge memorisation, (iii) communication, (iv) knowledge sharing, and (v) making a decision based on the aggregated knowledge. Enhancing this process necessitates the use of context (see also Figure 1.3) as follows:

- An agent shall represent knowledge differently based on the context (e.g., individual goals, time, or mental state).
- An agent shall filter knowledge before memorising it based on the context (e.g., individual goals).
- An agent shall communicate with other agents based on the context (e.g., individual goals, or shared goals).
- An agent shall share relevant knowledge based on the context (e.g., individual goals, shared goals, or organisational structures).

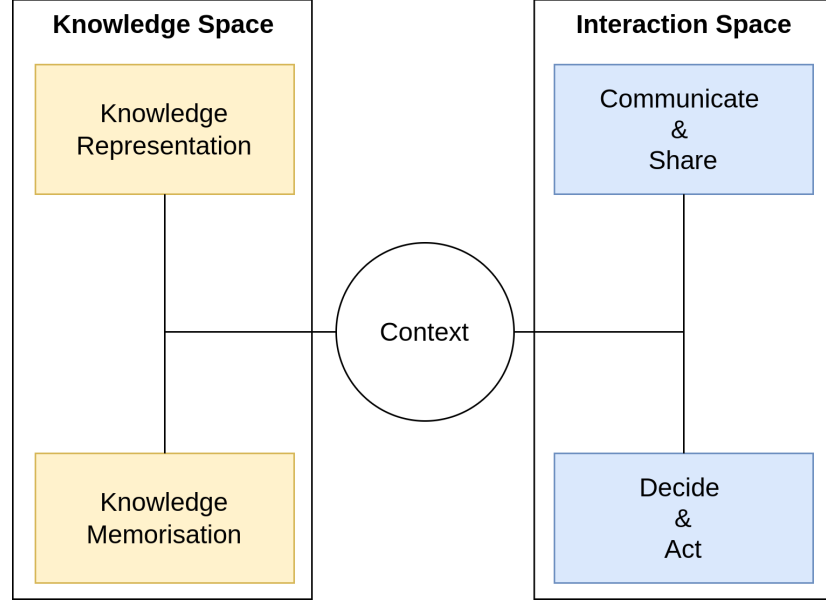


Figure 1.3: An illustration of the connection between the knowledge space and the interaction space of an agent via context

- An agent shall decide and act upon the situation.

Details of context and context awareness are provided in Appendix A.

Adaptability is crucial for agents to make decisions in the dynamic environment. Defining rules for decision-making can be manually intensive and impossible in the environment with the large number of states and actions. Deep Reinforcement Learning (DRL) [16], a decision-making framework, is well known for tackling this challenge. DRL is utilised to train agents to make decision based on two aspects: latent representations of knowledge and the reward function. In addition, existing multi-agent DRL (MA-DRL) strategies are categorised into three groups: Centralised Training and Centralised Execution (CTCE), Centralised Training and Decentralised Execution (CTDE) [9, 10], and Decentralised Training and Decentralised Execution (DTDE) [17, 18]. The adaptability of agents trained with CTDE or CTDE strategies can degrade when operating in the environment that lack central coordination. Agents trained with DTDE strategies can be more adaptable in the dynamic environment. However, these agents encounter aforementioned challenges in decentralised multi-agent systems. To alleviate these challenges, it is necessary to incorporate context into both latent representations of knowledge and the reward function in DTDE strategies.

In MAS, communication, knowledge relationships, and agent organisation structure are data with natural graph structure. Existing surveys [7, 19, 20] demonstrate the use of graph theories in solving challenges of communication between agents. In addition, Zhu *et al.* [6] conducted a survey that demonstrates the usage of graph algorithms in modelling various types of communication and

coordination among agents. Despite a wide range of graph-based approaches that have been applied in different aspects of MAS, representing the graph information in the dynamic environment remains an open challenge. Deep Graph Learning (DGL) [21], a representation learning framework, can potentially tackle this challenge by enabling agents to learn representations and patterns from the graph data. These representations can further incorporate into DRL to support agents make decisions under uncertainty and adapt effectively in dynamic environments [11, 22–25].

Research Objective

In this thesis, we aim to develop effective and novel methodologies that integrate Deep Graph Learning (DGL) and Deep Reinforcement Learning (DRL) into Decentralised Multi-Agent Systems (Dec-MAS) to enhance the communication and coordination among agents under uncertainty in the dynamic environment.

1.2 Research Hypotheses

This thesis aims to develop effective and novel methodologies that tackle challenges of communication and coordination in Decentralised Multi-Agent Systems (Dec-MAS). Our primary research idea is as follows:

Core Research Idea

Decision making in a decentralised strategy can be as optimal as a centralised strategy by integrating Deep Reinforcement Learning and Deep Graph Learning and support agents to effectively communicate and coordinate.

To validate this research idea, it is necessary to explore the set of hypotheses (refer to **Hypotheses**).

Hypotheses

- H1** A dynamically generated context aware protocol of interaction allow agents to communicate and coordinate effectively.
- H2** The combination of Deep Graph Learning (DGL) and Deep Reinforcement Learning (DRL) can support the synthesis of the interaction protocol.
- H3** Our decentralised strategies are as effective as central coordination strategy.

To handle uncertainties and adapt to dynamic environments, agents must utilise context in communication, coordination and decision-making. Hypothesis **H1** aim to validate the improvement in task performance of agents with the following context: individual goals, shared goals, time, mental state, and organisational structures. For example, the interaction protocol among agents can be dynamically synthesised upon the shared goals. To achieve **H1**, the context will be integrated into the reward function of DRL algorithms, such as Deep Q Networks (DQN) [26], Deep Deterministic Policy Gradient (DDPG) [27], and Proximal Policy Optimisation (PPO) [28]. In addition, the context will be used in strategies of aggregating the shared knowledge among agents. Furthermore, as organisational structures and knowledge relationships can be graph data, which can be represented by DGL, Hypothesis **H2** aim to validate whether the integration between DGL and DRL can support the interaction protocol synthesis among agents in the dynamic environment. To achieve this, the representation of knowledge relationships generated by DGL can be utilised for two purposes: (i) incorporating with the knowledge representation to help the agent make optimal decisions; and (ii) identifying and filtering irrelevant knowledge during knowledge aggregation. Moreover, Hypothesis **H3** aim to validate whether Dec-MAS with appropriate strategies can be as effective as Centralised Multi-Agent Systems (Cen-MAS). To achieve this, we will introduce strategies that synthesise interaction protocol dynamically and enhance the coordination among agents in Dec-MAS. The performance of Dec-MAS with these strategies will be compared to that of Cen-MAS in our empirical analysis.

1.3 Scope and Constraints

This thesis aims to explore and validate the research hypotheses outlined in Section 1.2, with a specific focus on multi-agent systems operating in simulated environments. The experiments will primarily involve a variety of Multi-Agent Deep Reinforcement Learning (MADRL) algorithms, such as Deep Q Networks (DQN) [26], Deep Deterministic Policy Gradient (DDPG) [27], and Proximal Policy Optimisation (PPO) [28]. In addition, Deep Graph Learning (DGL) algorithms [21], including Graph Neural Networks, Graph Attention Networks, and Graph Convolutional Networks, will be employed to model and represent graph-based data structures. The study will focus on cooperative agents, without accounting for adversarial or malicious behaviors. Moreover, we focus on operating multiple agents in the decentralised settings and comparing with those operated in the centralised settings. It is worth noting that agents in our experiments may be operated in resource-constrained scenarios.

Several constraints may affect both the direction and outcomes of this research. First, the experiments primarily rely on existing multi-agent benchmarking datasets [29–34]. Most of these datasets are simulated in 2D environments, which may not fully capture the complexity of real-world scenarios. However, customised versions of these datasets are designed to introduce features of the dynamic environment. Second, time constraints limit the number of algorithms tested, which may reduce the comprehensiveness and generalisability of the results. In addition, only a few domain-specific applications, such as autonomous navigation, traffic congestion avoidance, cooperative exploration in 2D environments, and resource allocation are chosen to validate the concepts outlined in Section 1.2. Third, the number of agents in the experiments will not exceed 100, which may impact the scalability of the proposed methodologies. Lastly, to support reproducibility, a fixed level of randomness is applied throughout the experiments.

1.4 Outline

The remainder of this thesis is divided into informative chapters and appendices. Chapter 2 consists of a detailed explanation of research methodology that is utilised to answer research questions and hypotheses in this thesis. Furthermore, Chapter 3 provides the information about our recent achievements and a detailed plan with timeline for the remaining work in this research project. In Appendix A, we provide a comprehensive literature review on Context-Aware Multi-Agent Systems (CA-MAS), including an overview of existing MAS and Context-Aware Systems (CAS), the integration of these systems, application domains as well as challenges and future directions of CA-MAS. Finally, Appendices B and C describe experiments that have been conducted for seeking the answer to Hypotheses **H1** and **H2** in this thesis, respectively.

Chapter 2

Research Methodology

2.1 Research Design

This study employs a quantitative approach to seek for the answer to the research hypotheses posed in Chapter 1. This facilitates statistical analysis of collected data from our experiments and allows us to measure precisely the agent performance across varying conditions and scenarios. The general measurement for the agent performance include cumulative rewards, reward distribution and number of steps for tasks completion. It is worth noting that a reward calculation varies across problem domains. Furthermore, other metrics, such as convergence rate, sample efficiency, coordination and communication efficiency, and stability of learned policies may be applied to research work in this thesis. Moreover, a deductive approach for research work will be applied. In particular, we start with a hypothesis and conduct experiments to validate it through data collection and analysis. The list of hypotheses can be found in Chapter 1.

The research work across this thesis follows an experimental design where multiple agents are trained in different simulated environments using deep reinforcement learning algorithms. All simulated environments at least consist of the following shared characteristics:

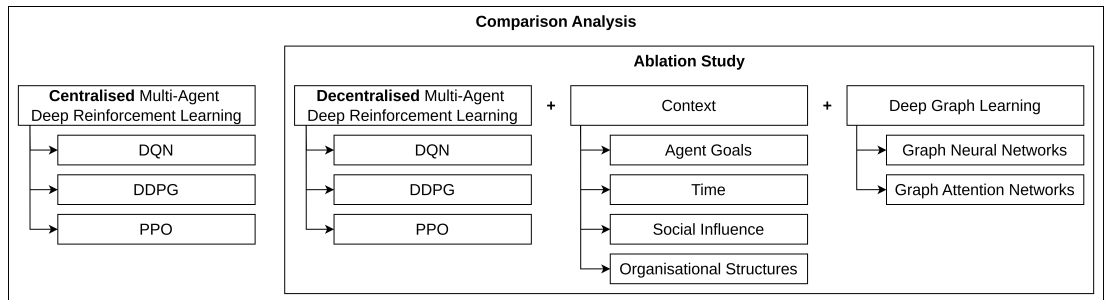


Figure 2.1: Overview of the experment design in this study

1. Agents are operated in the decentralised setting, facilitating peer-to-peer communication and coordination.
2. Agents have individual goals and limited observability at a certain time.
3. Agents are cooperative to achieve their individual goals or shared goals.
4. Agents share the same ontology of the environment. Hence, knowledge sharing is feasible among agents.
5. The information of a state in the environment may change dynamically.

These environments are also restricted accordingly to the scope and constraints outlined in Chapter 1. Aside from that, deep graph learning algorithms, such as Graph Neural Networks, Graph Convolutional Networks, Graph Attention Networks, and others may be utilised to model or represent graph data structures. During the evaluation of each research work, two primary techniques, such as comparison analysis and ablation study, will be applied (refer to Figure 2.1). The comparison analysis aims to compare the performance of the proposed methodology against existing ones. Meanwhile, the ablation study measures the contribution of each newly introduced component in the methodology of each research work.

2.2 Data Collection Methods

In this thesis, we focus on multi-agent reinforcement learning datasets that facilitate measuring agent performance and validating our hypotheses. Several existing datasets, as identified in the literature [29–34], align with our research objectives. These datasets are set in simulated 2D environments, featuring complex tasks across diverse problem domains, such as cooperative games, autonomous navigation and exploration, and sports. For training and evaluation, the number of episodes and steps will remain constant for each scenario. Key data attributes, including rewards, steps, and agent memory, will be recorded. Custom programs will be developed to visualise the collected data, utilizing techniques such as line charts for cumulative rewards, histograms for reward distributions, performance comparison tables, and others.

Aside from existing datasets, it will be necessary to develop new 2D environments with additional data types, such as knowledge graphs, to better support our experiments. We will also modify these datasets by introducing dynamic elements to simulate real-world complexities. For example, obstacles in the environment could appear at time t and disappear at a later time t' , where $t' > t$.

Similarly, while many of these datasets provide agents with full observability at any given state, we may need to adjust this to limited observability for each agent, in line with our experimental requirements.

This study’s data collection has several limitations. First, simulated environments may not fully reflect real-world conditions, making it essential to meticulously test the methodologies in this thesis for their generalisability. Second, the number of agents will not exceed 100, requiring validation of the methodologies for scalability. Lastly, the study covers a limited range of application domains, and hence, it is important to apply and evaluate the methodologies across different domains to ensure broader applicability.

2.3 Hypothesis Validation

This thesis aims to validate three primary hypotheses outlined in Chapter 1. In this section, we present an overview of the methods used to validate these hypotheses.

In Multi-Agent Systems (MAS), uncertainties may arise from novel environmental states, the presence of other agents, novelty of the agent’s historical knowledge (refer to Chapter 1). To handle such uncertainties, agents must incorporate context into communication, coordination, and the decision-making process. To investigate this, we conducted a comprehensive survey of context-aware systems (see also Appendix A). The survey is structured into three key parts: (i) a comprehensive review of existing Context-Aware Systems (CAS), (ii) a comprehensive review of current Multi-Agent Systems (MAS), and (iii) a detailed study of Context-Aware Multi-Agent Systems (CA-MAS) and their applications. Our aim is to examine how context is utilised within MAS and explore approaches that incorporate CAS characteristics into MAS to improve agents’ ability to handle uncertainty and adapt in dynamic environments. The most important concept highlighted in the survey is agent capabilities (Sense, Learn, Reason, Predict, and Act) in CA-MAS, where context can be integrated. Findings from existing CA-MAS indicate that agents must be context-aware to effectively adapt in such environments.

Findings in our comprehensive survey support Hypothesis **H1**. To further validate this, it is essential to conduct experiments. The first research study (see Appendix B) focuses on incorporating goal awareness into the communication and coordination of Decentralised Multi-Agent Systems (Dec-MAS). In this study, agents operate with individual goals and limited observability in a decentralised setting. Additionally, their uncertainties include novel environmental states and the presence of other agents. The agents navigate a 2D environment

where they must explore and make optimal decisions to reach their respective goals. This environment is structured as a multi-agent reinforcement learning (MARL) scenario. The Deep Deterministic Policy Gradient (DDPG) algorithm [27] is applied to each agent. A goal-aware coordination strategy was proposed to improve knowledge sharing among agents, where each agent shares relevant information only with those within its observation range and pursuing the same goal. Furthermore, a sparse reward function was used to encourage agents to move towards their goals and penalize invalid actions. The episodic rewards and the number of steps taken to complete tasks were recorded. Experimental results demonstrated that incorporating goal awareness into the DDPG algorithm improved agent coordination and outperformed the baseline without goal awareness, supporting Hypothesis **H1**.

The knowledge of the agent can be outdated due to rapid changes of the dynamic environment. To communicate and coordinate effectively, agents need to be aware of both goals and the time factor associated with the historical knowledge. To validate this, we conducted the second research study (see Appendix C) focusing on the integration of both goals and time into the communication and coordination of Dec-MAS. In this study, agents also have individual goals and limited observability in a decentralised setting. Not only novel environmental states and the appearance of other agents, but agent uncertainties also include novelty in an agent’s knowledge. This type of uncertainty is caused by the information obsolescence in dynamic environments. Addressing this necessitates agents to be aware of the time aspect associated to the shared knowledge during the communication and coordination sessions. There are four significant contributions in the second research work: (i) a method that represents agent’s mental state; (ii) the time-based measurement for estimating values of knowledge shared among agents during the communication and coordination session; (iii) the intrinsic reward function that motivates agents to explore new states and revisit previously known ones; and (iv) the goal-aware coordination strategy that controls the influence of shared knowledge on the agent’s current knowledge. In our experiments, the environment is structured as a multi-agent reinforcement learning (MARL) scenario. We designed distinct levels of complexity for the environment by controlling two following factors:

- The size of the environment is configurable. In particular, 10x10 and 20x20 are dimensions of the small environment and the large environment, respectively.
- The environment consists of obstacles surrounding objects that are agent goals. These obstacles appear dynamically at a certain time.

The DDPG algorithm is also applied to each agent in the environment. Agents are goal-aware during communication and coordination. Moreover, Multi-Layer Perceptron (MLP) is employed to represent states, observations, goals and mental states in the latent space. This aims to introduce the generalisability to our approach. In addition, these representations are integrated to DDPG to support agents make decisions. As many features are introduced, we conducted the ablation study to measure the impact of these components on the existing baseline (i.e., MADDPG [11]). Experimental results showed the performance improvement of MADDPG while incorporating each feature into communication and coordination among agents. This further supports Hypothesis **H1**.

Research studies 1 and 2 also introduce one critical challenge that the poor-performing agents negatively impact the performance of other agents during communication and coordination. Alleviating this challenge necessitates controlling three primary aspects: (i) social influence, (ii) knowledge verification; and (iii) knowledge aggregation. Validating this opens another research direction. In this thesis, we plan to investigate a novel approach for each aspect, contributing that research direction. This can lead to three following research studies:

RS1 Social Influence in Decentralized Multi-Agent Reinforcement Learning-

ing: This research study explores a novel approach to managing the social influence of shared knowledge among agents in Dec-MARL. As agents interact with peers, temporal relationships between them can be utilised to establish knowledge relationships. Both agent temporal relationships and knowledge relationships are utilised to generate a probability distribution. This distribution reflects the influence that peers exert on a given agent. By integrating this social influence into DRL algorithms, such as DDPG or PPO, agents can regulate the impact of shared knowledge on their actions. Specifically, agents will assign lower probabilities to shared knowledge leading to negative rewards and higher probabilities to knowledge yielding positive outcomes. The study will also empirically evaluate the effects of social influence on agents' decision-making processes. Finally, we will apply this control mechanism within Decentralized Multi-Agent Reinforcement Learning (Dec-MARL) strategies and compare the performance of these strategies. The research outcome includes a novel method that measures and regulates social influence in agent decision-making.

RS2 Temporal Knowledge Graph for Knowledge Verification in Decentralized Multi-Agent Reinforcement Learning:

This research study introduces a novel approach for temporal knowledge verification in Decentralized Multi-Agent Reinforcement Learning (Dec-MARL) using a temporal knowledge

graph. As agents navigate the environment, knowledge can become outdated, making the validity of knowledge relationships time-sensitive. Similarly, agent relationships are also temporal. Therefore, it is crucial to verify this temporal information to ensure that agents retain the knowledge leading to positive outcomes. To achieve this, we propose a novel approach that leverages a DGL algorithm, as highlighted in [35], to model temporal relationships. These representations are then integrated into DRL algorithms, such as DDPG or PPO, to enable agents to prune temporal knowledge that negatively affects their actions. Our approach will be evaluated through ablation studies and compared with existing MARL strategies. The research outcome include a novel method that supports agents in verifying knowledge before aggregation in Dec-MARL.

RS3 Aggregate What Matters: Contextual Knowledge Aggregation in Decentralized Multi-Agent Reinforcement Learning: This research study aims to investigate a novel method to help agents effectively aggregate relevant and critical knowledge during coordination in Dec-MARL. Often, shared knowledge from other agents can be redundant and unnecessary for optimal goal achievement. Therefore, a strategy is needed that enables agents to filter out trivial knowledge and aggregate the important one based on context, such as structural order or knowledge hierarchy. In particular, an agent’s position in the hierarchy may be higher if it is closer to achieving the goal. In this study, we propose an novel approach that leverages a DGL algorithm to represent both agent and knowledge hierarchies, integrating these representations into DRL algorithms, such as DDPG or PPO. This allows agents to select important knowledge before aggregation. The approach will be evaluated through ablation studies and compared with existing MARL strategies. The expected outcome is a novel method that enables agents to contextually aggregate essential information in Dec-MARL.

The evaluation metrics for these research studies will include step-wise rewards, episodic rewards, and the number of steps required to complete tasks. These metrics will be recorded for empirical analysis and comparison of methods. In addition, the performance of the proposed approach in each study will be compared against existing centralised MARL (Cen-MARL) strategies. The experimental results will be used to validate Hypotheses **H2** and **H3**.

Chapter 3

Research Plan

In this chapter, we present a comprehensive research plan to achieve our research goals. The plan details completed research work, potential publications, and the timeline for each phase of the study. Importantly, this research plan is structured to ensure the research question and hypotheses outlined in Chapter 1 are investigated and addressed systematically towards achieving research objectives.

3.1 Completed Research Work

Details of completed research work up to the time of this thesis review are provided below. This is intended to provide a clear understanding of the current progress and establish the foundation for the next phase of the research plan.

Literature Review

As part of this study, a comprehensive literature review was conducted on Context-Aware Multi-Agent Systems (CA-MAS) and their applications. This review outlines the key characteristics of both Multi-Agent Systems (MAS) and Context-Aware Systems (CAS), while also identifying the research gaps and challenges within each category. One potential solution to address these challenges is the integration of the two systems, resulting in CA-MAS, which is feasible due to the overlapping and complementary features of MAS and CAS. The review also includes a categorised list of existing CA-MAS approaches based on their application domains and stages in the general process of CA-MAS. The content of this literature review was organised as a comprehensive survey, which was submitted to the journal of ACM Computing Surveys. Further details of this survey are provided in Appendix A.

The First Research Work

The aim of this research was to validate Hypothesis **H1** (refer to Chapter 1). To achieve this, we introduced goal awareness into decentralised multi-agent systems to enhance coordination among agents. In many real-world scenarios, agents have individual goals and limited observability. Therefore, in this study, agents operated in a decentralised environment, where uncertainties at time t included novel environmental states and unseen agents. For simplicity, experiments were conducted in a 2D grid environment, with each agent tasked to explore the environment and reach a specific goal (i.e., an object placed within the environment).

During exploration, agents could encounter others and exchange knowledge if they shared the same goal, a process known as goal-aware knowledge sharing. The results showed that multi-agent systems with goal-aware coordination outperformed systems with independent agents. In addition, as more agents with the same goal were introduced, the fewer steps were needed to complete their tasks. This research also demonstrated the enhancement of agent coordination when goal awareness was integrated into multi-agent deep reinforcement learning algorithms, supporting Hypothesis **H1**.

One key future direction identified from the experiments is the need for methodologies to mitigate the negative impact of poorly performing agents on others, which is crucial for stabilizing multi-agent systems in decentralized settings. Further details are provided in Appendix B.

The Second Research Work

This research aimed to validate Hypothesis **H1** (refer to Chapter 1). To achieve this, we introduced time awareness into decentralized multi-agent systems to improve the effectiveness of knowledge sharing among agents. In dynamic environments, an agent’s knowledge of states can quickly become outdated, leading to ineffective coordination and information sharing. To address this, agents must account for the time factor associated with shared knowledge. In this study, agents faced uncertainties, including novel environmental states, unseen agents, and the validity of shared information.

The experiments were conducted in complex 2D environments with dynamically appearing obstacles. Additionally, agents were equipped with both goal awareness and time awareness. To assess the contribution of each feature, we performed an ablation study to evaluate their individual impacts on overall agent performance. The results demonstrated that integrating goal-aware and time-aware knowledge sharing significantly improved task performance, supporting Hypothesis **H1**. This content of this research was organised as a technical paper

and submitted to the Association for the Advancement of Artificial Intelligence (AAAI) conference. Further details are provided in Appendix C.

3.2 Potential Publications

One of the expected research outcomes of this thesis include several peer-reviewed publications. We aim to have at least one publication per hypothesis posed in Chapter 1. Note that each publication also aims to investigate and address the hypothesis and contribute to the answer to the research question. Six potential publication are provided below.

- P1** A Comprehensive Survey on Context-Aware Multi-Agent Systems: Techniques, Applications, Challenges and Future Directions (Appendix A)
- P2** Goal-Aware Multi-Agent Reinforcement Learning for Decentralized Coordination (Appendix B)
- P3** Contextual Knowledge Sharing in Multi-Agent Reinforcement Learning with Decentralized Communication and Coordination (Appendix C)
- P4** Social Influence in Decentralized Multi-Agent Reinforcement Learning
- P5** Temporal Knowledge Graph for Knowledge Verification in Decentralized Multi-Agent Reinforcement Learning
- P6** Aggregate What Matters: Contextual Knowledge Aggregation in Decentralized Multi-Agent Reinforcement Learning

3.3 Target Conferences/Journals for Publications

We aim to target the following conferences and journals to publish the findings of our research:

- International Conference on Autonomous Agents and Multiagent Systems (AAMAS) ([Link](#))
- The IEEE International Conference on Robotics and Automation (ICRA) ([Link](#))
- Conference on Neural Information Processing Systems (NeurIPS) ([Link](#))
- The Association for the Advancement of Artificial Intelligence (AAAI) Conference ([Link](#))

- International Joint Conferences on Artificial Intelligence (IJCAI) ([Link](#))
- International Conference on Machine Learning (ICML) ([Link](#))
- Journal of Machine Learning Research (JMLR) ([Link](#))
- Journal of Expert Systems with Applications ([Link](#))
- Journal of ACM Computing Surveys ([Link](#))

3.4 Research Timeline

A Gantt chart is used for presenting the timeline for the research work in this thesis (see also Figure 3.1).

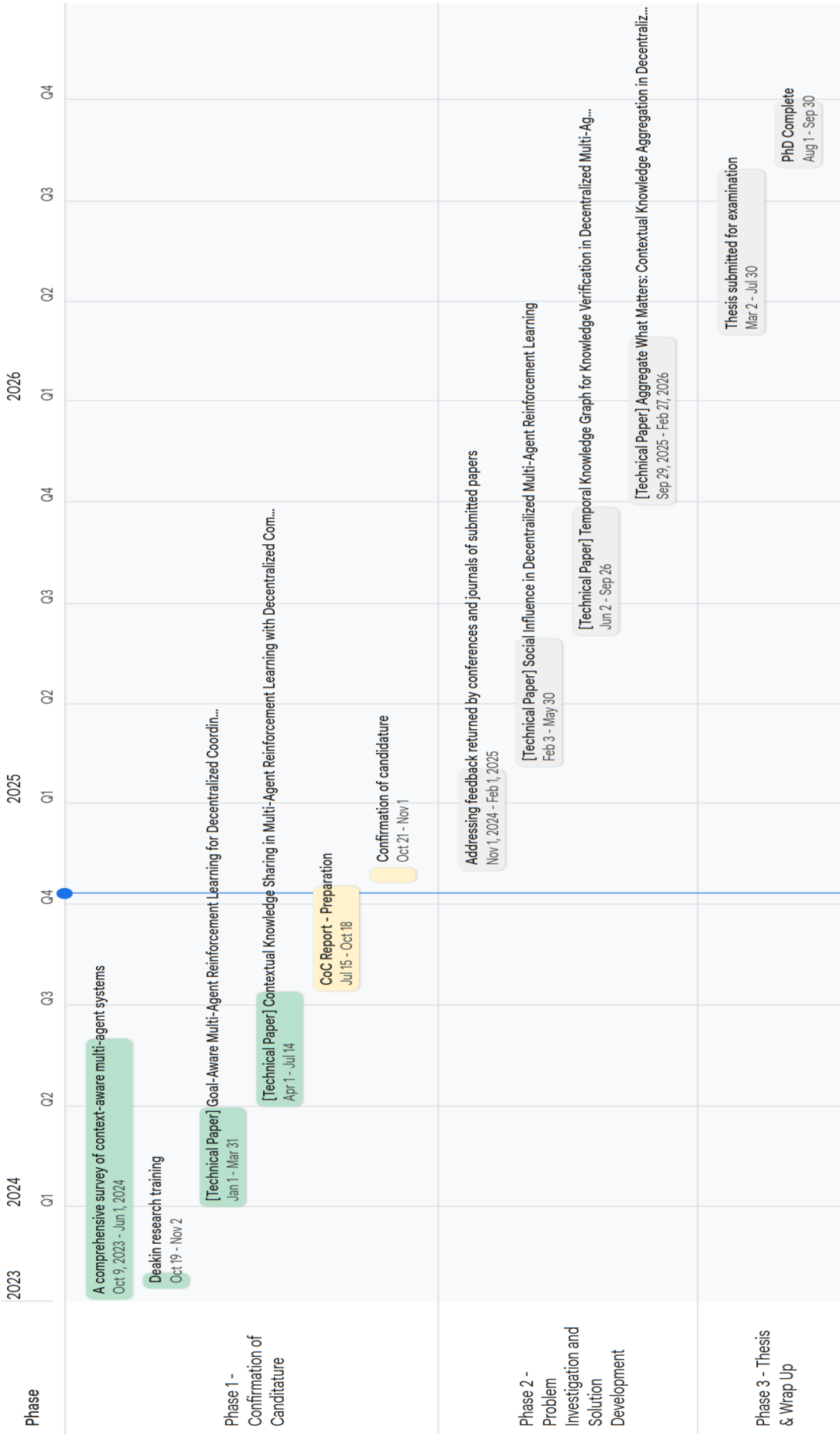


Figure 3.1: Research Timeline

Appendix A

Literature Review

This survey has been submitted to the journal of ACM Computing Surveys and is currently under the peer-review process.

A.1 Front Matter

Title: A Comprehensive Survey on Context-Aware Multi-Agent Systems: Techniques, Applications, Challenges and Future Directions

Hung Du, Srikanth Thudumu, Hy Nguyen, Rajesh Vasa and Kon Mouzakis

Abstract: Research interest in agents is rising, particularly in Artificial Intelligence (AI) techniques such as deep multi-modal representation learning, deep graph learning, and deep reinforcement learning, which suggest potential for human-like task performance within a multi-agent framework. However, challenges exist in enabling these agents to coordinate, learn, reason, predict, and navigate uncertainties in dynamic environments. Context awareness is crucial for enhancing multi-agent systems in such situations. Current research often addresses context-awareness or multi-agent system issues separately, leaving a gap in comprehensive surveys that explore both fields, especially regarding the five essential agent capabilities (Sense-Learn-Reason-Predict-Act). This survey fills that gap by presenting a unified architecture and taxonomy for developing Context-Aware Multi-Agent Systems (CA-MAS), which are vital for improving agent robustness and adaptability in real-world environments. We provide a comprehensive overview of state-of-the-art context-awareness and multi-agent systems, followed by a framework for CA-MAS development. We detail the properties of context-awareness and multi-agent systems, and a general process for context-aware systems, highlighting approaches from various domains such as

ambient intelligence, autonomous navigation, digital assistance, disaster relief management, education, energy efficiency & sustainability, IoT, and supply chain management. Finally, we discuss existing CA-MAS challenges and propose future research directions.

Keywords: Context-Aware Systems, Multi-Agent Systems, Deep Reinforcement Learning, and Artificial Intelligence

A.2 Introduction

In recent years, the Artificial Intelligence (AI) community has been approaching a paradigm shift, transitioning from creating AI systems for passive, structured tasks to developing AI systems that are capable of dynamic, agentic roles in diverse and complex environments. Additionally, developing autonomous agents with human-like task performance is progressing rapidly [36–38] due to the following breakthroughs:

1. **An agent can perceive contextual information:** The development of Context-Aware Systems (CAS) illustrates the agent’s ability to sense and model contextual information and utilizes such data for learning and reasoning purposes [39–49] (see also Section A.4).
2. **An agent can learn any sensory data:** Advancements in deep learning techniques show that multiple types of sensory data, including images, audio, text, and signals, can be transformed into machine-readable formats [50, 51]. These formats are subsequently encoded into learnable tokens through the use of Transformer architectures [52, 53]. Frameworks, such as joint representation, coordinated representation, and Encoder-Decoder facilitate the conversion between tokens of different data modalities [54–56]. These innovations enable an agent to represent and work with various type of sensory data.
3. **An agent can reason:** The semantic relationships of sensory data are encapsulated in the form of an ontology [57–59]. An agent can utilize these relationships for reasoning using rule-based, case-based, or graph-based approaches (refer to Section A.5.2). Moreover, advancements in Deep Reinforcement Learning (DRL) techniques enable the agent to plan and reason with goals (refer to Section A.5.2). Additionally, the representation of the ontology, generated by Graph Neural Networks (GNN) or their variants [21, 60], can be integrated with DRL to further enhance the agent’s reasoning capabilities.

4. **An agent can predict and take action:** The agent estimates and predicts future states based on its model of historical data and observations. This prediction process can be carried out using weight schemes, probabilistic models, or reward-based models (see also Section A.5.2). Additionally, DRL techniques demonstrate that the agent can choose optimal actions in various scenarios by interactively imitating the behaviors of other agents and humans [26, 61–65].
5. **An agent can coordinate with other agents:** The development of Multi-Agent Systems (MAS) demonstrates the agent’s capability of communicating and coordinating with other agents to achieve a goal within some set of constraints [3, 5, 7, 19, 20, 66–69] (see also Section A.3).

To understand dynamic and complex environments, an autonomous agent must possess a combination of five capabilities: Sense, Learn, Reason, Predict, and Act. For example, the agent learns and reasons about the behaviors, goals, and beliefs of other agents or humans. This understanding enables the agent to predict and act within the dynamic environment through self-evaluation [70–72]. Note that the limitations of a single autonomous agent such as inefficiency, high cost, and unreliability [7, 20] become apparent when addressing distributed complex tasks such as microgrid control [73], resource allocation in cloud computing [74], as well as in the realms of computer networking and security, and other complex tasks [3]. Overcoming these limitations requires the implementation of a system that allocates task responsibilities among autonomous agents, enabling effective coordination and communication during task resolution. Additionally, in a specific task, a collective of autonomous agents needs to adapt to changes of the environment by constantly learning and updating their knowledge over time. This results in the growing attention to the field of context-aware multi-agent systems where agents comprehend their knowledge according to perceived contextual information to adapt to any situation and optimally solve allocated tasks and achieve the global goal. Application domains of such systems include autonomous navigation, ambient intelligence, supply chain management, Internet of Things (IoT), disaster relief management, energy efficiency & sustainability, digital assistance and education, and other complex problems (see also Section A.5).

Multi-Agent Systems (MAS) comprise multiple autonomous agents that interact within a shared environment, autonomously making decisions to accomplish tasks or address complex problems. An autonomous agent in MAS is endowed with initial knowledge about a given task and possesses its own set of goals. While engaged in task-solving, the agent interacts with other agents or the environment to perceive and comprehend information. It independently makes decisions based

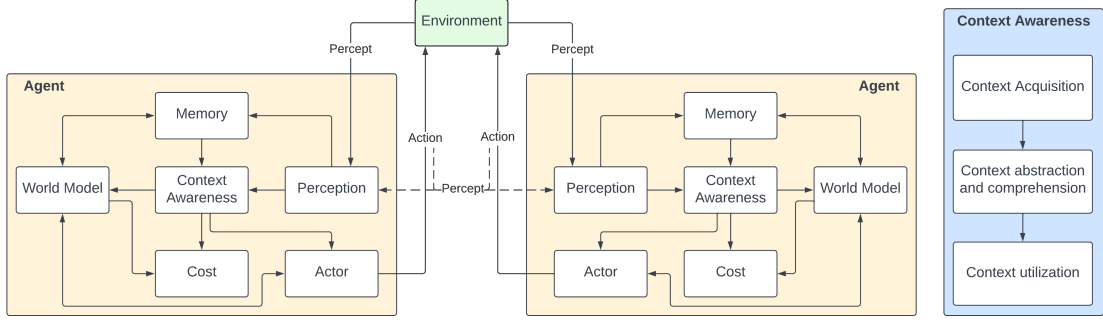


Figure A.1: An Overview of Context-Aware Multi-Agent Systems

on its objectives, existing knowledge, and observations, subsequently executing actions [4, 69]. Depending on the task’s characteristics, agents can collaborate or compete strategically to outperform opponents. These attributes confer flexibility and adaptability on agents in dynamic environments, making MAS suited for addressing complex problems. Furthermore, an autonomous agent forms its beliefs based on its knowledge and observations of the environment and other agents. Moreover, motivations for actions can vary among agents. Consequently, determining the optimal solution for a specific task necessitates effective communication, coordination, or competition in MAS. In cooperative settings, achieving consensus, and a shared agreement on a particular interest is imperative for autonomous agents. Conversely, in competitive scenarios, agents must analyze the behavior of opponents, anticipate negative outcomes, and devise strategies to address such challenges. It is worth noting that attaining consensus or understanding the behavior of other agents requires an autonomous agent to be aware of the context, including roles, organizational structure, situations, and location [5, 7]. Therefore, the integration of context-aware systems into multi-agent systems, as known as context-aware multi-agent systems, becomes essential.

Context-Aware Systems (CAS) pertain to systems that dynamically adapt to the environment by leveraging context to retrieve prominent information for tasks or problems. Context encompasses various perceptual elements such as people, location, physical or virtual objects, events, and other information delineating the situation of an autonomous agent in a specific environment [75]. The CAS process involves three primary stages: context acquisition, context abstraction and comprehension, and context utilization. [41, 42, 44]. Through this continuous process, autonomous agents acquire contextual cues to comprehend the current environmental state and undertake actions relevant to the situation. In the context of autonomous agents, contextual information encompasses task objectives, organizational structure, agent roles, and temporal aspects. Such context assists agents precisely retrieving relevant information for task accomplishment [70, 76, 77].

Considerable efforts have focused on either MAS or CAS. For instance, Dorri

et al.[3] conducted a survey outlining aspects of MAS, such as agent properties, organizational structures, coordination control, and communication. The survey also emphasized distinctions between MAS and related systems, such as object-oriented programming and expert systems (see Section A.3). Other surveys have delved into specific aspects of MAS, including consensus [7, 19, 20], communication [69], formation control [66–68], organizational structures [5], and deep reinforcement learning (DRL) for MAS [4, 8, 78–81]. On the other hand, surveys related to CAS can be categorized into two streams: fundamental overviews and domain-specific applications. Fundamental overviews highlight the general processes of CAS, context modeling approaches, and design principles for architecting CAS [39–41, 44, 45]. Research on domain-specific applications of CAS includes recommendation systems [43, 46, 49], Internet of Thing (IoT) [42], cloud and fog computing [47], and healthcare systems [48]. Despite the popularity of CAS and MAS, as well as the substantial research efforts in these fields, previous works have not considered all five capabilities of agents or framed them within the perspective of shared context awareness to achieve robustness and adaptability in dynamic environments. Therefore, there is a need for a coherent architecture, an organized taxonomy, and associated principles to build Context-Aware Multi-Agent Systems (CA-MAS). In this chapter, we review prior research that connects the five capabilities of agents and present a cohesive framework for constructing CA-MAS. Additionally, we highlight the existing challenges of CA-MAS and propose future directions to address them.

A.3 Multi-Agent Systems

An agent is an autonomous and computational entity (e.g., a software, a hardware component, or a combination of both) that operates within an environment to achieve specific tasks or goals [79, 81]. Furthermore, an agent is characterized by its ability to perceive its surroundings, make decisions based on available information, and execute actions to influence the environment. Such characteristics enable the sociability, autonomy, and adaptability of an agent in MAS. As highlighted by [82], intelligent properties of an autonomous agent encompass the following interconnected components:

1. **Perception:** The agent senses information and estimates the current environment state. Such information includes the visual, auditory, and other sensory input. For example, visual cues from images or videos allow the agent to plan pedestrians’ motion [83] or traffic scenarios [84, 85] to avoid collision. Furthermore, Oliveira *et al.* [86] proposed a MAS that proceeds

user interaction through audio and gestures to understand and addresses specific needs of elderly people. Moreover, acoustics, Radio Frequency (RF), optics, and magnetics are four general types of sensory data for underwater multi-agent systems [87].

2. **Memory:** The agent stores past, present, and predicted future states of the environment along with their intrinsic costs. Such information in the memory is accessible and modifiable by the world model to make temporal decisions or correct inconsistent and missing information about the current states of the world. On the other hand, the critic module learns from historical information to estimate their costs, supporting the agent in selecting a set of actions to optimally achieve its goal. Due to its limited memory capacity, the agent may not retain information for long periods. As it explores the environment, some data in its memory inevitably fades. Therefore, the experience replay mechanism proposed in [88] aims to assist the agent in re-learning forgotten information. However, not all forgotten information is necessarily worth re-learning, as it may not be immediately useful to the agent. Hence, Schaul *et al.* [89] proposed the prioritized experience replay method, which prioritizes information that frequently occurred with high rewards during the learning process.
3. **World Model:** The agent utilizes its current understanding of the world and past experiences to predict missing information and future states. Using the world model, the agent can anticipate the effects and outcomes of various actions to mitigate risks while searching for optimal strategies that include good actions and policies (see also Sections A.5.2 and A.5.2). However, it is important to acknowledge that gathering enough data to fully capture the environment’s behavior may not always be practical. Therefore, the predictions made by the world model could be inaccurate or uncertain.
4. **Actor:** The agent determines an optimal sequence of actions from proposed action plans to optimize the estimated future cost. After completing the optimization process, the agent selects an action from that sequence at a given time. Depending on the context, actions can be categorized into two types: discrete actions (e.g., controlling traffic lights, recommending from a finite set of items, or allocating energy to different areas) and continuous actions (e.g., physical control tasks, or adjusting heating, ventilation, and air conditioning settings in buildings based on weather conditions).
5. **Cost:** The agent assesses and evaluates its actions and knowledge based on the measured cost, which comprises two components: the intrinsic cost and

the critic. The intrinsic cost is predetermined by the agent’s behavior and remains constant throughout the learning process, providing insight into the nature of the agent’s behavior at different times. On the other hand, the critic is trainable during the learning process and aims to estimate long-term outcomes based on the current state of the world and the agent’s past experiences. An example of the critic component can be found in Actor-Critic methods [90–96]. These methods generally involve two stages: (1) the agent selects an action from the optimal action sequence using the *Actor* component; and (2) the *Critic* component estimates the cost of that action by approximating the expected future values of the reward. The agent’s overall objective is to take actions that minimize the average cost of the intrinsic cost and the critic.

6. Configurator (also known as the Context-Awareness component):

Depending on the situation presented by the environment, the agent adjusts configurations across its components to adapt to changes and progress towards its goals. For example, during the sensing process within the *Perception* component, the agent filters out irrelevant information for a given task. Additionally, the functions of the *Critic* component, learning weights of the world model, and policies can be dynamically configured to guide the agent towards specific sub-goals within a larger objective. While not all components are configurable in real-time, two strategies have been explored in the literature: online learning strategies and offline learning strategies [16, 97, 98].

The environment in a MAS serves as the operational space where agents operate and pursue goals. It consists of perceptual data and dynamic changes of state [3]. This environment provides the context guiding an agent’s decision-making process. As illustrated in Figure A.1, the task-solving process involves seven steps [3, 99]: perceiving information from the environment, storing it in memory, processing the information based on the task, comprehending task requirements, formulating a plan, executing actions, and learning from outcomes.

Multi-Agent Systems (MAS) comprise multiple autonomous agents interacting with each other and their environment to achieve goals. Four main advantages of using a MAS over a single-agent one include cost-effectiveness, reliability, scalability, and robustness in handling complex tasks [7, 20]. However, three main aspects of a MAS such as organizational structure, consensus, and formation control result in the complexity of MAS. Furthermore, three main features such as autonomy, communication, and society are three main features distinguishing between MAS, expert systems, and object-oriented programming (see also Table

A.1) [3]. For instance, each agent in MAS has its own goal(s) and can communicate with other agents in the network to make decisions and act on the environment, whereas objects in object-oriented programming or experts in an expert system are restricted by a set of pre-defined constraints of a particular problem. Depending on both organizational goals and agent-specific goals, relationships among agents are formed into one of the following categories: cooperation or competition. Cooperative MAS encompasses systems where agents collaborate with a shared objective of accomplishing common goals or addressing complex problems. Achieving this purpose necessitates organizing agents into groups with coordination, establishing communication protocols among them, and reaching consensus agreements [7, 100, 101]. On the other hand, competitive MAS characterizes systems in which agents operate with the primary goal of optimizing their individual objectives. To attain this, an agent engages in strategic decision-making, with the aim of outperforming or gaining advantages over other agents [102].

Table A.1: The difference between object-oriented programming, expert systems and multi-agent systems [3]

Characteristics	Object-oriented programming	Expert Systems	Multi-agent systems
Structure	Classes and objects	Knowledge base and inference engine	Agents and their interactions
Purpose	Modeling entities and behaviors	Emulating human expert decision-making	Autonomous, coordinated problem-solving
Communication	Method calls and message passing	Rule-based or logic-based reasoning	Interaction protocols and negotiation
Learning ability	None: Static code	Low: Limited learning modules	High: Agents can learn and adapt autonomously
Environment	Static, predefined environments	Static, predefined environments	Dynamic, open environments

An organizational structure, encompassing roles, relationships, and authority, plays a pivotal role in simplifying agents' models, controlling agents' behavior, and reducing uncertainty. Additionally, it facilitates the tracking and establishment of effective communication among agents when solving complex tasks. As shown in Figure A.2, ten organizational structures, such as (1) flat, (2) hierarchies, (3) holarchies, (4) coalitions, (5) teams, (6) congregations, (7) societies, (8) federations, (9) markets, and (10) matrix, have been identified [5]. The selection of an organizational structure depends on characteristics and agents' capabilities (see also Table A.2). Additionally, the combination of multiple organizational structures has been applied to existing multi-agent systems [5, 103]. Aside from organizational structures, three mechanisms that agents form an organization include dynamic reorganization [104–108], self-organization [108–111], or emergence

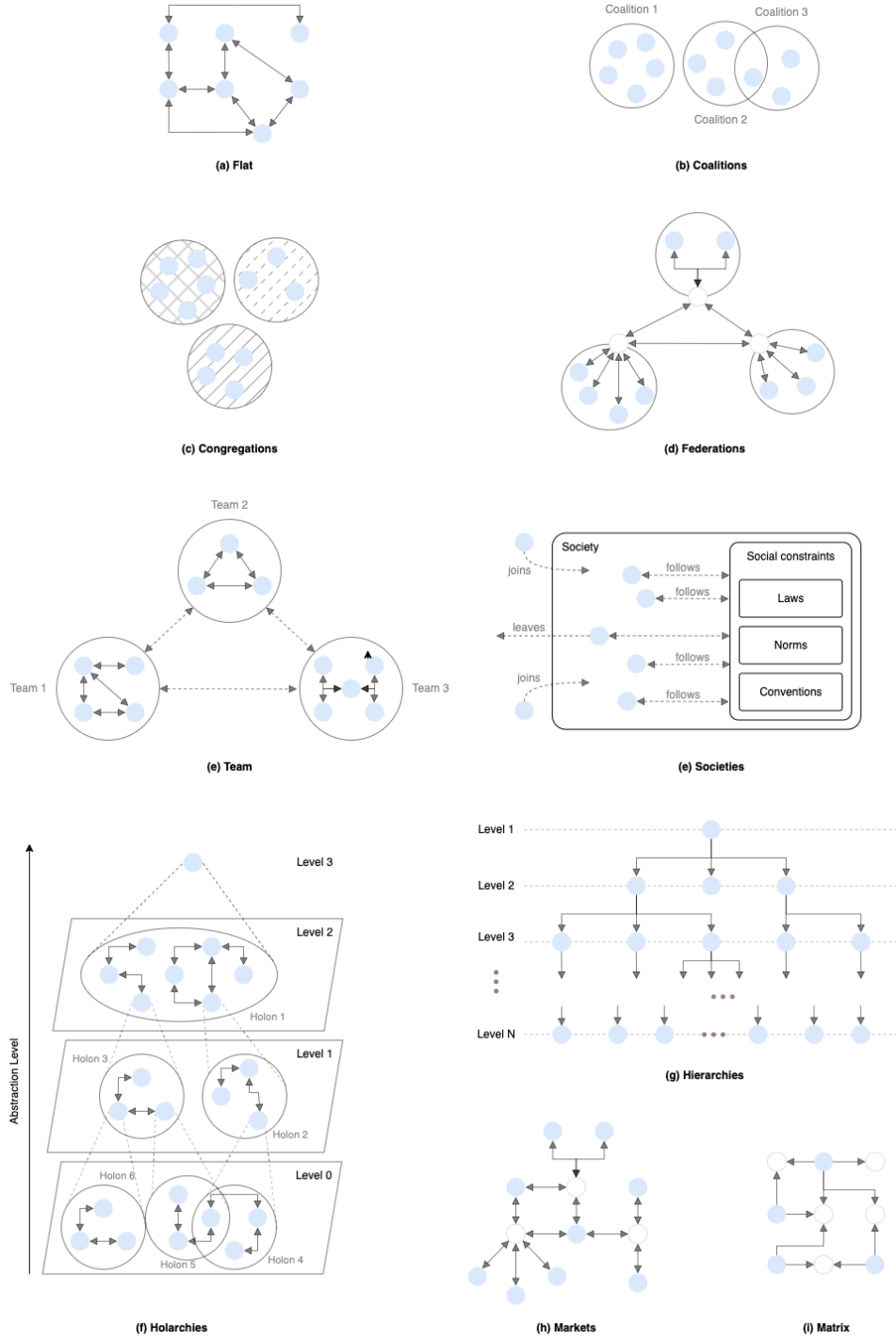


Figure A.2: Ten organizational structures of multi-agent systems. A blue circle symbolizes an agent, with its role depending on the organizational structure. For instance, in the federation structure (d), agents fulfill two distinct roles: (1) normal agents depicted as blue circles, and (2) federated agents shown as outlined circles. Similarly, in the market structure (h), there are two roles: buyers represented by blue circles and sellers by outlined circles. In the matrix structure (i), managers are represented as blue circles while workers are shown as outlined circles.

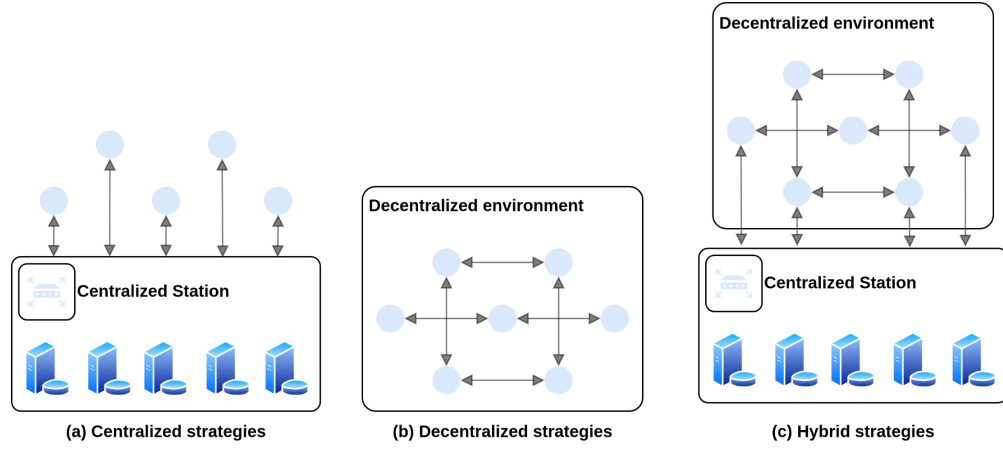


Figure A.3: Coordination strategies between agents. A blue circle represents an agent.

[110–114]. The selection of each mechanism depends on the following factors: (1) system demand, such as organizational goals and agent-specific goals; (2) external demand, such as changes of both environment and human intervention; (3) agents’ awareness of the existence of organization; and (4) complex and dynamic situations yield by the environment [108]. Once an organization has been formed, the coordination of agents can be operated through decentralized strategies, centralized strategies or hybrid strategies (see also Table A.3) [68, 115–117]. The selection of each of these strategies depends on four factors, such as real-time performance, scalability of system structure, robustness of decision-making, and solution optimality [68, 115]. The diversity of agents, inferred from differences in knowledge, capability, characteristics, behaviors, and other contextual information, represents a crucial dimension of agent organization. [118] demonstrated the impact of diversity in agents’ capabilities and behaviors on a holonic organization in MAS. Moreover, [119] investigated a technique focusing on behavioral diversity in MAS, aiming to control the trade-off between the heterogeneous behaviors of an agent resulting from knowledge exploration and the homogeneous behaviors obtained through knowledge sharing in multi-agent coordination tasks. Just as behavioral diversity strengthens agent performance, environmental diversity enhances the agent’s generalization capability in MAS [120].

Table A.2: Ten organizational structures of multi-agent systems. Each organizational structure is distinguished by the following characteristics: information flow, relationships among agents, and goals of the organization.

Organizational structure	Characteristics	Description
--------------------------	-----------------	-------------

Flat	Information Flow	Agents have access to information, reducing reliance on a central authority for dissemination. Communication among agents tends to adopt a peer-to-peer or distributed approach, facilitating sharing and collaboration across the network. [3, 5, 121]
	Relationships among agents	Agents maintain egalitarian relationships. Interactions among these agents involve cooperation, negotiation, and collaboration. [3, 5]
	Goals of the organization	The organization may prioritize flexibility and responsiveness to changes in the environment. [3, 5, 121]
Hierarchies	Information Flow	Information flows top-down from a central authority, with formalized communication channels and well-defined reporting relationships. Lower-level agents may have limited access to information beyond their immediate responsibilities, relying on superiors for guidance and updates. [5]
	Relationships among agents	Agents' relationships have lines of authority and subordination, with each agent having roles and responsibilities. Interactions are governed by rules. For instance, superiors make decisions, and subordinates follow directives. This dynamic is directive, with superiors guiding and instructing while subordinates execute tasks. [5]
	Goals of the organization	The organization emphasizes efficiency, control, and standardization for consistency across levels. Decision-making authority is centralized at the top, with lower-level agents primarily responsible for implementation rather than significant involvement in decision-making. [5]
Holarchies [122]	Information Flow	The flow includes top-down guidance and bottom-up feedback, enabling adaptability and self-organization within each holon. While overarching goals may exist, lateral communication and feedback loops occur between holons at the same level and across different levels. Agents within each holon share information to achieve collective goals and exchange information with holons at other levels. [5, 123, 124]

	Relationships among agents	Agents interact vertically (with higher and lower levels) and horizontally (with peers). There's a balance between autonomy and integration, allowing agents to self-organize within holons while aligning actions with larger goals. Functional hierarchies may form within holons for coordination, alongside cross-functional collaborations across holons. [5, 124, 125]
	Goals of the organization	Organizational goals are emergent and dynamic, shaped by interactions among holons at different levels. Each holon's goals and strategies reflect its context, capabilities, and relationships. The organization values adaptability and resilience, fostering continuous learning and evolution within the holarchy. [5, 124, 125]
Coalitions	Information Flow	Communication occurs within and between coalitions to coordinate activities and decision-making internally and externally. [5, 126, 127]
	Relationships among agents	Agents collaborate closely, pooling resources to address shared goals. There is a balance between autonomy and alignment, with agents working toward collective goals while retaining some independence. [5, 128, 129]
	Goals of the organization	Agents focus on achieving specific objectives or addressing common goals. The organization reflects collective needs and priorities, driven by a commitment to advancing mutual interests. A coalition is ceased once organizational needs no longer exist. Hence, flexibility and adaptability are important within a coalition structure. [5, 127–129]
Teams	Information Flow	The flow may be both vertical and horizontal, with communication occurring between team members, team leaders, and external entities. [5, 130]
	Relationships among agents	Members work closely, leveraging strengths to achieve shared objectives with trust and mutual respect. Clear roles and responsibilities ensure effective contributions to the team's efforts. [5, 131, 132]
	Goals of the organization	The team structure aims to establish authority for decision-making, fostering flexibility to adapt to changing circumstances, and enhancing collaboration among agents to work effectively towards shared objectives. [5, 131]
Congregations	Information Flow	An agent only exchange information with other agents in the same congregation. [5, 133]

	Relationships among agents	Agents in a congregation are cooperative. They establish trust and mutual understanding to facilitate teamwork and conflict resolution. [5, 133]
	Goals of the organization	A congregation aims find suitable collaborators for a specific problem by grouping agents with similar characteristics. This coupling of agents helps maximize their utility or reliability. [5, 134, 135]
Societies	Information Flow	Agents exchange information within a society. Information flows through broadcasting or decentralized peer-to-peer networks. The complexity of this flow depends on the society's size, interaction density among agents, and the level of centralization in information dissemination. [5]
	Relationships among agents	A society is an open system. Agents' interactions are formalized using contracts. They form social networks, hierarchies, alliances, or affiliations based on shared interests or organizational goals. [5, 136]
	Goals of the organization	The society structure aims to guide agents' behaviors and interactions through the collective objectives, shared values and social constraints, such as social laws, norms or conventions. [5, 136]
Federations	Information Flow	Agents or sub-groups have local information but share relevant data to achieve common objectives. Information flows through agent hubs or peer-to-peer networks. A coordinator agent integrates information, resolves conflicts, establishes the information flow among federations, and ensures consistency in shared information. [5]
	Relationships among agents	Agents within a federation operate independently but collaborate on shared goals, often characterized by negotiation and cooperation. [5]
	Goals of the organization	The federation structure aims to optimize system performance and achieve collective tasks which all agents work towards. Furthermore, while the structure supports flexibility and adaptability in goal setting and pursuit, it ensures local actions of individuals contribute to global objectives. [5]
Markets	Information Flow	Agents exchange price signals, bids, asks and other market information to facilitate market contracts. Information flows through peer-to-peer networks. Negotiation protocols and pricing mechanisms aim to coordinate agents' actions and allocate resources or tasks [5]

	Relationships among agents	A market is an open system. Agents in a market have buyer-seller relationships. They seek trading partners and negotiate or compete with other agents to seek out the most cost-effective solutions. Their interactions are formalized using contracts. [5, 134]
	Goals of the organization	A market creates a competitive environment that fosters cost-effective solutions for complex problems and maximizes agent-specific goals. The market structure enables the flexibility of the system in utilizing available resources to meet market demand. [5, 134]
Matrix	Information Flow	An agent reports to many managers and communicates with many peers. [5]
	Relationships among agents	Agents have dual reporting lines to both functional and project managers, fostering cross-functional relationships. Interactions are governed by formal rules. [5]
	Goals of the organization	The structure specify how multiple authorities influence the behaviours of agents or agent groups. This enhances organizational flexibility and responsiveness to changing conditions. [5]

Once an organizational structure is established, achieving consensus, and mutual agreement on a common value or state becomes essential for enabling information exchange among agents. This, in turn, requires an agent to formulate its Belief-Desire-Intention (BDI) model for coordination when addressing complex problems [137–141]. *Beliefs* represent the knowledge the agent has about the world, *Desires* are the goals or objectives the agent wants to achieve, and *Intentions* are the plans and actions the agent commits to in order to fulfill its desires. Once the BDI model is established in each agent, they begin communicating with one another. This introduces the dynamic state of communication for each agent. To achieve consensus among the agents, Amirkhani and Barshooi [7] outlined three categories of dynamic models: the single-integrator dynamic model, the double-integrator dynamic model, and the high-order dynamic model. Furthermore, two primary challenges in agent communication encompass data flow issues, such as data continuity and communication delays, and data quality concerns, including outliers, and missing, or incomplete information [19]. To overcome these challenges, the literature has explored nine communication protocols, including (1) leader-follower consensus, (2) group/cluster consensus, (3) scaled consensus, (4) finite-time consensus, (5) bipartite consensus, (6) sampled-data consensus, (7) quantized consensus, (8) network-based consensus, and (9) the combination of consensus with tracking mechanisms [7, 19, 20]. Additionally,

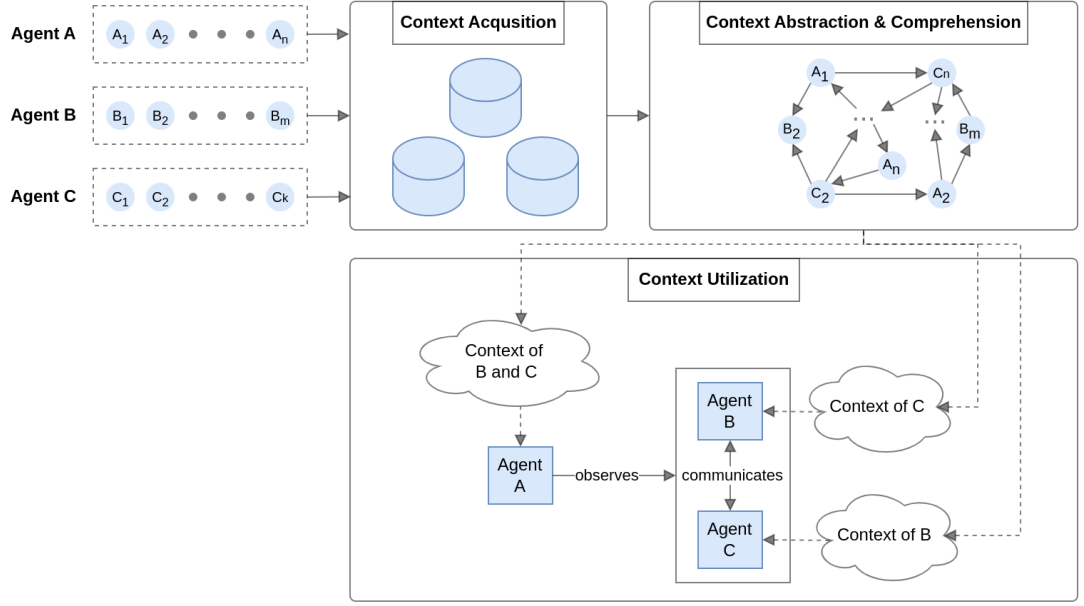


Figure A.4: An general process of context-aware systems. A blue circle represents sensed information from an agent. During the context utilization stage, one possible scenario is where Agent A observes communication between Agents B and C. In this situation, Agent A observes and understands the communication between Agents B and C by retrieving their contexts and relationships. Similarly, Agents B and C understand each other's information by retrieving each other's contexts.

optimizing the frequency of information transactions and computing resources per transaction is crucial to enhance communication flow for efficient coordination. To achieve this goal, existing trigger mechanisms controlling consensus can be divided into two groups: event-based consensus control and time-based consensus control [19, 20]. The former triggers communication based on situations exceeding a pre-defined threshold or changes of state, while the latter triggers communication periodically based on a pre-defined time interval or selected data samples. Moreover, while reaching consensus among agents, noise and uncertainties may affect both system performance and agent-specific performance. To address this challenge and ensure data quality, consensus control strategies such as robust consensus control and adaptive consensus control have been investigated. These techniques can be further categorized into two groups: feedback control and feed-forward control [7].

A.4 Context-Aware Systems

Context encompasses various types of information, such as people, location, physical or virtual objects, events, time, and other data that can be employed to introduce different dimensions of a situation or conceptual information regarding

specific circumstances [75]. Additionally, five key properties characterize context: type, value, time when sensed, source where the information was gathered, and confidence in information accuracy [39]. Furthermore, context can be categorized into two groups: intrinsic context and extrinsic context. In the context of MAS, the former specifies the internal factors of an agent (e.g., goal, task, behavior, belief, knowledge, etc.), while the latter focuses on external factors, such as the environment, situation, social influence, and more. Existing context modeling techniques in the literature fall into six categories: (1) key-value models, (2) markup schema models, (3) graphical models, (4) object-oriented models, (5) logical-based models, and (6) ontology-based models [142]. The selection of a context modeling technique depends on four factors, such as complexity, scalability, interoperability and reasoning assistance (see also Table A.3) [142, 143]. For instance, key-value models are suitable for situations where simplicity is crucial, although they lack two following capabilities: capturing the relationships within context and reasoning. To address these challenges, other context modeling techniques are employed. It is worth noting that ontology-based context models are widely utilized due to their capability for semantic reasoning and representation of context relationships through knowledge graphs [39, 144–146]. However, challenges associated with ontology-based context modeling include lack of generality [145], the difficulty in understanding ontology complexity and the substantial cost of maintaining ontologies [40]. To address the challenge of generality, Gu *et al.* [145] proposed a context modeling technique that leverages the Web Ontology Language (OWL) to enhance semantic context representation. Additionally, Horrocks *et al.* [147] introduced the InstanceStore system, which employs relational databases for semantic indexing and optimizes the performance of ontology-based reasoning. While modeling or reasoning about context, it is crucial to consider three aspects: (1) the quality of context, including accuracy and completeness of information; (2) relationships between context elements; and (3) the flow of context, such as time-invariant context, time-variant context, and consistency in context switching [40, 142, 148].

Table A.3: Characteristics and use cases of six categories of context modeling techniques. The rating scale includes Low, Medium and High. The best configuration features lower complexity, higher scalability, higher interoperability, and higher reasoning assistance. [142, 143]

Context Modeling	Complexity	Scalability	Interoperability	Reasoning Assistance	Use Cases
Key-value models	Low	High	Low	None	Simple applications, basic context storage

Markup schema models	Medium	Medium	Medium	Medium	Hierarchical context, structured data
Graphical models	Medium	High	Medium	Medium	Complex relationships, networked contexts
Object-oriented models	Medium	High	Medium	Medium	Encapsulated context, object-oriented applications
Logical-based models	High	Medium	High	High	Context reasoning, rule-based systems
Ontology-based models	High	Medium	High	High	Semantic interoperability, advanced reasoning, and inference

Context-Aware Systems (CAS) are autonomous systems that leverage context to dynamically adapt to situations and retrieve relevant information for tasks or problems. Depending on the level of agent autonomy and interactions among agents, there are two types of context awareness: (1) passive context awareness, where an agent constantly produces observations from the environment that can be utilized by other agents; and (2) active context awareness, where an agent continuously and autonomously acts based on its observations from the environment [42]. As illustrated in Figure A.4, the general process of CAS consists of three stages: (1) context acquisition, (2) context abstraction and comprehension, and (3) context utilization [41, 42]. Context acquisition involves the process of sensing and storing information. In the context of MAS, an agent can sense information from its knowledge, other agents, or the environment. It is noteworthy that an agent may sense multiple types of context, periodically coming from various sources [39, 40, 42]. Based on the architectural style, Figure A.5 shows three categories of context-aware systems [41]:

- **Stand-alone CAS:** An agent retrieves information from sources of context and store such contextual information independently.
- **Centralized CAS:** Contextual information is collected and stored in the centralized server. Agents can exchange the information with the server.
- **Decentralized CAS:** This architecture does not rely on a central server; instead, it comprises multiple interconnected agents, each functioning as a stand-alone CAS and exchanging the information with others in the network.

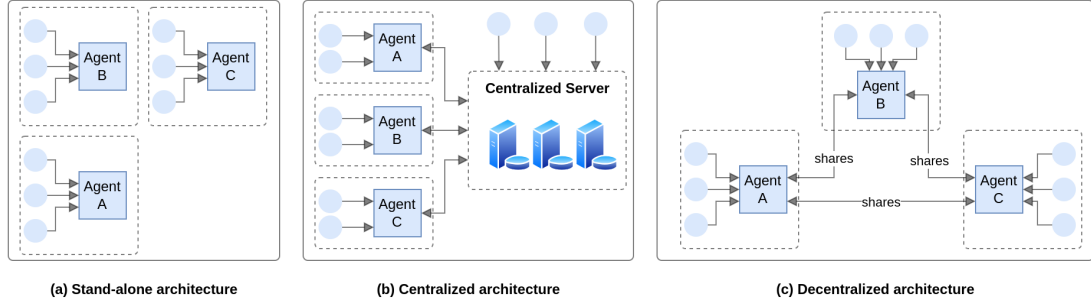


Figure A.5: Architectures of context-aware systems. A blue circle represents the information generated by diverse sensors.

Furthermore, it is crucial to recognize that contextual information obtained from agents constitutes raw data. As a result, this data requires pre-processing and encapsulation to enable the interpretation of its semantics for a specific task or problem, a procedure commonly referred to as context abstraction and comprehension. This process involves techniques that can be classified into two types: context modeling and context reasoning. Context reasoning techniques, in particular, provide agents with the capability to infer knowledge from imperfect context and uncertainty. These techniques can be further categorized into six groups: (1) supervised learning, (2) unsupervised learning, (3) rule-based algorithms, (4) fuzzy logic, (5) ontology-based reasoning, and (6) probabilistic reasoning [42, 45, 143]. During the context utilization stage, an agent employs contextual information through its triggering mechanisms [42, 44, 149, 150]. Additionally, this information is distributed to other agents, guiding their decisions in solving complex tasks.

A.5 Context-Aware Multi-agent Systems

Addressing time-varying uncertainties, such as the dynamics of an environment and the non-linearity of autonomous agents, poses a significant challenge in MAS. Overcoming this challenge entails agents being aware of changes in the environment or behavioral shifts of other agents within the system and consistently updating their beliefs based on observations. Achieving this goal requires the integration of CAS and MAS, resulting in what is termed *Context-Aware Multi-Agent Systems (CA-MAS)*.

Table A.4: Existing CA-MAS approaches across eight application domains, including autonomous navigation, ambient intelligence, supply chain management, internet of things, disaster relief management, energy efficiency and sustainability, digital assistance, and education. Four characteristics of CA-MAS include organizational structure, consensus protocol, context modeling, and reasoning and planning approach.

Application Domains	Characteristics	References
Autonomous navigation	Organizational Structure	Flat [83–85, 151–157]
	Consensus Protocol	Sampled-data consensus [83–85, 151–157]
	Context Modeling	Key-value models [83–85, 151–157]
	Reasoning and Planning	Goal-oriented algorithms [83–85, 151–157]
Ambient intelligence	Organizational Structure	Teams [158], coalitions [159]
	Consensus Protocol	Group/cluster consensus [158, 159]
	Context Modeling	Ontology-based models [158], key-value models [159]
	Reasoning and Planning	Graph-based algorithms [158], rule-based algorithms [159]
Supply chain management	Organizational Structure	Markets [160], holarchies [161]
	Consensus Protocol	Sampled-data consensus [160], group/cluster consensus [161]
	Context Modeling	Key-value models [160], ontology-based models [161]
	Reasoning and Planning	Case-based techniques [160], hybrid (rule-based algorithms and case-based techniques) [161]
Internet of things	Organizational Structure	Teams [162], flat [163], coalitions [164]
	Consensus Protocol	Hybrid (group/cluster consensus and sampled-data consensus) [162], sampled-data consensus [163], group/cluster consensus [164]
	Context Modeling	Key-value models [162], ontology-based models [163, 164]
	Reasoning and Planning	Goal-oriented algorithms [162, 163], rule-based algorithms [164]
Disaster relief management	Organizational Structure	Hybrid (teams and holarchies) [140, 165, 166]
	Consensus Protocol	Leader-follower consensus [140], hybrid (leader-follower consensus and sampled-data consensus) [165], group/cluster consensus [166]
	Context Modeling	Object-oriented models [140], key-value models [165], logic-based models [166]

	Reasoning and Planning	Hybrid (rule-based algorithms and case-based techniques) [140], goal-oriented algorithms [165], graph-based algorithms [166]
Energy efficiency and sustainability	Organizational Structure	Holarchies [167], teams [168], hierarchies [169]
	Consensus Protocol	Leader-follower consensus [167], group/cluster consensus [168], hybrid (leader-follower consensus and sampled-based consensus) [169]
	Context Modeling	Key-value models [167, 169], object-oriented models [168]
	Reasoning and Planning	Goal-oriented algorithms [167, 169], rule-based algorithms [168]
Digital assistance	Organizational Structure	Flat [170], teams [171]
	Consensus Protocol	Sampled-data consensus [170], group/cluster consensus [171]
	Context Modeling	Ontology-based models [170, 171]
	Reasoning and Planning	Graph-based algorithms [170], fuzzy rule-based algorithms [171]
Education	Organizational Structure	Hierarchies [172]
	Consensus Protocol	Group/cluster consensus [172]
	Context Modeling	Ontology-based models [172]
	Reasoning and Planning	Case-based techniques [172]

A.5.1 Awareness in MAS

Agent-based awareness techniques can be categorized into three groups [99]: (1) Active perception [173–175], where an agent chooses what, when, where, and how to perceive, and can decide whether or not act based on its observations in an environment; (2) situation awareness [176, 177], which involves reasoning about the current event and predicting future events as additional capabilities for agents to perform an optimal set of actions in a particular situation; and (3) context awareness, where context is utilized as the semantics of an event or a situation to enhance the agent’s comprehension (see also Section A.4). In this survey, we focus on the context-aware multi-agent systems.

In MAS, an agent is considered context-aware when its behavior and actions adapt to the context it senses. Additionally, an agent perceives both intrinsic and extrinsic context. The intrinsic context involves an agent’s goals, roles, historical data (e.g., knowledge, previous actions), intent, and observations. Extrinsic context is categorized into user-specific context (e.g., location, preferences, calen-

dar, weather, behavior from the user), agent-specific context (knowledge or intent sharing among agents, observations about roles or behavior of other agents), and system-specific context (system requirements, policies, organizational structures, and communication protocols followed by agents for optimal problem-solving in specific situations). Additionally, Picard *et al.* [108] emphasized the impact of agents' organizational awareness on their adaptation, flexibility, and coordination. Table A.4 illustrates that various organizational structures, such as flat, teams, coalitions, holarchies, markets, and hierarchies (see also Section A.3), are primarily utilized to arrange CA-MAS across diverse application domains. Within these structures, the consensus protocols among agents include sampled-data consensus, group/cluster consensus, and leader-follow consensus. Agent-specific context involves knowledge or intent sharing among agents and an agent's observations about roles or behavior of other agents. It is noteworthy that agents in CA-MAS can be either communicating agents or non-communicating agents [83, 178]. A communicating agent exchanges information or intent with other agents in the system via the communication protocol. On the other hand, a non-communicating agent does not exchange information with other agents via a communication protocol; however, its behavior can be sensed or predicted based on its actions. For instance, in a competitive MAS setting, an agent performs actions based on observations and predictions about the actions of opponents [179]. Another example is in autonomous driving agents, where, to avoid collisions on the road, agents must act based on observations [83, 85, 157, 178].

A.5.2 The General Process of CA-MAS

The general process of CA-MAS comprises five phases: Sense-Learn-Reason-Predict-Act [85, 180–182]. Details of each phase are specified below.

Sense

An agent in MAS gathers contextual information from its sensors, detects context patterns, and handles temporal changes of context (see also Figure A.4). Context can be sensed from a stand-alone node, a centralized server, or through communication and interaction among agents (see also Figure A.5). As the context is sensed from multiple sources, relationships between contexts may exist. These relationships are, then, represented as the context graph denoted as $G = (V, E)$ where V is a set of context and the edge E is a subset of $V \times V$. Note that there are two types of G : (1) a directed graph where a directed edge is represented as an ordered pair of context c_i and c_j and denoted by (c_i, c_j) or $c_i \rightarrow c_j$; and (2) an undirected graph where an edge between c_i and c_j is denoted by $(c_i, c_j) \in E$ and

Flow dependency

Learn		Reason	Predict & Act
Context Modeling	Representation Learning		
Key-value Models	Weighting Mechanisms	Graph-based Reasoning	Probabilistic Models
	Learning-based Mechanisms	Goal-oriented Reasoning	Weighting Schemes
			Reward-based Models
		Rule-based Reasoning	
		Case-based Reasoning	
Logic-based Models	Weighting Mechanisms	Graph-based Reasoning	Probabilistic Models
Object-oriented Models		Rule-based Reasoning	Weighting Schemes
		Case-based Reasoning	
Ontology-based Models		Rule-based Reasoning	
		Case-based Reasoning	
		Graph-based Reasoning	
		Goal-oriented Reasoning	Reward-based Models

 Not Applicable
  Method or Technique

Figure A.6: The flow dependency of techniques in Learn, Reason, Predict and Act. Note that the reading order is from left to right.

interprets the relationship between those context. Additionally, a context pattern is defined as a subgraph of G and denoted as $H = (V', E')$ where $V' \subseteq V$ and $E' \subseteq E$. This pattern supports an agent to make decisions in a specific situation. Furthermore, the constant change in the environment increases the number of contexts, followed by the number connections in the context graph as:

$$N_E = \frac{N_V \times (N_V - 1)}{2} \quad (\text{A.1})$$

where N_E is the number of connections, and N_V is the number of contexts on the graph. This leads to a high complexity in the search space, even though an agent might only need to concentrate on a specific subset of the context to solve a task. To overcome the challenge of managing context changes in dynamic environments, Julien *et al.* [183] applied the Network Abstractions model [184]. The model estimates the weighted distance, denoted as $d(c_r, c_k)$, between the reference context ($c_r \in V$) and another context ($c_k \in V$) by calculating the shortest path between them as [184]:

$$d(c_r, c_k) = \min (d(c_r, c_{k-1}) + w(c_{k-1}, c_k)) \quad (\text{A.2})$$

where $d(c_r, c_{k-1})$ is the minimum weighted distance of all possible paths between c_r and c_{k-1} , and $w(c_{k-1}, c_k)$ is the cost of the edge between c_{k-1} and c_k . The cost function of the model is, then, estimated by determining both $d(c_r, c_k)$ and the number of hops between c_r and c_k as [184]:

$$u = (D_{\max}, H_{\max}, V, h) \quad (\text{A.3})$$

where $D_{\max}$ represents the maximum distance between c_r and c_k , $H_{\max}$ indicates the maximum number of consecutive hops during which $D_{\max}$ has not changed, V is the distance vector capturing the displacement and direction between c_r and c_k , and $h \leq H_{\max}$ denotes the count of consecutive hops for which $D_{\max}$ has remained unchanged. When an agent navigates in the dynamic environment, c_r is shifted. This results in the update of Equations A.2 and A.3. Hence, the model helps the agent handle context changes from the dynamics of the environment.

Learn

An agent learns representations of information along with the sensed context for a specific task. This procedure involves two stages: context modeling (refer to Section A.4) and representation learning, which includes weighting mechanisms and learning-based approaches.

Key-value models, object-oriented models, logic-based models, and ontology-based models have been widely used in existing CA-MAS across application domains (see also Table A.4). Due to the simplicity and scalability of the key-value model, it has been primarily utilized in autonomous navigation to manage various types of contextual information: (1) the state representing the environment’s configuration, denoted as $s \in \mathcal{S}$; (2) the action of the agent, denoted as $a \in \mathcal{A}$; and (3) the observation sensed by the agent, denoted as $o \in \mathcal{O}$ [83–85, 151–153, 155–157]. Similarly, Nadi and Edrisi [165] employed key-value pairs to model the states (s) and actions (a) of agents within the field of disaster relief management. Furthermore, the simplicity of key-value pairs facilitates the vectorization of context, represented by $\vec{c} = (c_1, c_2, \dots, c_n)$, where $\vec{c}$ is a vector encompassing contextual information. This approach has been applied in various domains such as autonomous navigation [154], supply chain management [160], the Internet of Things [162], and energy efficiency & sustainability [167, 169]. Furthermore, Wan and Alagar [185] utilized key-value models to capture both internal and external contexts of agents, associating these contexts with trust attributes, such as safety, security, reliability, and availability, to support the modeling of trust among agents forming a coalition or team. Moreover, Haiouni et al. [159] applied key-value pairs to model the world state in a set-theoretic planning domain [186]. Apart from key-value models, CA-MAS which employs an object-oriented model to represent contexts such as weather forecast data and points of interest data was proposed by [168] to optimize planning costs and resource utilization for electric vehicle fleet routing in urban operations. Furthermore, Yusuf and Baber [166] utilized logic-based models to capture the relationships between various contextual information, including types of fuel, fuel conditions, the existence of fire, and location within the domain of disaster relief management. When the semantics of contextual information are required to model complex situations or problems, ontology-based models are employed [158, 161, 163, 164, 170–172].

CA-MAS with weighting mechanisms are categorized into two groups: statistical approaches [154, 160, 162, 167, 169] and graph-based approaches [166, 183]. Furthermore, the weighting mechanism encompasses three components: (1) vector representation, where context values are transformed into a machine-readable format and stored in a vector; (2) weight estimation, where weights are measured by aggregating between context vectors and other vectors representing other entities in the system; and (3) weight utilization, where estimated weights are used for goal-oriented optimization. Kwon and Sadeh [160] proposed an approach that estimates the weight of an agent by aggregating between the vector of contextual information and the utility vector of that agent for automating negotiation tasks between buyers and sellers and maximize their payoff. Furthermore, Lee

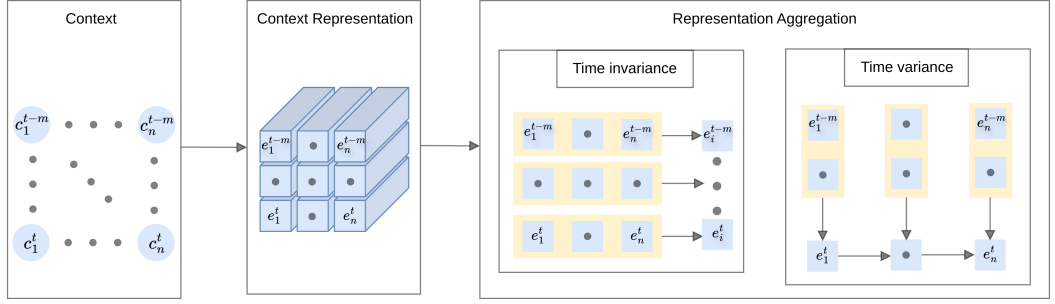


Figure A.7: The general process of learning-based approach in existing CA-MAS. n is a number of agents, t is the current time, and $t - m$ indicates m steps before t . Note that there are two main settings of aggregation: (1) time invariance illustrating the aggregation process is not influenced by time; and (2) time variance showing historical data of each agent is considered.

and Kabir [154] proposed the density-aware MAS that utilizes agents' density distribution as context vectors to optimize the spatial exploration of a team of robots. Moreover, domain-specific contextual information, such as utility usage [167, 169] and mobile usage [162], gathered from agents, is stored in a vector for weight estimation in recommendation systems. Aside from statistical approaches, Yusuf and Baber [166] represented each type of contextual information collected by unmanned aerial vehicles (UAVs) as a node within a Bayesian Belief Network (BBN) and calculated their joint probability as [187]:

$$p(c_1, c_2, \dots, c_n) = \prod_{i=1}^n p(c_i | B(c_i)) \quad (\text{A.4})$$

where c_i denotes a context node in the directed acyclic graph G (see also Section A.5.2), and $B(c_i)$ represents the set of parent nodes of c_i . Specifically, the joint probability of three contexts (c_1 , c_2 , and c_3), where c_1 and c_2 are parents of c_3 , is determined as follows:

$$\begin{aligned} p(c_1, c_2, c_3) &= p(c_1) \times p(c_2) \times p(c_3 | c_1, c_2) \\ \text{subject to } p(c_3 | c_1, c_2) &= \frac{p(c_1, c_2 | c_3) \times p(c_3)}{p(c_1, c_2)} \end{aligned} \quad (\text{A.5})$$

The probability $p(c_i | B(c_i))$ represents the likelihood of a context node given its parent nodes. Yusuf and Baber [166] assigned probabilities to each contextual information based on its likelihood of occurrence.

As the agent's observation space expands, the volume of context vectors and their dimensions increase, leading to the high-dimensional dilemma, also known as the curse of dimensionality [26, 188–191]. Furthermore, due to uncertainty and ambiguity, integrating the weighting scheme with goal-oriented reasoning frame-

works (see also Section A.5.2) becomes challenging. Overcoming these challenges necessitates the use of learning-based techniques for two reasons. First, contextual information can be represented in low-dimensional vector space. Second, learning parameters can be optimized according to a given goal or policy. Figure A.7 illustrates the overall process of representing contextual information in MAS using learning-based methods. The vector representation of time-invariant context, represented as e , can be determined as follows:

$$e = \phi(c, W_c) \quad (\text{A.6})$$

where ϕ represents the embedding function with a non-linear approximation, and W_c denotes the embedding weights. The selection of the embedding function is influenced by the modality of the context [51, 192]. Since an agent can perceive various contexts from other agents, dependencies between contexts may exist and can be calculated as follows:

$$e_i = \phi \left(\bigcup_{j=0}^N e_j, W_l \right) \quad (\text{A.7})$$

where $\bigcup$ indicates the combination of a set of embedding vectors, N is the number of contexts, i and j are the indices of c_i and c_j respectively, and W_l represents the embedding weights for aggregation. Moreover, the vector representation of time-variant context can be estimated as follows:

$$e_i^t = \phi \left(\bigcup_{j=0}^N e_j^{t-1}, W_k \right) \quad (\text{A.8})$$

where t and $t - 1$ represents the current state and the past state of the environment, respectively. An agent's observations from the traffic environment are represented using a Multi-Layer Perceptron (MLP) as follows: $e = \text{MLP}(o)$. Similarly, the CA-MAS model proposed by Fan et al. [155] employed an MLP with an extraction layer [193] to encode an agent's observations of obstacles in the multi-agent particle environment [11]. This method also utilized a 1D convolutional network to aggregate embedding vectors of an agent's observations and actions to select an optimal action. Additionally, in the CA-MAS model proposed by Qi and Zhu [151], YOLO [194] was used to detect visual objects in the simulated V-REP environment [195], aiding the agent's path planning activities. Everett et al. [83] formulated the agent's state as a sequence of information comprising two types: observable states and unobservable states, denoted by s^o and s^h , respectively. Specifically, s^o includes the agent's position, velocity, and radius, while s^h

includes the goal position, preferred speed, and orientation. Everett et al. [83] further aggregated the agent's state with those of other agents, represented using Long Short-Term Memory (LSTM) [196], as follows:

$$e = \phi \left(\bigcup (s, \text{LSTM}(\tilde{s}^o_1, \dots, \tilde{s}^o_n)), W_s \right) \quad (\text{A.9})$$

where e is the aggregated state embedding of the agent, W_s indicates the embedding weights of the aggregation, $\tilde{s}$ represents the other agent's state, and n is a number of agents. LSTM was also applied in the CA-MAS model proposed by Tao et al. [152] to represent both the agent's observations and those of other agents. Beyond features across agents at a specific time, this approach also incorporates historical information of successive events. These representations were then aggregated with the semantic representation of the latest observed motions, enabling the agent to learn both the spatial dependencies and temporal coherence of moving agents.

Pedestrian trajectories are also crucial for estimating uncertainty in autonomous driving. To achieve this goal, Katyal *et al.* [197] proposed a CA-MAS where the integration of pedestrian trajectories and agents' observations was represented via the Encoder-Decoder architecture consisting of stacked LSTM. Moreover, to help the agent learn the correlation between the control actions of other agents, Chen and Chaudhari [153] introduced a CA-MAS in autonomous navigation. They used an MLP to represent driver types and an Induced Set Attention Block (ISAB) from the Set Transformer [198] to encode the control actions of autonomous vehicles along with their correlations. Aside from using pedestrian trajectories as contextual cues, Xie *et al.* [85] utilized congestion patterns as context, which were represented by Graph Convolution Network integrated with Variational Auto-Encoder (GCN-VAE) [199, 200] for trajectory prediction. Furthermore, Wu *et al.* [157] proposed CA-MAS in the heterogeneous traffic where the agent needs to understand other agents' driving actions for making instant response. In this system, Gated Recurrent Unit (GRU) [201] was applied to represent agents' historical observations and driving actions, denoted by o_i^{t-1} and β , respectively. Additionally, Graph Attention Network (GAT) [202] was employed to extract the temporal relationship between o_i^t and β .

Reason

Agents analyze information or formulate a set of plans based on the sensed context to achieve their objectives. As shown in Table A.4, various reasoning models, including rule-based reasoning, case-based reasoning, graph-based reasoning, and goal-oriented reasoning have been employed.

Rule-based reasoning involves a predefined set of rules governing how agents respond to specific conditions or events. In the context of CA-MAS, Wan and Alagar [185] incorporated a set of rules to model trust, while Ferrando and Onaindi [203] utilized defeasible rules to formulate arguments supporting the planning process. To enhance the reasoning capabilities of agents in complex situations, Haiouni and Maamri [159] proposed the Hierarchical Colored Petri Net, an extension of the Petri Net [204], which applies agent hierarchies to the planning process in CA-MAS. In the domain of urban planning, Jelen *et al.* [168] implemented a rule-based routing algorithm using contextual information to identify urban parking lots or charging stations for electric vehicles. Meanwhile, Yousaf *et al.* [164] enhanced agent reasoning capabilities by applying rules to identified matching patterns in the context ontology. Furthermore, agents utilize their world model and intrinsic motivation (see also Section A.3) to formulate plans supporting their goals. To represent the mental states of agents, the BDI model is applied. Jakobson *et al.* [140] extended the traditional BDI model by adding the Plan module, which consists of context ontology and enhances agents' recognition of dynamic situations. An extension of rule-based reasoning is fuzzy logic, where agents can represent and process information with a certainty degree for a particular situation based on semantic rules. In this regard, Castellano *et al.* [171] employed a neuro-fuzzy network, representing fuzzy rules as a neural network architecture to map contextual information to specific situations.

Case-Based Reasoning (CBR) empowers agents to retrieve, reuse, revise, and retain cases that represent their past experiences. This aims to support agents to adapt and make decisions in similar situations [205]. Given the agent's contextual information, retrieving a set of all relevant cases can be formulated as:

$$f(c) = \{v \in V \mid g(c, v) \geq \alpha\} \quad (\text{A.10})$$

where f is the retrieval function that identifies relevant cases based on the current context c , v is a case in a set of all possible cases V , g is the similarity function that measures how closely a case v matches the current context c , and α is the similarity threshold. In CA-MAS proposed by Kwon and Sadeh [160], CBR is applied to retrieve similar cases based on contextual information during negotiation tasks. In the domain of multi-dimensional learning, Vladioiu and Constantinescu [172] employed CBR with agents' ontology to retrieve learning scenarios tailored to the user's interests. Furthermore, Fu and Fu [161] equipped agents with both rule-based reasoning and CBR, integrated on top of their context ontology. This integration allows agents to derive high-level business context from low-level contextual information for collaborative cost management in the

supply chain.

Graph-based reasoning endows agents with the capability to analyze complex interactions, dependencies, and patterns within graph structures. CA-MAS in [170] personalized recommendations for users by matching their contextual information with patterns identified within agents' context ontology. Agents in both [206] and [207] utilized meta-models, encompassing agents' contextual information, relationships among such information, and inter-agent relationships, for reasoning in specific situations. In another approach, Olaru *et al.* [158] introduced the Tri-Graph, a union of graphs representing restricted environments, contextual information, and agent topology. This graph captures context patterns for matching, filtering and reasoning about information. Aside from pattern matching algorithms, a graphical context model with probabilistic estimation such as Bayesian Belief Networks (BBN) have been applied to support agents in reasoning about fire spread in the field of disaster relief management [166].

In goal-oriented reasoning, agents prioritize and plan actions based on their goals. While other reasoning techniques that rely on pre-programmed conditions, patterns, or cases, agents with goal-oriented reasoning capability adapt to any situation to optimally achieve their goals. Additionally, every goal-oriented CA-MAS consists of an optimization process with a cost function. Riabchuk *et al.* [169] applied the mean squared error as the cost function that illustrate the accuracy of the typical energy consumption of operating a device in approximating the actual one:

$$\mathcal{L}^i = \frac{1}{n} \sum_d \sum_j \frac{1}{k} (y_{d,j}^i - \hat{y}_d^i)^\top (y_{d,j}^i - \hat{y}_d^i) \quad (\text{A.11})$$

where i is the index of the i^{th} device, n is the total number of uses of devices, k is the usage duration in hours, $\hat{y}_d^i$ is a typical energy consumption of the i^{th} device at usage date d , and $y_{d,j}^i$ is a true energy consumption of the i^{th} device at usage date d with index j . Similarly, Twardowski and Ryzko [162] employed the regularized square error to optimize the recommendation system of mobile usage based on user preferences as [208]:

$$\mathcal{L}(\theta) = \frac{1}{2} \sum_{(i,j,k) \in R} (y_{ijk} - \hat{y}_{ijk})^2 + \Omega(\theta) \quad (\text{A.12})$$

where i , j , and k represents the i^{th} user, the j^{th} item and the k^{th} context, respectively, and $\Omega(\theta)$ is the regularization term which control magnitude of learning parameters θ . Additionally, to find the best learning parameters, Stochastic Gradient Descent (SGD) [209] was applied [162]. Furthermore, as a student in [85], the agent is required to reason about congestion patterns for collision avoidance taught by the teacher. Specifically, Gaussian Mixture Model (GMM) [210] was

employed to estimate the representation of congestion patterns [85]:

$$Q(o) = \sum_i^{M_Q} \lambda_i q_i(o) \quad (\text{A.13})$$

where Q is the teacher's GMM, $q_i(o)$ is the i^{th} mixture component in a Gaussian distribution, λ_i is the mixture weight, and M_Q is the hyperparameters of the total number of mixtures. Additionally, the student's GMM is denoted as $P(o) = \sum_j^{M_P} \omega_j p_j(o)$ where M_P may differ from M_Q [85]. To support the reasoning procedure of a student, Xie *et al.* [85] aims to minimize the distance between $Q(o)$ and $P(o)$ as follows:

$$\min_{\{p_j\}, \alpha, \beta} \mathcal{L} = \sum_{i,j} \alpha_{i,j} \mathbb{D}_{\text{KL}}(p_j(o) || q_i(o)) + \mathbb{D}_{\text{KL}}(\alpha || \beta) \quad (\text{A.14})$$

where $\mathcal{L}$ is L1 loss term, α and β are obtained by decomposing the mixture weights $\omega_j = \sum_i^{M_Q} \alpha_{ij}$ and $\lambda_i = \sum_j^{M_P} \beta_{ij}$, respectively, and $\mathbb{D}_{\text{KL}}$ is the KL-divergence between two pattern distributions.

Moreover, reasoning with feedback, which is from both the environment and other agents, supports an agent to adapt to situations. To attain this, Reinforcement Learning (RL) techniques are applied. In RL, an agent aims to maximize its rewards by selecting an optimal action based on a policy at a particular state [16]. In conjunction to three properties in Section A.5.2, there are two additional properties in Multi-Agent Reinforcement Learning (MARL) determined by a fully observable Markov Decision Process (MDP) [211]. With N agents, these properties include $\mathcal{S}$, $\mathcal{A} = \{\mathcal{A}^1, \dots, \mathcal{A}^N\}$, $\mathcal{O} = \{\mathcal{O}^1, \dots, \mathcal{O}^N\}$, $\mathcal{P}$ denoting the transition probability between states, $\mathcal{R} = \{\mathcal{R}^1, \dots, \mathcal{R}^N\}$ representing a set of agent's rewards, and the discount factor $\gamma \in [0, 1)$. In Partially Observable Markov Decision Process (POMDP) [212, 213] where the agent does not have full observability of the environment state, the agent maintains a belief about the state based on its observations and actions. Therefore, there are two additional properties, such as the observation probability and the belief state update function denoted by $\mathcal{Z}$ and b' , respectively. MARL approaches have been studied in the literature [8, 78, 80, 81]. In this survey, we focus on approaches that have been applied in CA-MAS. To support an agent to reason about unconventional situations and human requests in disaster relief management, Nadi and Edrisi

[165] utilized n -step Q-learning as [214]:

$$Q(s, a) = (1 - \alpha_n)Q_{n-1}(s, a) + \alpha_n[r(s, a) + \gamma \max_{a'} Q_{n-1}(s', a')] \\ \text{subject to } \alpha_n = \frac{1}{1 + N_{s,a}} \quad (\text{A.15})$$

where s' and a' are the future state and action, respectively, and $N_{s,a}$ represents the number of visited pairs of state and action until step n . Moreover, understanding other agents' intents support the agent to construct its goals for maximizing its expected utility. To achieve this, Qi and Zhu [151] modeled the agent's goal with its utility function and parameterized that function by using linear function approximation as follows:

$$g_i = \arg \max_g u(g \mid b_{-i}; h, \theta) \quad (\text{A.16})$$

where g_i represents the goal of the i^{th} agent, b_{-i} indicates the belief of intents of all other agents, h is the observation history, and θ represents all intent combinations of agents. Note that $\theta = (\theta_{ik,jl}) \in \mathbb{R}^{m \times n \times m}$ represents the intrinsic value of an agent i . In this matrix, m is the number of goals, n is the number of agents, and $\theta_{ik,jl}$ denotes the utility for agent i to pursue goal g_k when agent j is intended for goal g_l . Then, the utility function of the agent is estimated as [151]:

$$u(g_{i,k} \mid b_{-i}; h, \theta) = \theta_{ik,ik} + \sum_{j \neq i} \sum_l \theta_{ik,jl} p(g_{jl} \mid h) \quad (\text{A.17})$$

where $p(g_{jl} \mid h)$ is the probability of the event of agent j pursuing goal g_l given the history. In RL setting, Equation A.17 can be approximated as Q-value function as follows: $Q(h, g_{ik}) \approx u(g_{i,k} \mid b_{-i}; h, \theta)$. Additionally, the semi-gradient State-Action-Reward-State-Action (SARSA) algorithm [215] was utilized to update θ as follows:

$$\theta_{t+1} = \theta_t + \alpha \delta \nabla Q(h_t, g_t; \theta_t) \quad (\text{A.18})$$

$$\bar{r}_{t+1} = \bar{r}_t + \beta \delta \quad (\text{A.19})$$

where δ the estimation error of the average reward [216], α and β are learning rates. Furthermore, deep RL (DRL) techniques for goal-oriented reasoning were applied in existing CA-MAS [83, 84, 153, 155–157, 163] that employed learning-based techniques to represent high-dimensional sensory input. To support the agent to reason about sequences of human activities, the combination of Universal Value Function Approximators (UVFAs) [217] and Policy Continuation with Hindsight Inverse Dynamics (PCHID) [218] has been utilized in [163]. Specifi-

cally, UVFAs aim to approximate the reward associated with achieving specific goals $r_g : \mathcal{S} \times \mathcal{A} \rightarrow \mathbb{R}$. In addition, the technique formulates policies to get the optimal set of actions from state and goal pairs as follows: $\pi : \mathcal{S} \times \mathcal{G} \rightarrow \mathcal{A}$. PCHID aims to optimize the reasoning process after unifying n steps of goals and policies [163]:

$$\theta_k = \arg \min_{\theta} \sum_{s_t^i, s_{t+1}^i, a_t^i, i \in 1 \dots k} ||f_{\theta_k}((s_t, g_{t+1}), (s_{t+1}, g_{t+1}) - a_t)||^2 \quad (\text{A.20})$$

where f is the SGD model [209]. Moreover, Deep Q Network (DQN) [26] was applied in [156] to facilitate the intent sharing process of agents in highway merging scenarios. To reason about agents' observations and control inputs given driver types, Chen and Chaudhari [153] employed Double DQN [219] which consists of an online DQN and its periodic copy as the offline DQN. Additionally, the technique decouples the action selection from the action evaluation for reducing the issue of overestimation. To handle the large state space due to the action delay of agents in the autonomous navigation, Chen *et al.* [84] and Fan *et al.* [155] adopted Deep Deterministic Policy Gradient (DDPG) [220] with the Actor-Critic (AC) algorithm [92] in CA-MAS. In these approaches, the actor samples actions based on agents' observations and policy gradient, and the critic evaluates each pair of observation and action and returns Q-value as feedback to improve the performance of the actor. For scalability and stability of the reasoning process, Everett [83] utilized the Asynchronous Advantage Actor-critic (A3C) method [221] which is the extension of AC and parallelizes agents experiences in either GPUs or CPUs. Furthermore, to reason about agents' driving actions and their instant response which are drawn from the two distributions (see also Section A.5.2), the Proximal Policy Optimization (PPO) [28] was employed in [157].

Predict and Act

An agent projects scenarios or events that may occur in the near future. Once the projection is available, the agent performs its action to optimally achieve its goals, such as minimizing the estimation error or maximizing the cumulative rewards. However, in CA-MAS with pre-defined rules, cases, or patterns [140, 158–161, 164, 170–172, 185, 203, 206, 207], agents may lack prediction capability due to restricted conditions, weakening their ability to deal with uncertainties. To address this challenge, agents are equipped with predictive models that estimate near-future events using weighting schemes [154, 162, 167–169], probabilities [85, 152, 166, 197], or rewards [83, 84, 151, 153, 155–157, 163, 165].

Predictive models with weighting schemes encompass regression, classification,

and weighted prediction. In [168], agents employed a regression model to forecast charging station utilization and a classification model to determine parking lot availability for electric vehicle fleet routing. Similarly, in [169], agents utilized a classification model to assess user availability at specific times. This availability information, combined with data on user device usage, served as inputs for a regression model to predict energy consumption at those times. Furthermore, agents using weighted prediction models aim to forecast future data points based on weighted distributions [154, 167] or aggregated weights [162].

Agents with probabilistic prediction models aim to project future stages based on either probabilistic event chains or probabilistic distributions. For trajectory planning, agents in [197] and [152] predict the intents of other agents by analyzing the probability distribution of observations within autonomous navigation environments. Similarly, an agent in [85] predicts future trajectories based on the probability distribution of its historical data. Furthermore, agents in [166] represent fire spread by projecting events through a probabilistic chain involving factors such as fuel types, fire presence, fuel conditions, and fire locations, utilizing a Bayesian Belief Network (BBN).

An agent using reward-based models forecasts its sequential actions based on either the probability distribution [83, 84, 151, 156, 165] or the weighted distribution [153, 155, 157, 163] of its observations and historical data. These actions are evaluated through policy and value functions within the reinforcement learning setting. The expected values of these functions are referred to as rewards. The agent’s goal is to minimize predictions that result in negative rewards and maximize those that lead to positive rewards.

A.6 Research Gaps and Future Directions

CA-MAS is an emerging research topic. Although extensive efforts have been made on the integration between CAS and MAS in diverse problem domains, there are research directions worthy of future explorations.

Many context-aware multi-agent systems relax organizational constraints, causing two challenges. First, ineffective context sharing as one agent’s irrelevant context introduces noise, reducing performance. Picard *et al.* [108] identified two perspectives in MAS: the agent-centered view and the organization-centered view. They also emphasized the connection between organizational awareness in MAS and the agents’ ability to adapt and maintain autonomy. Furthermore, Esmaeili *et al.* [118] conducted an empirical study to demonstrate the effects of diversity on MAS performance. Similarly, Corkill *et al.* [222] showed that agent organizations help to reduce the complexity of MAS and enhance system performance. Despite

the existing literature on MAS, the impact of agent organizations has not been thoroughly explored in the context of CA-MAS. The second challenge is about security and privacy concerns. Due to the nature of CA-MAS, sensitive contextual information such as identity and location can be shared among agents. However, untrusted agents may exist without proper organizational structure that brings authority. These agents may threaten the system and grant unauthorized access that violates privacy regulations. Addressing this challenge requires investigating methodologies that incorporate organizational structures in CA-MAS to improve security and effective context sharing.

The assumptions of consensus among agents have also been relaxed in existing CA-MAS. This leads to several issues in CA-MAS, such as uncertainty and ambiguity, where there is incomplete information and semantic mismatch between such information, as well as conflicts among agents [7, 19, 20, 223, 224]. Addressing these issues enhances the robustness of agents in dealing with uncertainties and improves the effectiveness of learning strategies with joint information when solving specific tasks. However, solutions for these issues have not been actively discussed in the literature of CA-MAS. The application of consensus can be one potential solution. Research works such as those by [167] and [85] have attempted leader-follower consensus to improve information sharing among agents. Besides this type of consensus, it is worth investigating techniques that utilize other types of consensus [7, 19, 20] in CA-MAS for diverse problem domains.

Agents' ontology provides the semantic meaning of information, supporting agents in comprehending situations for effective problem-solving. Many existing CA-MAS have employed ontology for both learning and reasoning tasks (see also Sections A.5.2 and A.5.2); however, these CA-MAS heavily depend on pre-defined rules or patterns. This reliance affects the robustness of agents in dynamic environments where uncertain rules or patterns are constantly emerging. The integration between CA-MAS and goal-oriented reasoning frameworks (see also Section A.5.2) addresses such limitations. However, agents' ontology is not utilized in these systems. This may be due to two reasons: (1) the complexity of representing agents' ontology; and (2) the integration between those representations and goal-oriented techniques. Just as Graph Neural Network (GNN) and its variants [21] can potentially be utilized to address the first challenge; the Variational Auto-Encoder (VAE) architecture [199, 200] can be applied to produce the compatible learning representations for goal-oriented methods. As this demonstrates the feasibility of the integration, it would be interesting to explore novel approaches that integrate agents' ontology into goal-oriented CA-MAS to enhance the context comprehension of agents.

Appendix B

Goal-Aware Multi-Agent Reinforcement Learning for Decentralized Coordination

B.1 Front Matter

Title: Goal-Aware Multi-Agent Reinforcement Learning for Decentralized Coordination

Hung Du, Srikanth Thudumu, Hy Nguyen, Rajesh Vasa and Kon Mouzakis

Abstract: Multi-agent Reinforcement Learning (MARL) is gaining prominence in system where multiple agents collaborate to solve complex tasks in a dynamic environment. Existing MARL techniques often assume agents share a common goal and operate under centralized control. In real-world scenarios, and recent use cases leveraging Large Language Models (LLMs), agents have individual goals and limited observability, making coordination and adaptability more challenging. To address this issue, we propose an approach that endows agents with goal-aware coordination capabilities. Specifically, each agent is trained to coordinate with other agents within its communication range in the decentralized setting. Additionally, agents assess and integrate knowledge from others based on individual goals. We evaluate our approach in a 2D grid environment containing obstacles. Our findings indicate goal-aware experience sharing enhances overall performance and does not decrease the performance of individual agent.

Keywords: Multi-Agent Systems, Deep Reinforcement Learning, and Autonomous Navigation

B.2 Introduction

There is emerging evidence that shows sophisticated AI agents capable of solving complex tasks and achieving human-like performance in some contexts [225]. However, single agents are limited in terms of scalability, adaptability, and reliability; parallelisation can improve time to complete in some tasks for these single task agents, but does not improve ability in handling more complex tasks where we need specialist agents for sub-tasks [7, 20, 73, 74]. To address these challenges, multi-agent system architectures have emerged where agents communicate, coordinate and handle complex tasks in dynamic environments often making use of Multi-agent Reinforcement Learning (MARL) to handle the dynamics [225].

In MARL, an agent communicates and interacts with other agents within the same environment. This supports the agent to make decisions based on both its own understanding of the world and its observations of the actions taken by the other agents. Often, a naive design tactic is applied allowing an open communication between all agents which generates a large amount of information within the environment. This forces us to provision an environment with sufficient bandwidth, low latency and high compute. The core challenge is however, the requirement for the agent to have a smart filter that can assess the value of information against the goal and that which assists with coordination.

Addressing this filtering challenge requires agents adopt a communication strategy that contextually determines situations and targets of communication. Communication challenges have been considered in prior works; notably, Jiang and Ly [226] proposed an Attentional Communication Model (ATOC) that encodes each agent’s local observations, aggregates such representations and utilizes the aggregated information to instruct the agent when to communicate. Extending this, Liu *et al.* [227] developed a framework that combines agents’ observations to facilitate the selection of agents during communication. In addition, Liu *et al.* [23] applied graph-based estimation to enable agents to form communication groups and determine the events of communication.

After establishing communication, agents exchange and aggregate information to facilitate coordination. Sunehag *et al.* [228] introduced value-decomposition networks (VDN), enabling agents to learn from the joint actions of other agents. Gupta *et al.* [10] explored Centralized Training and Decentralized Execution (CTDE), where agents are trained using a centralized critic for cooperative tasks but execute actions with their own actor in a decentralized manner. Although these techniques enable agents to learn joint actions in a centralized setting, they are unable to select an optimal strategy in decentralized settings. To address this, Rashid *et al.* [13] proposed QMIX, which employs a monotonic value func-

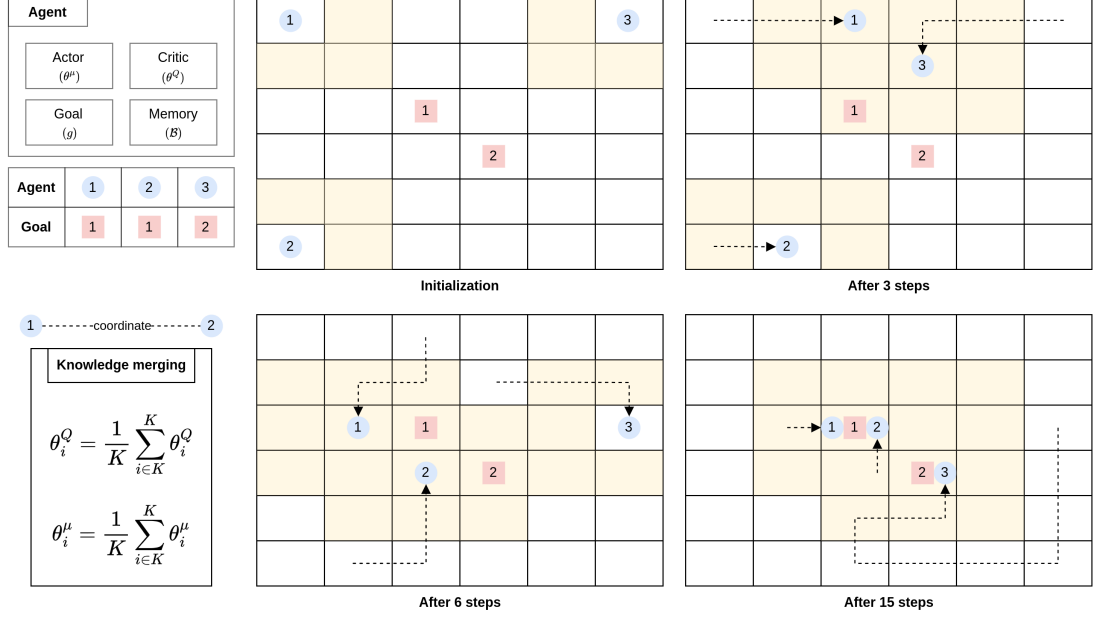


Figure B.1: The illustration of the coordination strategy in our framework. A blue circle indicates an agent, while a red square marks an agent’s goal. A filled yellow cell denotes the observation range around an agent. Initially, each agent’s position is set. At the third step, Agents 1 and 3 encounter each other within their observation ranges but do not coordinate since they do not share the same goal. At the sixth step, Agents 1 and 2 meet within their observation ranges and initiate the coordination process. Consequently, they reach their goal with one extra step, whereas Agent 3 reaches its goal with seven additional steps.

tion to allow agents to choose actions based on local state-action values relative to the global value. To account the underlying topology between agents, which represents the relationships between state-action values, Xu *et al.* [229] proposed the Multi-Graph Attention Network (MGAN). MGAN uses Graph Convolutional Networks (GCN) to represent this topology and integrates it with the joint actions of agents. Recently, Nayak *et al.* [15] utilized Graph Neural Networks (GNN) in both the actor and critic to aggregate local observations from all agents, using this aggregated information for action selection during decentralized execution. Ruan *et al.* [14] developed a Graph-based Coordination Strategy (GCS) to explore coordinated behaviors by decomposing the joint team policy into graph representations. Building on these, Pesce and Montana [230] introduced a method that uses a state-dependent communication graph to regulate information flow within the system, determining how information is shared and disseminated among agents.

Existing research in MARL with a focus on improving communication and coordination assumes that all agents share the same goal and can directly communicate and exchange their states and observations with each other at every time step. In practical scenarios, and real-world distributed computational constructs, agents have individual goals and limited observability (of behaviour of

other agents), and there is a need for techniques for these scenarios. In this paper, we propose an approach where: (1) each agent has its own goal and limited observations; (2) an agent only observes and communicates with other agents within its observation range; and (3) an agent exchanges knowledge with other agents during the communication stage only if they share the same goal. For our experiments, we focus on multi-agent navigation where agents cooperate to navigate towards their respective goals in a 2D grid environment. The rest of the chapter is organized as follows: we summarize related work in Section B.3, provide the problem formulation and our approach in Sections B.4 and B.5, respectively, demonstrate experimental results in Section B.6, and present our conclusions in Section B.7.

B.3 Related Work

In the context of Multi-Agent Reinforcement Learning (MARL), Tan [17] demonstrated that agents using Independent Q-Learning (IQL) in a cooperative manner outperform those that operate independently. Because agents make decisions independently and have their own observations, MARL algorithms can be framed within a Decentralized Partially Observable Markov Decision Process (Dec-POMDP) [231] (refer to Section B.4). To stabilize the learning process, the concept of Centralized Training with Decentralized Execution (CTDE) [10] has been explored. In CTDE, agents share global information and a centralized critic that evaluates state-action pairs during training. Each agent then uses its own actor to execute actions independently. Existing MARL algorithms [13, 15, 229, 230] that facilitate coordination have utilized CTDE. However, CTDE assumes that agents pursue organizational goals and share experiences to train the centralized critic, which may impact the robustness of agents in real-world scenarios characterized by two conditions: (1) agents may have different individual goals, and (2) an agent cannot communicate with others beyond its observation range. To address these challenges, our approach employs a fully decentralized MAS where each agent has its own goal, communicates only with agents within its observation range, and coordinates with agents sharing the same goal during the communication.

In decentralized setting, agents coordinate with each other by sharing their knowledge and observations. Christinanos et al. [232] introduced the shared experience actor-critic algorithm, which enables agents to learn from the trajectories of other agents. However, sharing trajectories can be costly, as agents may accumulate a large volume of experiences while navigating dynamic environments. Furthermore, trajectory sharing may not be feasible in domains, such as

finance and healthcare, where data protection standards are required. To address these challenges, agents can share knowledge through their learning weights. Researchers have explored various approaches to facilitate this, including transfer learning [233], federated learning [234], and other techniques within the context of MARL [235]. In our approach, we employ weight merging as a strategy to consolidate knowledge among agents with shared goals during communication.

B.4 Problem Preliminary

In our approach, multiple agents make decisions independently, each with different observations. This approach is therefore modeled as a decentralized partially observable Markov decision process (Dec-POMDP) [231] defined by the following tuple: $(n, \mathcal{S}, \{\mathcal{A}_i\}_{i=1}^n, T, \{\mathcal{R}_i\}_{i=1}^n, \{\mathcal{O}_i\}_{i=1}^n, P, \gamma)$. Here, n represents the number of agents, $\mathcal{S}$ is the set of states, $\{\mathcal{A}_i\}_{i=1}^n$ denotes the set of action sets for each agent, $T : \mathcal{S} \times \mathcal{A}^n \rightarrow \mathcal{S}'$ is the state transition probability function following the joint actions $\mathcal{A}^n = (a_1, a_2, \dots, a_n)$, $\{\mathcal{R}_i\}_{i=1}^n$ is the set of rewards for each agent, $\{\mathcal{O}_i\}_{i=1}^n$ represents the set of observations for each agent, $P : \mathcal{S} \times \mathcal{A}^n \rightarrow \mathcal{O}'$ is the observation probability function, and $\gamma \in [0, 1]$ is the discount factor.

Our approach defines an agent's goals and observations as its intrinsic context and the agent's position and shared knowledge as the extrinsic context. An agent is initialized with a goal, denoted by g_i . Additionally, the agent has its own observation range and only communicate with other agents that are within the range. During the communication process, an agent shares its knowledge and obtains the other agents' knowledge if they have the same goal.

Policy gradient (PG) algorithms allow an agent to learn the optimal policy π by adjusting parameters θ^π that maximizes the expected cumulative rewards. REINFORCE [236] achieves this by applying the gradient of the objective function as follows:

$$\nabla_{\theta} J(\pi_{\theta}) = \mathbb{E}[Q^{\pi}(s, a) \nabla_{\theta} \log \pi_{\theta}(a|s; \theta^{\pi})] \quad (\text{B.1})$$

where J is the objective function, Q is the state-action value function from Q-learning [237], and $\log \pi_{\theta}(a|s; \theta^{\pi})$ represents the log likelihood of selecting action a given state s , parameterized by θ^{π} .

The natural Actor-Critic (AC) method [92] addresses the high variance REINFORCE suffers during the learning process that can lead to instability and slow convergence. This approach includes an actor, which uses the policy π with parameters θ^{μ} to select an optimal action in a given state, and a critic, which evaluates the chosen action using either a state value function $V(s; \theta^w)$ or a state-action value function $Q(s, a; \theta^w)$. To assess the importance of an action

and reduce variance in policy gradient estimation, an advantage function is applied: $A(s, a) = Q(s, a) - V(s)$ where the loss functions for the actor and critic are estimated separately as follows:

$$\begin{aligned}\mathcal{L}_{\text{actor}}(\theta^\mu) &= -\mathbb{E}[\log \pi_\theta(a|s)A^\pi(s, a)] \\ \mathcal{L}_{\text{critic}}(\theta^w) &= \mathbb{E} \left[(R(s, a, s') + \gamma V^\pi(s'; \theta^w) - V^\pi(s; \theta^w))^2 \right]\end{aligned}\tag{B.2}$$

In our work, each agent is treated as an independent actor-critic allowing us to leverage prior-art.

B.5 Our Approach

Our approach aims to enhance the learning and exploration processes by equipping agents with goal-aware capabilities for coordination. We formulate this problem within a 2D grid environment. Algorithm 1 illustrates our approach. Furthermore, detailed explanations of each component of our approach are provided below.

B.5.1 Environment and Rewards

We construct a 2D grid environment denoted by $M^{w \times h}$ (see also Figure B.1). This environment contains n agents and m objects, where the position of an agent or an object is represented by (x, y) . An agent’s goal, denoted by $g = (x, y)$, is the position of an object that the agent aims to move towards.

At each time step, the current state of an agent is represented by the agent’s current position: $s_i^t = (x_i^t, y_i^t)$. An agent can choose from five possible actions: staying, moving up, moving down, moving left, or moving right within the boundaries of the environment. Multiple agents can occupy the same cell. An agent’s task is considered complete if it reaches its goal and remains in that position.

The sparse reward function of an agent is defined as:

$$R(s_i^t) = \begin{cases} 1 & \text{if } s_i^t = g_i \\ -\lambda_{\text{stay}} & \text{if } (s_i^{t-1} = s_i^t) \wedge (s_i^t \neq g_i) \\ 1 - \Delta(s_i^t, g_i) & \text{if } (s_i^{t-1} \neq s_i^t) \\ -1 & \text{otherwise} \end{cases}\tag{B.3}$$

The reward value ranges between -1 and 1. An agent receives a reward of 1 if its position matches its goal. If the agent remains in a cell that is not its goal, it is penalized by $\lambda_{\text{stay}} \in (0, 1)$. To incentivize movement towards the goal, an agent

Algorithm 1 Decentralized Federated Multi-Agent Coordination algorithm

```

1: Randomly initialize critic network per agent  $Q_i(s, a|\theta_i^Q)$  with  $\theta_i^Q$  and its
   target network  $\theta_i^{Q'} \leftarrow \theta_i^Q$ 
2: Randomly initialize actor network per agent  $\mu_i(s|\theta_i^\mu)$  with  $\theta_i^\mu$  and its target
   network  $\theta_i^{\mu'} \leftarrow \theta_i^\mu$ 
3: Initialize relay buffer per agent  $\{\mathcal{B}_i\}_{i=1}^n$ 
4: for episode = 1,  $M$  do
5:     Initialize observation state per agent  $s_i^1$ 
6:     for  $t = 1, T$  do
7:         Get observations of each agent  $\{\mathcal{O}_i^t\}_{i=1}^n$ 
8:         for each agent  $i$  do
9:             if  $i$  is not terminated then
10:                 Identify other agents  $\{j\}_{j \neq i}^n$  where  $(x_j^t, y_j^t) \in \mathcal{O}_i^t \wedge g_i = g_j$ 
11:                 if  $\{j\}$  exists then
12:                     Update  $\theta_i^Q$  and  $\theta_i^\mu$  according to the coordination strategy
13:                 end if
14:             end if
15:         end for
16:         Identify agents  $\{i\}$  that have not been terminated
17:         for each agent  $i$  do
18:             Select action  $a_i^t = \mu(s_i^t|\theta_i^\mu)$  according to the current policy
19:             Execute action  $a_i^t$  and observe reward  $r_i^t$  and the new state  $s_i^{t+1}$ 
20:             Store the transition  $(s_i^t, a_i^t, r_i^t, s_i^{t+1})$  in  $\mathcal{B}$ 
21:             Sample a random minibatch of  $N$  transitions  $(s_i^h, a_i^h, r_i^h, s_i^{h+1})$  from
                 $\mathcal{B}$ 
22:             Set  $y_i^h = r_i^h + \gamma Q'(s_i^{h+1}, \mu'(s_i^{h+1}|\theta_i^{\mu'})|\theta_i^{Q'})$ 
23:             Update critic by minimizing the loss:

$$\mathcal{L}_{\text{critic}} = \frac{1}{N} \sum_{i=1}^N \left( y_i^h - Q(s_i^h, a_i^h|\theta_i^Q) \right)^2$$

24:             Update the actor policy using the sampled policy gradient:

$$\nabla_{\theta^\mu} J \approx \frac{1}{N} \sum_{i=1}^N \nabla_a Q(s, a|\theta_i^Q) \Big|_{s=s_i^h, a=\mu(s_i^h)} \nabla_{\theta^\mu} \mu(s|\theta_i^\mu) \Big|_{s_i^h}$$

25:             Update target networks:

$$\begin{aligned} \theta_i^{Q'} &\leftarrow \tau \theta_i^Q + (1 - \tau) \theta_i^{Q'} \\ \theta_i^{\mu'} &\leftarrow \tau \theta_i^\mu + (1 - \tau) \theta_i^{\mu'} \end{aligned}$$

26:             if agent  $i$  reaches its goal then
27:                 Terminate  $i$ 
28:             end if
29:         end for
30:     end for
31: end for

```

receives a reward of $1 - \Delta(s_i^t, g_i)$, where Δ is the geometric distance between s_i^t and g_i , for each move, indicating that the closer the agent is to the goal, the higher the reward it receives.

B.5.2 State, Action and Relay Buffer

Each agent possesses its own actor-critic system. The actor’s goal is to choose the optimal action based on the agent’s current state, while the critic’s role is to evaluate the state-action pair. Similar to the Deep Deterministic Policy Gradient (DDPG) algorithm [27], we utilize deep neural networks in both the actor and the critic to model the state and action. Additionally, each agent has its own memory, known as the relay buffer $\mathcal{B}$, which stores up to H experiences of the agent. An experience consists of the tuple $(s_i^h, a_i^h, r_i^h, s_i^{h+1})$ from past interactions.

The actor network of the i^{th} agent, denoted by $\mu_i(s|\theta_i^\mu)$, is initialized with random weights and parameterized by θ_i^μ . Given the current state of the agent, the network aims to generate the weight distribution for five actions as: $z = \mu_i(s|\theta_i^\mu)$. This distribution is then converted into the probability distribution as follows:

$$\sigma(z_l) = \frac{e^{z_l}}{\sum_{k=1}^K e^{z_k}} \quad (\text{B.4})$$

where $l, k \in K$ are indices of actions. The exploration-exploitation dilemma is commonly controlled by the use of ϵ with a specific threshold depending on the task setting [16]. However, the choice of ϵ is not robust because it varies across scenarios. To address this challenge, we apply the multinomial sampling on the probability distribution of actions. This aims to ensure two facets: (1) all actions have a chance to be selected; and (2) an action with the high probability will be more likely to be selected.

The critic network of the i^{th} agent, denoted by $Q_i(s, a|\theta_i^Q)$, is also initialized with random weights and parameterized by θ_i^Q . Given the state and the corresponding selected action, the network aims to generate a value that can be utilized to evaluate a pair of state and action.

B.5.3 Coordination Strategy

An agent has a specific observation range, illustrated in Figure B.1. This range consists of C cells surrounding the agent’s current position and within the environment boundaries. The range is denoted by $c \in \mathbb{Z}^+$. As the agent moves through the environment, it may encounter other agents within its observation range (refer to Figure B.1), facilitating the establishment of communication. During this communication phase, the agent shares its goal and identifies other agents with

the same objective. Instead of exchanging entire historical experiences, which can be costly, agents with the same goal share their knowledge through weight sharing as follows:

$$\theta_i^Q = \frac{1}{K} \sum_{i \in K}^K \theta_i^Q \quad (\text{B.5})$$

$$\theta_i^\mu = \frac{1}{K} \sum_{i \in K}^K \theta_i^\mu \quad (\text{B.6})$$

where $K \leq N$ is the number of agents that share the same goal within the range.

B.6 Experiments

B.6.1 Experiment Details

The 2D grid environment is sized at 10×10 , denoted as $M^{10 \times 10}$. Each agent's actor network consists of two fully-connected multi-layer perceptrons, each layer containing 128 neuron units. This configuration is also used for the agent's critic network. Adam [238] is employed as the optimizer for learning the neural network parameters, with a learning rate of 10^{-4} for the actor and 10^{-3} for the critic. The discount factor γ is set to 0.99, and the soft target update rate τ is set to 10^{-3} . In addition, we applied the Manhattan distance and $\lambda_{\text{stay}} = 0.5$ for Equation B.3. Furthermore, the batch size of the relay buffer $\mathbb{B}$ is 64, and the number of episode is 300. We estimate the average reward of an agent at each episode as follows:

$$\text{AvgR} = \frac{1}{T_i} \sum_{t=1}^{T_i} r_i^t \quad \text{where } T_i \leq \mathcal{T} \quad (\text{B.7})$$

where T_i is the total number of steps taken by the i^{th} agent in one episode, and $\mathcal{T}$ is the maximum number of steps that an agent is allowed to take per episode. The overall performance of the system is then estimated as:

$$R_{\text{overall}} = \frac{1}{M} \sum_{i=1}^M \frac{1}{N} \sum_{j=1}^N \text{AvgR}_j \quad (\text{B.8})$$

where M is the number of episode, and N is the number of agents.

In our experiments, we aim to validate whether the coordination among agents support to reduce number of steps for achieving the goal. Hence, $\mathcal{T}$ is the set of the following values $\{100, 150, 200\}$. We also aim to evaluate the impact of coordination on overall performance. Therefore, we set up the following scenarios:

1. **Independent agents versus cooperative agents:** Two agents start at

specified positions, such as $(x_1, y_1) = (1, 1)$ and $(x_2, y_2) = (1, 9)$. In the first system, these agents do not share the same goal. Meanwhile, in the second system, they do share the same goal $g_1 = (4, 4)$. We compare the performance of S1 and S2 to assess the effects of coordination between multiple agents.

2. **Multiple agents with a limited number of steps:** This scenario begins with two agents at specified positions, and we gradually increase the number of agents up to four. All agents in the environment share the same goal, $g_1 = (4, 4)$. We aim to determine whether having more agents reduces the number of steps needed to achieve the goal.

It is worth noting that each goal is specific to an individual agent and is not treated as a collective goal for the organization.

B.6.2 Results and Discussion

The system with cooperative agents outperforms the one with independent agents (see Table B.1). Specifically, the overall performance of the system with independent agents (**Case 1**) is -0.08. In contrast, the maximum performance with two cooperative agents (**Case 4**) is 0.185, and with three cooperative agents (**Case 6**), it is 0.35. Additionally, an independent agent may be lazy or perform poorly. However, this issue is mitigated when agents collaborate. For example, Agent 2's performance improves significantly when it starts coordinating with other agents. This improvement may be due to the support and motivation provided by the knowledge and interaction of the other agents, which enhances Agent 2's navigation capability.

No.	Case	$\mathcal{T}$	Agent 1		Agent 2		Agent 3		R_{overall}
			AvgR	AvgS	AvgR	AvgS	AvgR	AvgS	
1	Two agents without coordination	200	0.28	150	-0.43	200	N/A	N/A	-0.08
2	Two agents with coordination	200	0.26	155	0.06	185	N/A	N/A	0.16
3	Two agents with coordination	150	0.28	115	0.08	142	N/A	N/A	0.18
4	Two agents with coordination	100	0.31	82	0.06	98	N/A	N/A	0.185
5	Three agents with coordination	200	0.36	130	0.34	143	0.33	141	0.34
6	Three agents with coordination	150	0.38	108	0.33	117	0.33	120	0.35
7	Three agents with coordination	100	0.36	83	0.32	84	0.32	88	0.33

Table B.1: The experimental results for seven cases. AvgR denotes the agent's average reward, while AvgS represents the average number of steps the agent took in each episode.

When an agent is introduced to a system with coordination, it not only enhances the performance of other agents but also reduces the number of steps each

agent takes to complete an episode. In **Case 1**, Agent 2 took an average of 200 steps, yet it still failed to reach its goal, resulting in negative rewards. However, this issue is resolved when agents coordinate. Specifically, Agent 2 began receiving more positive rewards and taking fewer steps to reach its goal in **Cases 2** and **5**. Similarly, in **Case 5**, Agent 1 achieved higher average rewards and took fewer steps to reach its goal. It is worth noting that the performance of a well-performing agent can be affected by agents with poor performance (see **Cases 1** and **2**). This may be due to knowledge merging, where a well-performing agent acquires the negative experiences of poorly performing agents.

When the maximum number of steps per episode ($\mathcal{T}$) decreases, agents with coordination maintain their performance while using fewer steps (see **Cases 3**, **4**, **6**, and **7**). This may be because a lower $\mathcal{T}$ reduces the number of steps an agent spends wandering in an episode, thereby decreasing the number of negative experiences stored in the relay buffer ($\mathcal{B}$). Consequently, the agent’s performance either remains unchanged or improves moderately.

B.7 Conclusion and Future Work

We proposed a novel Multi-Agent Reinforcement Learning (MARL) approach that endows agents with goal-aware coordination capabilities. This approach was applied to a multi-agent navigation challenge and evaluated within a 2D grid environment containing obstacles. Our results showed that the multi-agent system with coordination outperformed the system with independent agents; showing task completion in fewer steps. There are several potential directions for future work. Our experiments revealed that our coordination strategy also impacts the performance of well-performing agents. Therefore, it is essential to explore alternative coordination strategies to mitigate this effect. The robustness of agents in more complex environments and with different reward shaping functionalities remain an open problem.

Appendix C

Contextual Knowledge Sharing in Multi-Agent Reinforcement Learning with Decentralized Communication and Coordination

This survey has been submitted to the Association for the Advancement of Artificial Intelligence (AAAI) conference and is currently under the peer-review process.

C.1 Front Matter

Title: Contextual Knowledge Sharing in Multi-Agent Reinforcement Learning with Decentralized Communication and Coordination

Hung Du, Srikanth Thudumu, Hy Nguyen, Rajesh Vasa and Kon Mouzakis

Abstract: Decentralized Multi-Agent Reinforcement Learning (Dec-MARL) has emerged as a pivotal approach for addressing complex tasks in dynamic environments. Existing Multi-Agent Reinforcement Learning (MARL) methodologies typically assume a shared objective among agents and rely on centralized control. However, many real-world scenarios feature agents with individual goals and limited observability of other agents, complicating coordination and hindering adaptability. Existing Dec-MARL strategies prioritize either communication or coordination, lacking an integrated approach that leverages both. This paper presents a novel Dec-MARL framework that integrates peer-to-peer communication and coordination, incorporating goal-awareness and time-awareness into

the agents’ knowledge-sharing processes. Our framework equips agents with the ability to (i) share contextually relevant knowledge to assist other agents, and (ii) reason based on information acquired from multiple agents, while considering their own goals and the temporal context of prior knowledge. We evaluate our approach through several complex multi-agent tasks in environments with dynamically appearing obstacles. Our work demonstrates that incorporating goal-aware and time-aware knowledge sharing significantly enhances overall performance.

Keywords: Multi-Agent Systems, Deep Reinforcement Learning, Autonomous Navigation, Decentralized Multi-Agent Reinforcement Learning, Multi-Agent Coordination and Communication, and Context-Aware Knowledge Sharing

C.2 Introduction

Cooperative Multi-Agent Reinforcement Learning (MARL) has emerged as a critical research area due to its potential to overcome the limitations of single-agent systems in addressing complex, real-world problems. While single-agent systems have demonstrated success in achieving human-like performance in specific scenarios [225], they often face limitations in terms of scalability, adaptability, and reliability, especially when dealing with complex tasks that require specialized agents [7, 73, 74]. To address these limitations, the multi-agent system (MAS) architecture has gained prominence, enabling agents to communicate, coordinate, and tackle complex tasks in dynamic environments. MARL plays a key role in handling such dynamics [81, 225]. Among the various approaches within MARL, the Centralized Training and Decentralized Execution (CTDE) paradigm [9, 10] is popular for cooperative tasks [11–15, 239, 240]. This approach employs a centralized critic during training to develop decentralized policies for agents, which are then executed independently. Although widely adopted, CTDE-based algorithms encounter significant difficulties in environments with large joint state-action spaces and inherent stochasticity. Moreover, these algorithms typically assume that agents share a common goal and depend on centralized control. However, many real-world situations involve agents with individual objectives and limited observability of others, leading to potential miscoordination, sub-optimal policies, and reduced adaptability.

The Decentralized Training and Decentralized Execution (DTDE) paradigm [17, 18] aims to address the limitations of the CTDE approach by relaxing the assumptions of full observability and centralized control. In a fully decentralized setting, each agent operates with its own goals and observations, communicates with other agents within its observation range, and coordinates during these com-

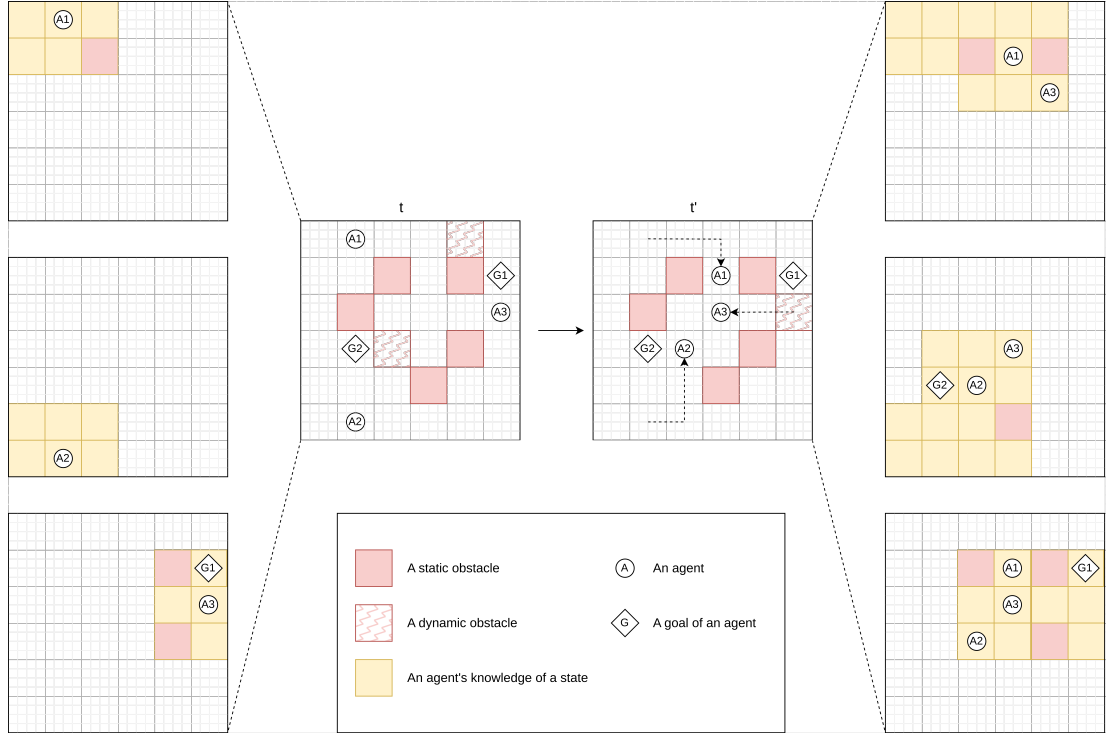


Figure C.1: An illustration of a fully decentralized environment with multiple agents ($t < t'$). While the goal of Agents 1 and 2 is G1, that of Agent 3 is G2. Note that at time t' , Agent A3 is unaware of an obstacle that has occurred in a position it previously encountered, rendering its knowledge about that location obsolete. Additionally, during a communication session with A3, Agents A1 and A2 must be aware of the outdated information provided by A3 to select the optimal action.

munication sessions. The agent then uses the acquired observations and knowledge to optimize its objectives. This approach has the potential to enhance the robustness and adaptability of agents in handling uncertainties. However, DTDE-based algorithms can face significant challenges, including (a) exhaustive exploration due to the absence of a centralized coordinator and limited observability, and (b) inefficient sharing of experience and knowledge caused by the growing number of agents and the rapid obsolescence of information.

A promising approach to reducing the exhaustive exploration of independent agents in DTDE-based algorithms is to establish a communication protocol among agents. A straightforward communication scheme allows agents to share their local observations, which can then be used to optimize their local policies toward individual goals [18, 226, 241–243]. While this approach reduces exploration time, it also introduces a significant amount of irrelevant information, which increases learning complexity and can degrade performance. Several strategies have been proposed to address this challenge. For example, agents can be instructed on when to communicate [23, 227, 244]. In team settings, incentive-based communication schemes have been used to filter out trivial information and promote coordination toward a global objective [245]. Additionally, integrating Graph Neural Networks (GNNs) with agent feature embeddings and mutual information has been applied to eliminate irrelevant information [246]. Other methods focus on pruning irrelevant agents from communication sessions by leveraging agents’ identities [247] and personalized communication topology [248]. Furthermore, approaches to address communication bandwidth limitations have been introduced by advising mechanisms [249, 250] and message pruning [251, 252]. Although these strategies show promise in addressing communication challenges among agents, they often assume that decentralized agents share the same local objective. However, even within the same team or coalition, agents may pursue different individual goals (see also Figure C.1). Therefore, goal awareness becomes essential to improve the effectiveness of communication among agents.

Coordination strategies facilitate the efficient sharing of experience and knowledge among independent agents in DTDE-based algorithms. One widely-used strategy involves utilizing a global value, estimated by aggregating the local values of states and actions across agents [13, 253]. Furthermore, graph-based approaches [14, 15, 22, 230] have been employed to represent the relationships between observations or agents, which are then used to enhance agent coordination. Advising mechanisms [249, 250] also play a crucial role by encouraging experienced agents to offer guidance to less experienced agents, based on their knowledge. These mechanisms further motivate agents to explore novel states in the environment, which can be also achieved by estimating intrinsic rewards

through weighted mutual information between agents’ novel states [254]. However, these strategies assume that observations and knowledge remain constant over time. In practical scenarios, the value of information decays and eventually becomes invalid, leading to sub-optimal policies. Therefore, time awareness is essential for improving the effectiveness of coordination among agents.

In this paper, we propose a novel Dec-MARL framework designed to address two key challenges in fully decentralized settings: exhaustive exploration and inefficient knowledge sharing among agents. Our framework integrates peer-to-peer communication and coordination, incorporating both goal awareness and time awareness to provide agents with two primary capabilities. First, goal-aware communication enables agents to exclude irrelevant agents during communication sessions. Second, agents can retrieve relevant observations and share their knowledge by understanding the goals of other agents. Additionally, we introduce a time factor that decays the value of information over time, along with a novel intrinsic reward mechanism that encourages agents to explore new states in the environment. We evaluate our framework using complex multi-agent tasks in a grid world environment where obstacles dynamically appear. Our experiments demonstrate that our framework enhances agents’ exploration and knowledge sharing in fully decentralized environments.

C.3 Related Work

Decentralized Training and Decentralized Execution (DTDE)

Approaches in cooperative MARL typically fall into two categories: Centralized Training and Decentralized Execution (CTDE) [9, 10] and DTDE [17]. CTDE-based methods [11–15, 239, 240] have demonstrated the stability of training multiple agents for cooperative tasks in complex environments. These approaches often assume that agents have unlimited access to all states in the environment and rely on centralized control for assessing agents’ actions. However, such assumptions are not feasible in many real-world scenarios where environments are dynamic, agents have limited observability, and may pursue different individual goals. As a result, the robustness of CTDE-based approaches diminishes in these situations. Conversely, DTDE-based methods [17, 18, 255–258] do not require full observability or centralized control among agents. Despite this, DTDE approaches often encounter challenges such as exhaustive exploration and inefficient knowledge sharing, due to several factors: (a) the absence of efficient communication and coordination strategies; (b) the lack of centralized control; (c) the increasing number of agents; and (d) the rapid changes in information within

dynamic environments. To address these challenges, we propose a novel DTDE-based framework that equips agents with goal awareness and time awareness and integrates communication and coordination among agents in fully decentralized settings.

Communication

Communication is vital in overcoming the challenge of exhaustive exploration in MARL approaches. A naive design involves establishing a communication protocol among all agents within the same environment [226, 241–243]. However, this approach can hinder agent performance due to the curse of dimensionality, as the number of agents increases and information overload becomes an issue. Existing methods that aim to mitigate this challenge can be categorized into three groups: (a) communication-triggering instructions [23, 227, 244]; (b) filtering out irrelevant information [245, 246, 251, 252]; and (c) filtering out irrelevant agents [247, 248]. These methods typically operate within the CTDE framework and assume that agents share the same local objectives. Our framework differs from these approaches in two key ways: (i) agents operate in a fully decentralized setting with limited communication; and (ii) agents are equipped with goal awareness, enabling them to understand the goals of other agents before initiating communication sessions.

Coordination

Coordination strategies are crucial for effective knowledge sharing among agents. Existing approaches often involve aggregating information among agents [13, 232, 253] and improving this process by also considering the relationships between pieces of information [14, 15, 22, 230]. These strategies, however, typically operate within the CTDE framework, which can hinder coordination in many real-world scenarios. Another form of coordination involves motivating agents to explore novel states in the environment through advice [249, 250] or intrinsic rewards [254, 259]. However, these approaches often overlook the fact that the value of information decays over time and can become obsolete, leading to inefficient knowledge sharing. Our framework addresses this by incorporating both time awareness and goal awareness to enhance agent coordination. We also introduce a novel reward function that uses a time-aware intrinsic reward to motivate agents to explore new states and revisit previously known states to refresh their knowledge.

C.4 Problem Preliminary

We formulate our framework as the Decentralized Multi-Agent Reinforcement Learning (Dec-MARL) where the decision-making process of an agent follows Partially Observable Markov Decision Process (POMDP) [231]. This is defined as follows: $(n, \mathcal{S}, \{\mathcal{A}_i\}_{i=1}^n, T, \{\mathcal{R}_i\}_{i=1}^n, \{\mathcal{O}_i\}_{i=1}^n, P, \gamma)$ where n is the number of agents, $\mathcal{S}$ is the set of states, $\{\mathcal{A}_i\}_{i=1}^n$ denotes the set of action sets for each agent, $T : \mathcal{S} \times \mathcal{A}^n \rightarrow \mathcal{S}'$ is the state transition probability function following the joint actions $\mathcal{A}^n = (a_1, a_2, \dots, a_n)$, $\{\mathcal{R}_i\}_{i=1}^n$ is the set of rewards for each agent, $\{\mathcal{O}_i\}_{i=1}^n$ represents the set of observations for each agent, $P : \mathcal{S} \times \mathcal{A}^n \rightarrow \mathcal{O}'$ is the observation probability function, and $\gamma \in [0, 1]$ is the discount factor. Furthermore, a set of individual goals of agents is denoted by $\{\mathcal{G}_i\}_{i=1}^n$ where goals can be defined in terms of states $\mathcal{G} \subseteq \mathcal{S}$ [217]. Notably, an agent can only access its own local observations and learn an independent policy π_i to maximize its own goal in the decentralized setting.

In a fully decentralized environment, agents' observations can be limited and vary from one another (see also Figure C.1). As a result, an agent only possesses knowledge of the states it has experienced, leaving other states unknown. Moreover, the value of acquired knowledge diminishes over time due to the environment's dynamics. This necessitates that agents consider the time factor associated with such knowledge when adjusting their policies. In our framework, we model this behavior by introducing a mental state for each agent, denoted by $\{\mathcal{M}_i\}_{i=1}^n$. It is important to note that, within our framework, all agents share the same ontology and bounded environment. Consequently, the mental state of an agent encompasses all masked states in the environment, defined as follows: $\mathcal{M}_i = \{(s, m_t, d_t)\}_{s \in \mathcal{S}}$, where m represents the masked label of state s at time step t (e.g., *empty*, *obstacle*, *unknown*, or other), and d_t denotes the duration since the last visit. Additionally, m_t dynamically changes in response to the environment.

C.5 Method

In this section, we propose a novel Dec-MARL framework that equips agents with goal awareness and time awareness for addressing challenges of communication and coordination in full decentralized environments. Furthermore, detailed explanations of each component of our framework are provided below.

C.5.1 Representations and Value Approximation

To facilitate generalization, our framework encodes the following properties of the agent: (s, g, o, m, a) . Specifically, we define $f_x(x) \rightarrow \mathbf{e}_x \in \mathbb{R}^k$ as the

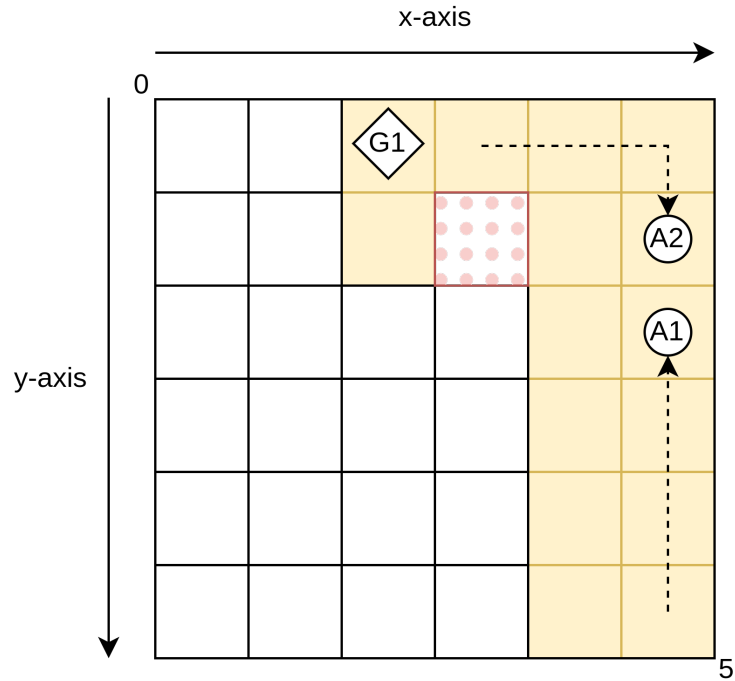


Figure C.2: A demonstration of the intrinsic reward guiding Agent 1 (A1) to choose an action that optimizes both the goal-oriented objective and the exploration of uncertainty. The filled yellow boxes represent knowledge of A1 in terms of that position, the red box filled by dots is obstacle, and the remaining are unknown to A1. In this scenario, the optimal action for A1 is to move towards the (4, 2) position, as it strikes a balance between both objectives.

representation function, which could involve methods such as one-hot encoding, Multi-Layer Perceptron (MLP), categorical encoding, image-based encoding, or other representation techniques. Here, x represents one of the agent’s properties, and k denotes the dimensionality of the embedding. It is important to note that both f and k can vary depending on the specific agent property. Furthermore, the mental state of an agent is represented as follows:

$$\mathbf{e}_{\mathcal{M}} = \bigcup_{(s,m) \in \mathcal{M}} (\mathbf{e}_s \oplus \mathbf{e}_m) \quad (\text{C.1})$$

where $\oplus$ is the concatenation operation between two embeddings between $\mathbf{e}_s$ and $\mathbf{e}_m$, and $\bigcup$ is the aggregation function (e.g., summation, dot product, average pooling, or other). Our framework applies the average pooling. Note that the scheme of Equation C.1 excludes the time factor.

Understanding its current goal and recent mental state is essential for an agent to adjust its actions in two key ways: (a) moving toward the goal based on its belief about future states, or (b) exploring uncertain states that could be advantageous for achieving the current goal. As illustrated in Figure C.2, there are situations where the agent must balance these two aspects to maximize rewards. Inspired by the Universal Value Function Approximator (UVFA) [217], our framework integrates both the agent’s goal and mental state into the construction of the policy function as $\pi : \mathcal{S} \times \mathcal{G} \times \mathcal{M} \rightarrow \mathcal{A}$. The corresponding action-value function is then defined as $Q(s, a, g, \mathcal{M}; \theta^Q) \approx Q_{g, \mathcal{M}}^*(s, a)$ where θ^Q is learning parameters.

C.5.2 Time Awareness and Intrinsic Rewards

A reward provided by the environment is designed to guide an agent toward achieving its goal, commonly referred to as an extrinsic reward. In decentralized training, the agent does not have access to the global state. Therefore, exploring novel observations that are based on the agent’s local observations and potentially beneficial for future outcomes can be encouraged by using an intrinsic reward. The novelty of an observation is often estimated using a utility function with count-based mechanisms [254] as follows:

$$u_i^t(o) = \frac{1}{\mathcal{N}_o} \quad (\text{C.2})$$

where u_i^t is dependent on the local observations of agent i at time t , and $\mathcal{N}_o$ is the frequency of the observation. Additionally, u_i^t can vary between agents. Equation C.2 indicates that the novelty of an observation decreases as it occurs more

frequently in the agent’s experience. However, in many practical scenarios, an observation may become novel again despite its high frequency. This is due to the dynamics of the environment. Hence, instead of using count-based mechanisms, we introduce the time factor that measures the novelty of an observation as:

$$u_i^t(o) = e^{\frac{1}{2}d_{t'}} \quad (\text{C.3})$$

where e is the exponential function, $d_{t'} \in \mathcal{M}_i$, and $t' \leq t$. In addition, $d_{t'}$ is controllable according to the application domains. From our empirical analysis, we would suggest keeping $d_{t'}$ as small as possible with the time increment less than 0.1 in situations where information is gradually changed (see also Figure C.3). Equation C.3 satisfies the following two conditions: (a) the value of the observation decays over time after being uncovered by the agent; and (b) the observation becomes novel again after being re-discovered by the agent. Furthermore, the value of u_i^t can be integrated with embeddings in Equation C.1 and convert $\mathbf{e}_{\mathcal{M}}$ into the time-aware scheme as follows:

$$\mathbf{e}_{\mathcal{M}}^t = \bigcup_{(s,m) \in \mathcal{M}} (u_i^t(s) \cdot (\mathbf{e}_s \oplus \mathbf{e}_m)) \quad (\text{C.4})$$

where $u_i^t(s) \in \mathbb{R}$ is a scalar value.

In addition to being integrated with the embedding of the mental state of an agent, we introduce a novel reward estimation that combines both the extrinsic reward and u_i^t as the intrinsic reward as follows:

$$r_i^s = (1 - \alpha)r_{\text{ext}} + \alpha \frac{1}{|\mathcal{M}|} \sum_{s' \in \mathcal{M}} u_i^t(s') \quad (\text{C.5})$$

where r_{ext} is the extrinsic reward of the agent and can be customized in terms of application domains, $|\mathcal{M}|$ is the number of states in the agent’s mental state, and $\alpha \in [0, 1]$ is the dampening factor that balances two types of rewards. Moreover, as shown in Equation C.5, the intrinsic reward increases when the agent continues to explore new states or revisits old ones. However, the agent is not solely biased toward exploration; instead, it aims to move toward states that balance both factors.

C.5.3 Integration of Communication and Coordination

In fully decentralized settings, it is essential for agents to communicate and share relevant observations and knowledge. While relevant observations can accelerate an individual agent’s exploration, relevant knowledge can enhance their per-

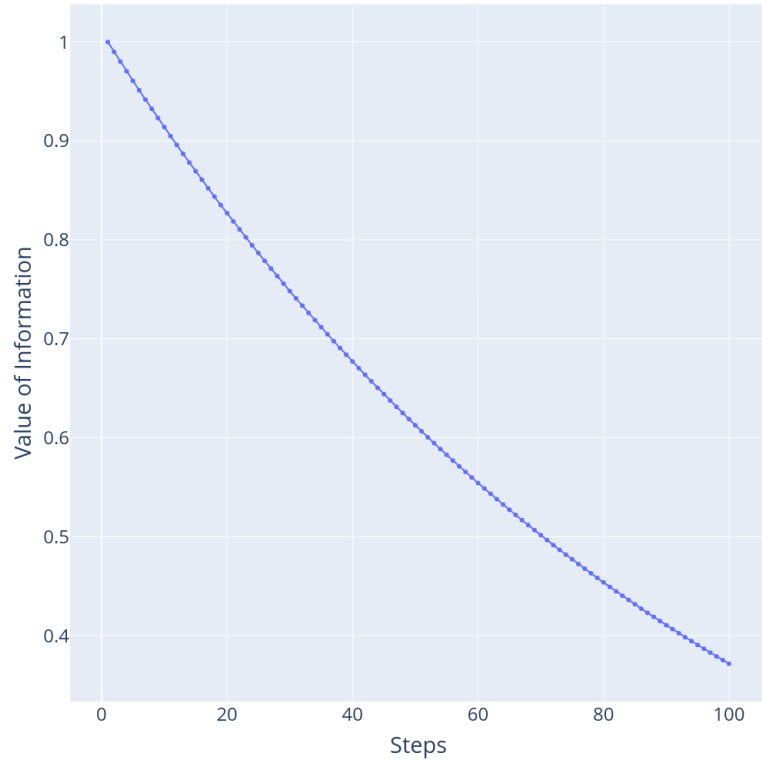


Figure C.3: An illustration of utilizing Equation C.3 to estimate the novelty of information over 100 steps where $d_{t'}$ is estimated with the time increment of 0.01 as: $t' = t + 0.01$. Importantly, in this graph, we assume that the information is not reflected by an agent per step.

formance in achieving their goals. However, shared information can have both positive and negative impacts on an agent’s policy and action value function [246, 250]. Therefore, it is important for agents to carefully evaluate the information they receive before incorporating it into their current policy and action value function. In our framework, we propose a strategy that integrates communication and coordination, incorporating goal awareness and time awareness. This strategy consists of three phases: Share-Reason-Aggregate. The details of each phase are specified below.

Share

As the agent navigates the environment, it may encounter other agents within its observation range, allowing for the establishment of communication and coordination sessions. During a communication session, the agent broadcasts its goal to identify two types of agents: (a) agents who share the same goal, known as current peers, and (b) agents who have relevant knowledge from their experience but do not share the same goal, referred to as current advisors. It is worth noting that peers do not necessarily have prior experience of the given goal. Once this identification process is complete, the agents initiate the coordination session. Both peers and advisors retrieve observations relevant to the goal. The retrieval mechanisms can differ depending on the problem domain. In our framework, a peer retrieves both observations from its mental state and learning parameters such as θ^π and θ^Q . Additionally, inspired by [258], each agent in our framework is equipped with a heuristic planning capability that is activated only when the agent is in the role of an advisor. Specifically, in discrete observation spaces, such as a 2D map (x, y) , an advisor estimates the shortest path comprising observations between the agent’s current position and the given goal. Advisors do not share their learning parameters, as these parameters are optimized for different goals that may not align with the agent’s current goal. It is important to note that agents in our framework share the same ontology and bounded environment, making observations and knowledge transferable among them.

Reason

After the knowledge-sharing process, the agent activates its reasoning capability rather than blindly following the acquired observations and knowledge. To achieve this, our framework equips agents with a rule-based reasoning capability. First, the agent reflects on its mental state using the latest and novel observations

shared by peers and advisors as follows:

$$\mathcal{M}_i = \bigcup_{j=1; s \in \mathcal{S}}^K \begin{cases} (s, m_t, d_t)_i, & \text{if } (d_t)_i < (d_t)_j \\ (s, m_t, d_t)_j, & \text{if } (d_t)_i > (d_t)_j \vee (s)_j \notin \mathcal{M}_i \end{cases} \quad (\text{C.6})$$

where $\bigcup$ is the set union function, K is the total number of peers and advisors and j represents the index of a peer or an advisor. Second, to determine whether to update its learning parameters, the agent estimates the overlap ratio between its mental state and the observations shared by each peer. In our framework, this overlap ratio between discrete observations is calculated using the Jaccard similarity as:

$$J(\mathcal{M}_i, \mathcal{M}_j) = \frac{|\{(s, m_t)\}_i \cap \{(s, m_t)\}_j|}{|\{(s, m_t)\}_i \cup \{(s, m_t)\}_j|} \quad (\text{C.7})$$

where $J \in [0, 1]$, with $J = 0$ indicating that agents i and j have no overlapping observations, and $J = 1$ indicating a complete match between their mental states. Here, s represents the known state of an agent. It is important to note that this estimation takes place before agents update their mental states with the newly acquired observations. The primary objective is to encourage the agent to integrate novel knowledge obtained from its peers.

Aggregate

After selecting peers based on the overlap ratio, the agent updates its learning parameters as follows:

$$\theta_i^\pi = (1 - \beta)\theta_i^\pi + \beta \frac{1}{K} \sum_{j=1}^K \theta_j^\pi \quad (\text{C.8})$$

$$\theta_i^Q = (1 - \beta)\theta_i^Q + \beta \frac{1}{K} \sum_{j=1}^K \theta_j^Q \quad (\text{C.9})$$

where K represents the total number of selected peers, and β is the dampening factor that balances the agent's own learning parameters with those aggregated from its peers. Our empirical analysis indicates that Equations C.8 and C.9 can sometimes lead to situations where poor-performing agents negatively impact the performance of others during the coordination session. Therefore, we recommend keeping β as low as possible.

C.6 Experiments

C.6.1 Environments and Tasks

To evaluate our framework, we designed a 2D map with dynamically appearing obstacles. The environment comes in two sizes: Base (10 x 10) and Large (20 x 20), to test the scalability of our framework. Each environment features 3 objects surrounded by obstacles, with the number and positions of these obstacles being static. We created two difficulty levels for the environment: Easy and Hard. In the Easy environment, obstacles remain unchanged over time, while in the Hard environment, obstacles can appear and disappear dynamically. Specifically, in the Hard environment, an obstacle may appear at time t and disappear at a later time t' , where $t' > t$, or vice versa. The Hard environment is designed to assess how well agents in our framework handle environmental dynamics.

Combining the environment sizes with the difficulty levels results in four distinct environments: Base-Easy, Base-Hard, Large-Easy, and Large-Hard. In each environment, an agent starts at a predefined position far from its goal, with agents being placed in different areas far from one another. There are two scenarios regarding the agents' goals: (i) all agents pursue the same goal, and (ii) there are two distinct goals, with at least two agents pursuing the first goal and the remaining agents pursuing the second goal. Moreover, multiple agents can occupy the same cell. An agent's task is considered complete if it reaches its goal and remains in that position.

C.6.2 Implementation Details

We conducted our experiments a complex 2D environment with fully decentralized settings. Here, $s_i^t = (x_i^t, y_i^t)$, $\mathcal{G} \subset \mathcal{S}$, m can be one of the following labels: *empty*, *obstacle*, *object*, *agent*, or *unknown*, and a can be one of the following options: *left*, *right*, *up*, *down*, or *stay*. Furthermore, we utilized categorical encoding functions to represent agent's properties (s , g , o , m , a) in their embeddings as: $\mathbf{e}_s \in \mathbb{R}^{64}$, $\mathbf{e}_g \in \mathbb{R}^{16}$, $\mathbf{e}_o \in \mathbb{R}^{64}$, $\mathbf{e}_m \in \mathbb{R}^{16}$, and $\mathbf{e}_a \in \mathbb{R}^{16}$.

We implemented the Actor-Critic method [92] for each agent in our framework. This method includes an actor, which uses the policy π with learning parameters θ^μ to select an action in a given state, and a critic, which evaluates the chosen action using an action value function Q with learning parameters θ^w . In addition, we followed the implementation details of the Deep Deterministic Policy Gradient (DDPG) algorithm [27]. Each agent's actor network consists of two fully-connected Multi-Layer Perceptrons (MLP), each layer containing 128 neuron units. This configuration is also used for the agent's critic network. Adam

[238] is employed as the optimizer for learning the neural network parameters, with a learning rate of 10^{-4} for the actor and 10^{-3} for the critic. The discount factor γ is set to 0.99, and the soft target update rate τ is set to 10^{-3} . Furthermore, the batch size of the relay buffer $\mathcal{B}$ is 64.

We also designed a sparse reward function of an agent as follows:

$$R(s_i^t) = \begin{cases} 1 & \text{if } s_i^t = g_i \\ -\lambda_{\text{stay}} & \text{if } (s_i^{t-1} = s_i^t) \wedge (s_i^t \neq g_i) \\ (r_{\text{agg}})_i^t & \text{if } (s_i^{t-1} \neq s_i^t) \\ -1 & \text{otherwise} \end{cases} \quad (\text{C.10})$$

The reward value ranges between -1 and 1. An agent receives a reward of 1 if its position matches its goal. If the agent remains in a cell that is not its goal, it is penalized by $\lambda_{\text{stay}} \in (0, 1)$. In our experiments, we applied $\lambda_{\text{stay}} = 0.5$ to encourage an agent to keep moving. Moreover, to incentivize movement towards the goal, an agent receives a reward of r_{agg} that is the same as Equation C.5. Specifically, $r_{\text{ext}} = 1 - \Delta(s_i^t, g_i)$, where Δ is the geometric distance between s_i^t and g_i , for each move, indicating that the closer the agent is to the goal, the higher the reward it receives. Inspired by [258], our experiments utilized the shortest path between s_i^t and g_i for Δ as follows: $\Delta(s_i^t, g_i) = \min(d(s_i^t, g_i))$. From our empirical analysis, we found that $\alpha \in [0.1, 0.5]$ in Equation C.5 tends to yield high outcomes, and hence, $\alpha = 0.1$ is the selected value for our experiments. Furthermore, the number of episodes and steps per episode are 100 and 300, respectively. Hence, we applied the time increment of 0.01 for d_t in Equation C.3 for gradually decaying the value of agent's knowledge. The average reward of an agent at each episode is estimated as follows:

$$\text{AvgR} = \frac{1}{T_i} \sum_{t=1}^{T_i} r_i^t \quad \text{where } T_i \leq \mathcal{T} \quad (\text{C.11})$$

where T_i is the total number of steps taken by agent i in one episode, and $\mathcal{T}$ is the maximum number of steps that an agent is allowed to take per episode. The overall performance of the system is then estimated as:

$$R_{\text{overall}} = \frac{1}{M} \sum_{i=1}^M \frac{1}{N} \sum_{j=1}^N \text{AvgR}_j \quad (\text{C.12})$$

where M is the number of episode, and N is the number of agents. We aim to evaluate the performance of an agent in our framework based on both the average rewards and the number of steps an agent taken until reaching its goal. In terms

of agent’s coordination and knowledge aggregation, we set a threshold for J in Equation C.7 as $J \leq 0.5$. This is designed to encourage the agent to learn from the substantial amount of novel knowledge shared by its peers. Additionally, we set $\beta = 0.1$ in Equations C.8 and C.9 to prevent the agent’s current knowledge from being overwhelmed by the influx of new knowledge.

In our experiments, we designed the following types of agents:

1. **Independent Agents with DDPG (A^1):** This type of agent follows the pure implementation of multi-agent DDPG (MADDPG) [11]. However, agents are operated in a fully decentralized setting instead of adopting the framework of Centralized Training with Decentralized Execution (CTDE). Since this type of agent does not have time awareness, the intrinsic reward in Equation C.5 is always set to 0.
2. **A^1 with Mental State (A^2):** In comparison to A^1 , an additional feature of this type of agent is the utilization of mental state per agent ($\mathcal{M}_i$). Note that A^2 still does not have time awareness.
3. **A^2 with Time Awareness (A^3):** A^3 is the extension of A^2 by having an additional feature of time awareness. However, A^3 is still an independent agent without the capability of communication and coordination.
4. **A^3 with Communication and Coordination (A^4):** A^4 extends A^3 by being equipped with the capability of communication and coordination. However, A^4 agents does not have goal awareness during the communication and coordination sessions. Hence, an agent is always an advisor of the other agent. Additionally, it shares its observations regardless the other agent’s goal.
5. **A^4 with Goal Awareness for Coordination (A^5):** A^5 is the ultimate type of agent in our experiments. This type of agent have both time awareness and goal awareness for communication and coordination. Hence, A^5 is equipped with all features in this study.

This design aims to evaluate the impact of each component that is integrated into an independent agent in the fully decentralized setting.

C.6.3 Results and Discussion

Scenario 1

Table C.1 presents the experimental results for the first scenario, where all agents pursue the same goal. The integration of both mental state and time-awareness in

independent agents within a fully decentralized setting (A^2 through A^5) generally yields better outcomes compared to A^1 . Specifically, in the Base-Easy environment, A^5 outperforms the other agents, completing tasks with 5% fewer steps on average. In the Base-Hard environment, not only does the performance of all agent types improve, but the number of steps taken is also reduced by 15% compared to the Base-Easy environment. Notably, A^2 outperforms the others in this scenario, potentially due to dynamic obstacles creating pathways that allow faster goal achievement. Furthermore, A^4 and A^5 excel in the Large-Easy and Large-Hard environments, respectively, highlighting the importance of time-awareness for effective exploration in larger observation spaces. Additionally, to achieve higher outcomes when dealing with dynamic environments, time-aware agents must communicate and coordinate with each other. Interestingly, we observed that agents only reached their goals in a few episodes within the large environments. A potential solution is to increase the number of episodes and the maximum steps per episode. As these numbers increase, it is also crucial to select an appropriate value for $d_{t'}$ in Equation C.3.

	Base-Easy	Base-Hard	Large-Easy	Large-Hard
A^1	0.118 ± 0.06	0.176 ± 0.07	0.233 ± 0.04	0.243 ± 0.03
A^2	0.135 ± 0.06	0.207 ± 0.06	0.232 ± 0.04	0.225 ± 0.03
A^3	0.132 ± 0.05	0.198 ± 0.05	0.239 ± 0.04	0.236 ± 0.04
A^4	0.106 ± 0.06	0.168 ± 0.06	0.216 ± 0.04	0.237 ± 0.04
A^5	0.139 ± 0.06	0.191 ± 0.05	0.229 ± 0.04	0.235 ± 0.03

Table C.1: The overall performance (R_{overall}) of all agent types when pursuing a single goal across four different environments.

Scenario 2

By comparing Table C.2 with Table C.1, we observe that agents generally achieve higher rewards in the second scenario compared to the first. As illustrated in Table C.2, the overall performance of A^2 through A^5 continues to surpass that of A^1 . Moreover, time-aware agents equipped with communication and coordination capabilities (A^4 and A^5) excel in three environments: Base-Easy, Base-Hard, and Large-Hard. Although A^5 does not outperform the independent agents in the Large-Easy environment, it is notable that A^5 tends to take fewer steps and reaches its goals in more episodes than the other types of agents.

Ablation Study

We conducted an ablation study to assess the contribution of each additional feature for independent agents in a fully decentralized environment. The first

	Base-Easy	Base-Hard	Large-Easy	Large-Hard
A^1	0.134 ± 0.05	0.203 ± 0.05	0.249 ± 0.03	0.251 ± 0.03
A^2	0.144 ± 0.05	0.22 ± 0.05	0.242 ± 0.05	0.243 ± 0.04
A^3	0.112 ± 0.05	0.223 ± 0.05	0.242 ± 0.04	0.247 ± 0.04
A^4	0.163 ± 0.05	0.208 ± 0.05	0.224 ± 0.03	0.246 ± 0.04
A^5	0.144 ± 0.05	0.225 ± 0.05	0.232 ± 0.04	0.233 ± 0.03

Table C.2: The overall performance (R_{overall}) of all agent types when at least two agents pursue the same goal, while the remaining agents pursue a different goal across four distinct environments.

feature examined was the mental state of an agent (A^2), which generally enhances the performance of A^1 in the Base environments across both scenarios. However, this feature alone is insufficient for agents operating in the Large environments. To address this limitation, time awareness was introduced as an additional feature (A^3). The results in Tables C.1 and C.2 highlight the improvement of agents equipped with both mental state and time awareness compared to A^1 . When communication and coordination were integrated into A^3 , performance improvements were observed in three environments—Base-Easy, Base-Hard, and Large-Hard—across both scenarios. Furthermore, enhancing agent performance in Hard environments is crucial for managing dynamics in a fully decentralized setting. The introduction of goal-awareness also led to performance gains in the Base environments. Our ablation study demonstrates significant improvements in the performance of independent agents within a decentralized setting when equipped with mental state, time awareness, goal awareness, and a strategy that integrates communication and coordination.

C.7 Conclusion and Future Work

We proposed a novel Decentralized Multi-Agent Reinforcement Learning (DecMARL) framework that aims to address two key challenges such as exhaustive exploration and inefficient knowledge sharing among agent in the fully decentralized settings. Our framework introduces several innovative aspects: (i) the incorporation of an agent’s mental state with time awareness; (ii) time-aware intrinsic rewards that motivate agents to explore novel states, potentially aiding in the achievement of their individual goals; (iii) the integration of communication and coordination; and (iv) the inclusion of goal awareness within this integration to facilitate efficient knowledge sharing. Experimental results demonstrate that our framework progressively enhances the performance of independent agents in fully decentralized 2D environments, where observation spaces may vary in size

and obstacles can appear dynamically. Several potential directions for future work have emerged. Our experiments indicate that agents may require additional training time to achieve their goals in larger environments. Therefore, it is crucial to meticulously evaluate configurations related to time awareness to attain this. Additionally, while agents in our framework engage in peer-to-peer communication, they do not form any organizational structure despite having the same goal. Establishing an organization based on overlapping goals may accelerate and stabilize the exploration process, making it a promising area for further investigation.

Bibliography

- [1] X.-W. Chen and X. Lin, “Big data deep learning: challenges and perspectives,” *IEEE access*, vol. 2, pp. 514–525, 2014.
- [2] S. Dargan, M. Kumar, M. R. Ayyagari, and G. Kumar, “A survey of deep learning and its applications: a new paradigm to machine learning,” *Archives of Computational Methods in Engineering*, vol. 27, pp. 1071–1092, 2020.
- [3] A. Dorri, S. S. Kanhere, and R. Jurdak, “Multi-agent systems: A survey,” *Ieee Access*, vol. 6, pp. 28 573–28 593, 2018.
- [4] Z. Zhou, G. Liu, and Y. Tang, “Multi-agent reinforcement learning: Methods, applications, visionary prospects, and challenges,” *arXiv preprint arXiv:2305.10091*, 2023.
- [5] B. Horling and V. Lesser, “A survey of multi-agent organizational paradigms,” *The Knowledge engineering review*, vol. 19, no. 4, pp. 281–316, 2004.
- [6] C. Zhu, M. Dastani, and S. Wang, “A survey of multi-agent reinforcement learning with communication,” *arXiv preprint arXiv:2203.08975*, 2022.
- [7] A. Amirkhani and A. H. Barshooi, “Consensus in multi-agent systems: a review,” *Artificial Intelligence Review*, vol. 55, no. 5, pp. 3897–3935, 2022.
- [8] P. Hernandez-Leal, B. Kartal, and M. E. Taylor, “A survey and critique of multiagent deep reinforcement learning,” *Autonomous Agents and Multi-Agent Systems*, vol. 33, no. 6, pp. 750–797, 2019.
- [9] L. Kraemer and B. Banerjee, “Multi-agent reinforcement learning as a rehearsal for decentralized planning,” *Neurocomputing*, vol. 190, pp. 82–94, 2016.
- [10] J. K. Gupta, M. Egorov, and M. Kochenderfer, “Cooperative multi-agent control using deep reinforcement learning,” in *Autonomous Agents and Mul-*

- tiagent Systems: AAMAS 2017 Workshops, Best Papers, São Paulo, Brazil, May 8-12, 2017, Revised Selected Papers 16*. Springer, 2017, pp. 66–83.
- [11] R. Lowe, Y. I. Wu, A. Tamar, J. Harb, O. Pieter Abbeel, and I. Mordatch, “Multi-agent actor-critic for mixed cooperative-competitive environments,” *Advances in neural information processing systems*, vol. 30, 2017.
 - [12] J. Foerster, G. Farquhar, T. Afouras, N. Nardelli, and S. Whiteson, “Counterfactual multi-agent policy gradients,” in *Proceedings of the AAAI conference on artificial intelligence*, vol. 32, no. 1, 2018.
 - [13] T. Rashid, M. Samvelyan, C. S. De Witt, G. Farquhar, J. Foerster, and S. Whiteson, “Monotonic value function factorisation for deep multi-agent reinforcement learning,” *Journal of Machine Learning Research*, vol. 21, no. 178, pp. 1–51, 2020.
 - [14] J. Ruan, Y. Du, X. Xiong, D. Xing, X. Li, L. Meng, H. Zhang, J. Wang, and B. Xu, “Gcs: Graph-based coordination strategy for multi-agent reinforcement learning,” in *Proceedings of the 21st International Conference on Autonomous Agents and Multiagent Systems*, ser. AAMAS ’22. Richland, SC: International Foundation for Autonomous Agents and Multiagent Systems, 2022, p. 1128–1136.
 - [15] S. Nayak, K. Choi, W. Ding, S. Dolan, K. Gopalakrishnan, and H. Balakrishnan, “Scalable multi-agent reinforcement learning through intelligent information aggregation,” in *International Conference on Machine Learning*. PMLR, 2023, pp. 25 817–25 833.
 - [16] R. S. Sutton and A. G. Barto, *Reinforcement learning: An introduction*. MIT press, 2018.
 - [17] M. Tan, “Multi-agent reinforcement learning: Independent vs. cooperative agents,” in *Proceedings of the tenth international conference on machine learning*, 1993, pp. 330–337.
 - [18] J. Jiang and Z. Lu, “I2q: A fully decentralized q-learning algorithm,” *Advances in Neural Information Processing Systems*, vol. 35, pp. 20 469–20 481, 2022.
 - [19] J. Qin, Q. Ma, Y. Shi, and L. Wang, “Recent advances in consensus of multi-agent systems: A brief survey,” *IEEE Transactions on Industrial Electronics*, vol. 64, no. 6, pp. 4972–4983, 2016.

- [20] Y. Li and C. Tan, “A survey of the consensus for multi-agent systems,” *Systems Science & Control Engineering*, vol. 7, no. 1, pp. 468–482, 2019.
- [21] Z. Wu, S. Pan, F. Chen, G. Long, C. Zhang, and S. Y. Philip, “A comprehensive survey on graph neural networks,” *IEEE transactions on neural networks and learning systems*, vol. 32, no. 1, pp. 4–24, 2020.
- [22] W. Böhmer, V. Kurin, and S. Whiteson, “Deep coordination graphs,” in *International Conference on Machine Learning*. PMLR, 2020, pp. 980–991.
- [23] Y.-C. Liu, J. Tian, N. Glaser, and Z. Kira, “When2com: Multi-agent perception via communication graph grouping,” in *Proceedings of the IEEE/CVF Conference on computer vision and pattern recognition*, 2020, pp. 4106–4115.
- [24] B. Tang, Y. Zhong, U. Neumann, G. Wang, S. Chen, and Y. Zhang, “Collaborative uncertainty in multi-agent trajectory forecasting,” *Advances in Neural Information Processing Systems*, vol. 34, pp. 6328–6340, 2021.
- [25] Y. Zhang, Q. Yang, D. An, and C. Zhang, “Coordination between individual agents in multi-agent reinforcement learning,” in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 35, no. 13, 2021, pp. 11 387–11 394.
- [26] V. Mnih, K. Kavukcuoglu, D. Silver, A. A. Rusu, J. Veness, M. G. Bellemare, A. Graves, M. Riedmiller, A. K. Fidjeland, G. Ostrovski *et al.*, “Human-level control through deep reinforcement learning,” *nature*, vol. 518, no. 7540, pp. 529–533, 2015.
- [27] T. P. Lillicrap, J. J. Hunt, A. Pritzel, N. M. O. Heess, T. Erez, Y. Tassa, D. Silver, and D. Wierstra, “Continuous control with deep reinforcement learning,” *CoRR*, 2015.
- [28] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, “Proximal policy optimization algorithms,” *arXiv preprint arXiv:1707.06347*, 2017.
- [29] M. Carroll, R. Shah, M. K. Ho, T. Griffiths, S. Seshia, P. Abbeel, and A. Dragan, “On the utility of learning about humans for human-ai coordination,” *Advances in neural information processing systems*, vol. 32, 2019.
- [30] G. Papoudakis, F. Christianos, L. Schäfer, and S. V. Albrecht, “Benchmarking multi-agent deep reinforcement learning algorithms in

- cooperative tasks,” in *NeurIPS Datasets and Benchmarks*, 2020. [Online]. Available: <https://api.semanticscholar.org/CorpusID:235417602>
- [31] X. Pan, M. Liu, F. Zhong, Y. Yang, S.-C. Zhu, and Y. Wang, “Mate: Benchmarking multi-agent reinforcement learning in distributed target coverage control,” *Advances in Neural Information Processing Systems*, vol. 35, pp. 27 862–27 879, 2022.
- [32] M. Bettini, A. Prorok, and V. Moens, “Benchmarl: Benchmarking multi-agent reinforcement learning,” *Journal of Machine Learning Research*, vol. 25, no. 217, pp. 1–10, 2024.
- [33] B. Ellis, J. Cook, S. Moalla, M. Samvelyan, M. Sun, A. Mahajan, J. Foerster, and S. Whiteson, “Smacv2: An improved benchmark for cooperative multi-agent reinforcement learning,” *Advances in Neural Information Processing Systems*, vol. 36, 2024.
- [34] S. Goel, Y. Wei, P. Lymperopoulos, matthias. scheutz, and J. Sinapov, “Novelgym: A flexible ecosystem for hybrid planning and learning agents designed for open worlds,” in *Adaptive Agents and Multi-Agent Systems*, 2024. [Online]. Available: <https://api.semanticscholar.org/CorpusID:266844576>
- [35] B. Cai, Y. Xiang, L. Gao, H. Zhang, Y. Li, and J. Li, “Temporal knowledge graph completion: A survey,” in *International Joint Conference on Artificial Intelligence*, 2022. [Online]. Available: <https://api.semanticscholar.org/CorpusID:246063616>
- [36] S. Bubeck, V. Chandrasekaran, R. Eldan, J. Gehrke, E. Horvitz, E. Kamar, P. Lee, Y. T. Lee, Y. Li, S. Lundberg *et al.*, “Sparks of artificial general intelligence: Early experiments with gpt-4,” *arXiv preprint arXiv:2303.12712*, 2023.
- [37] A. I. Hauptman, B. G. Schelble, N. J. McNeese, and K. C. Madathil, “Adapt and overcome: Perceptions of adaptive autonomous agents for human-ai teaming,” *Computers in Human Behavior*, vol. 138, p. 107451, 2023.
- [38] L. Wang, C. Ma, X. Feng, Z. Zhang, H. Yang, J. Zhang, Z. Chen, J. Tang, X. Chen, Y. Lin *et al.*, “A survey on large language model based autonomous agents,” *Frontiers of Computer Science*, vol. 18, no. 6, pp. 1–26, 2024.
- [39] M. Baldauf, S. Dustdar, and F. Rosenberg, “A survey on context-aware systems,” *International Journal of ad Hoc and ubiquitous Computing*, vol. 2, no. 4, pp. 263–277, 2007.

- [40] C. Hoareau and I. Satoh, “Modeling and processing information for context-aware computing: A survey,” *New Generation Computing*, vol. 27, pp. 177–196, 2009.
- [41] S. Lee, J. Chang, and S.-g. Lee, “Survey and trend analysis of context-aware systems,” *Information-An International Interdisciplinary Journal*, vol. 14, no. 2, pp. 527–548, 2011.
- [42] C. Perera, A. Zaslavsky, P. Christen, and D. Georgakopoulos, “Context aware computing for the internet of things: A survey,” *IEEE communications surveys & tutorials*, vol. 16, no. 1, pp. 414–454, 2013.
- [43] A. Abbas, L. Zhang, and S. U. Khan, “A survey on context-aware recommender systems based on computational intelligence techniques,” *Computing*, vol. 97, pp. 667–690, 2015.
- [44] U. Alegre, J. C. Augusto, and T. Clark, “Engineering context-aware systems and applications: A survey,” *Journal of Systems and Software*, vol. 117, pp. 55–83, 2016.
- [45] P. Pradeep and S. Krishnamoorthy, “The mom of context-aware systems: A survey,” *Computer Communications*, vol. 137, pp. 44–69, 2019.
- [46] S. Kulkarni and S. F. Rodd, “Context aware recommendation systems: A review of the state of the art techniques,” *Computer Science Review*, vol. 37, p. 100255, 2020.
- [47] M. S. U. Islam, A. Kumar, and Y.-C. Hu, “Context-aware scheduling in fog computing: A survey, taxonomy, challenges and future directions,” *Journal of Network and Computer Applications*, vol. 180, p. 103008, 2021.
- [48] H. Vahdat-Nejad, Z. Abbasi-Moud, S. A. Eslami, and W. Mansoor, “Survey on context-aware healthcare systems,” in *2021 IEEE 11th annual computing and communication workshop and conference (CCWC)*. IEEE, 2021, pp. 1190–1196.
- [49] M. Casillo, F. Colace, D. Conte, M. Lombardi, D. Santaniello, and C. Valentino, “Context-aware recommender systems and cultural heritage: a survey,” *Journal of Ambient Intelligence and Humanized Computing*, vol. 14, no. 4, pp. 3109–3127, 2023.
- [50] Y. LeCun, Y. Bengio, and G. Hinton, “Deep learning,” *nature*, vol. 521, no. 7553, pp. 436–444, 2015.

- [51] S. Pouyanfar, S. Sadiq, Y. Yan, H. Tian, Y. Tao, M. P. Reyes, M.-L. Shyu, S.-C. Chen, and S. S. Iyengar, “A survey on deep learning: Algorithms, techniques, and applications,” *ACM Computing Surveys (CSUR)*, vol. 51, no. 5, pp. 1–36, 2018.
- [52] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, “Attention is all you need,” *Advances in neural information processing systems*, vol. 30, 2017.
- [53] T. Lin, Y. Wang, X. Liu, and X. Qiu, “A survey of transformers,” *AI open*, vol. 3, pp. 111–132, 2022.
- [54] W. Guo, J. Wang, and S. Wang, “Deep multimodal representation learning: A survey,” *Ieee Access*, vol. 7, pp. 63 373–63 394, 2019.
- [55] T. Baltrušaitis, C. Ahuja, and L.-P. Morency, “Multimodal machine learning: A survey and taxonomy,” *IEEE transactions on pattern analysis and machine intelligence*, vol. 41, no. 2, pp. 423–443, 2018.
- [56] P. Xu, X. Zhu, and D. A. Clifton, “Multimodal learning with transformers: A survey,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 2023.
- [57] R. Möller and B. Neumann, “Ontology-based reasoning techniques for multimedia interpretation and retrieval,” in *Semantic Multimedia and Ontologies: Theory and Applications*. Springer, 2008, pp. 55–98.
- [58] M. N. Asim, M. Wasim, M. U. G. Khan, N. Mahmood, and W. Mahmood, “The use of ontology in retrieval: a study on textual, multilingual, and multimedia retrieval,” *IEEE Access*, vol. 7, pp. 21 662–21 686, 2019.
- [59] S. Ji, S. Pan, E. Cambria, P. Marttinen, and S. Y. Philip, “A survey on knowledge graphs: Representation, acquisition, and applications,” *IEEE transactions on neural networks and learning systems*, vol. 33, no. 2, pp. 494–514, 2021.
- [60] Z. Zhang, P. Cui, and W. Zhu, “Deep learning on graphs: A survey,” *IEEE Transactions on Knowledge and Data Engineering*, vol. 34, no. 1, pp. 249–270, 2020.
- [61] P. F. Christiano, J. Leike, T. Brown, M. Martic, S. Legg, and D. Amodei, “Deep reinforcement learning from human preferences,” *Advances in neural information processing systems*, vol. 30, 2017.

- [62] C. Wirth, R. Akrou, G. Neumann, and J. Fürnkranz, “A survey of preference-based reinforcement learning methods,” *Journal of Machine Learning Research*, vol. 18, no. 136, pp. 1–46, 2017.
- [63] A. Hussein, M. M. Gaber, E. Elyan, and C. Jayne, “Imitation learning: A survey of learning methods,” *ACM Computing Surveys (CSUR)*, vol. 50, no. 2, pp. 1–35, 2017.
- [64] J. Jara-Ettinger, “Theory of mind as inverse reinforcement learning,” *Current Opinion in Behavioral Sciences*, vol. 29, pp. 105–110, 2019.
- [65] R. Rafailov, A. Sharma, E. Mitchell, C. D. Manning, S. Ermon, and C. Finn, “Direct preference optimization: Your language model is secretly a reward model,” *Advances in Neural Information Processing Systems*, vol. 36, 2024.
- [66] Y. Liu and R. Bucknall, “A survey of formation control and motion planning of multiple unmanned vehicles,” *Robotica*, vol. 36, no. 7, pp. 1019–1047, 2018.
- [67] H. Litimein, Z.-Y. Huang, and A. Hamza, “A survey on techniques in the circular formation of multi-agent systems,” *Electronics*, vol. 10, no. 23, p. 2959, 2021.
- [68] Y. Liu, J. Liu, Z. He, Z. Li, Q. Zhang, and Z. Ding, “A survey of multi-agent systems on distributed formation control,” *Unmanned Systems*, pp. 1–14, 2023.
- [69] C. Zhu, M. Dastani, and S. Wang, “A survey of multi-agent deep reinforcement learning with communication,” *Autonomous Agents and Multi-Agent Systems*, vol. 38, no. 1, p. 4, 2024.
- [70] S. Hong, X. Zheng, J. Chen, Y. Cheng, J. Wang, C. Zhang, Z. Wang, S. K. S. Yau, Z. Lin, L. Zhou *et al.*, “Metagpt: Meta programming for multi-agent collaborative framework,” *arXiv preprint arXiv:2308.00352*, 2023.
- [71] C. Fernando, D. Banarse, H. Michalewski, S. Osindero, and T. Rocktäschel, “Promptbreeder: Self-referential self-improvement via prompt evolution,” *arXiv preprint arXiv:2309.16797*, 2023.
- [72] A. Madaan, N. Tandon, P. Gupta, S. Hallinan, L. Gao, S. Wiegrefe, U. Alon, N. Dziri, S. Prabhunoye, Y. Yang *et al.*, “Self-refine: Iterative refinement with self-feedback,” *Advances in Neural Information Processing Systems*, vol. 36, 2024.

- [73] A. Kantamneni, L. E. Brown, G. Parker, and W. W. Weaver, “Survey of multi-agent systems for microgrid control,” *Engineering applications of artificial intelligence*, vol. 45, pp. 192–203, 2015.
- [74] F. De la Prieta, S. Rodríguez-González, P. Chamoso, J. M. Corchado, and J. Bajo, “Survey of agent-based cloud computing applications,” *Future generation computer systems*, vol. 100, pp. 223–236, 2019.
- [75] G. D. Abowd, A. K. Dey, P. J. Brown, N. Davies, M. Smith, and P. Steggles, “Towards a better understanding of context and context-awareness,” in *Handheld and Ubiquitous Computing: First International Symposium, HUC’99 Karlsruhe, Germany, September 27–29, 1999 Proceedings 1*. Springer, 1999, pp. 304–307.
- [76] C. Qian, X. Cong, C. Yang, W. Chen, Y. Su, J. Xu, Z. Liu, and M. Sun, “Communicative agents for software development,” *arXiv preprint arXiv:2307.07924*, 2023.
- [77] W. Chen, Y. Su, J. Zuo, C. Yang, C. Yuan, C. Qian, C.-M. Chan, Y. Qin, Y. Lu, R. Xie *et al.*, “Agentverse: Facilitating multi-agent collaboration and exploring emergent behaviors in agents,” *arXiv preprint arXiv:2308.10848*, 2023.
- [78] L. Busoniu, R. Babuska, and B. De Schutter, “A comprehensive survey of multiagent reinforcement learning,” *IEEE Transactions on Systems, Man, and Cybernetics, Part C (Applications and Reviews)*, vol. 38, no. 2, pp. 156–172, 2008.
- [79] T. T. Nguyen, N. D. Nguyen, and S. Nahavandi, “Deep reinforcement learning for multiagent systems: A review of challenges, solutions, and applications,” *IEEE transactions on cybernetics*, vol. 50, no. 9, pp. 3826–3839, 2020.
- [80] K. Zhang, Z. Yang, and T. Başar, “Multi-agent reinforcement learning: A selective overview of theories and algorithms,” *Handbook of reinforcement learning and control*, pp. 321–384, 2021.
- [81] S. Gronauer and K. Diepold, “Multi-agent deep reinforcement learning: a survey,” *Artificial Intelligence Review*, pp. 1–49, 2022.
- [82] Y. LeCun, “A path towards autonomous machine intelligence version 0.9. 2, 2022-06-27,” *Open Review*, vol. 62, no. 1, 2022.

- [83] M. Everett, Y. F. Chen, and J. P. How, “Motion planning among dynamic, decision-making agents with deep reinforcement learning,” in *2018 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE, 2018, pp. 3052–3059.
- [84] B. Chen, M. Xu, Z. Liu, L. Li, and D. Zhao, “Delay-aware multi-agent reinforcement learning for cooperative and competitive environments,” *arXiv preprint arXiv:2005.05441*, 2020.
- [85] X. Xie, C. Zhang, Y. Zhu, Y. N. Wu, and S.-C. Zhu, “Congestion-aware multi-agent trajectory prediction for collision avoidance,” in *2021 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2021, pp. 13 693–13 700.
- [86] J. D. Oliveira, D. C. Engelmann, D. Kniest, R. Vieira, and R. H. Bordini, “Multi-agent interaction to assist visually-impaired and elderly people,” *International Journal of Environmental Research and Public Health*, vol. 19, no. 15, p. 8945, 2022.
- [87] Z. Zhou, J. Liu, and J. Yu, “A survey of underwater multi-robot systems,” *IEEE/CAA Journal of Automatica Sinica*, vol. 9, no. 1, pp. 1–18, 2021.
- [88] L.-J. Lin, *Reinforcement learning for robots using neural networks*. Carnegie Mellon University, 1992.
- [89] T. Schaul, J. Quan, I. Antonoglou, and D. Silver, “Prioritized experience replay,” in *4th International Conference on Learning Representations, ICLR 2016, San Juan, Puerto Rico, May 2-4, 2016, Conference Track Proceedings*, Y. Bengio and Y. LeCun, Eds., 2016. [Online]. Available: <http://arxiv.org/abs/1511.05952>
- [90] D. V. Prokhorov and D. C. Wunsch, “Adaptive critic designs,” *IEEE transactions on Neural Networks*, vol. 8, no. 5, pp. 997–1007, 1997.
- [91] R. S. Sutton, D. McAllester, S. Singh, and Y. Mansour, “Policy gradient methods for reinforcement learning with function approximation,” *Advances in neural information processing systems*, vol. 12, 1999.
- [92] J. Peters and S. Schaal, “Natural actor-critic,” *Neurocomputing*, vol. 71, no. 7-9, pp. 1180–1190, 2008.
- [93] D. Silver, G. Lever, N. Heess, T. Degris, D. Wierstra, and M. Riedmiller, “Deterministic policy gradient algorithms,” in *International conference on machine learning*. Pmlr, 2014, pp. 387–395.

- [94] V. Mnih, A. P. Badia, M. Mirza, A. Graves, T. Lillicrap, T. Harley, D. Silver, and K. Kavukcuoglu, “Asynchronous methods for deep reinforcement learning,” in *International conference on machine learning*. PMLR, 2016, pp. 1928–1937.
- [95] T. P. Lillicrap, J. J. Hunt, A. Pritzel, N. Heess, T. Erez, Y. Tassa, D. Silver, and D. Wierstra, “Continuous control with deep reinforcement learning,” in *4th International Conference on Learning Representations, ICLR 2016, San Juan, Puerto Rico, May 2-4, 2016, Conference Track Proceedings*, Y. Bengio and Y. LeCun, Eds., 2016. [Online]. Available: <http://arxiv.org/abs/1509.02971>
- [96] T. Haarnoja, A. Zhou, P. Abbeel, and S. Levine, “Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor,” in *International conference on machine learning*. PMLR, 2018, pp. 1861–1870.
- [97] S. Levine, A. Kumar, G. Tucker, and J. Fu, “Offline reinforcement learning: Tutorial, review, and perspectives on open problems,” *arXiv preprint arXiv:2005.01643*, 2020.
- [98] A. K. Shakya, G. Pillai, and S. Chakrabarty, “Reinforcement learning algorithms: A brief survey,” *Expert Systems with Applications*, p. 120495, 2023.
- [99] S. A. Mostafa, M. S. Ahmad, A. Ahmad, and M. Annamalai, “Formulating situation awareness for multi-agent systems,” in *2013 International Conference on Advanced Computer Science Applications and Technologies*. IEEE, 2013, pp. 48–53.
- [100] A. Torreno, E. Onaindia, A. Komenda, and M. Štolba, “Cooperative multi-agent planning: A survey,” *ACM Computing Surveys (CSUR)*, vol. 50, no. 6, pp. 1–32, 2017.
- [101] Z. H. Ismail, N. Sariff, and E. G. Hurtado, “A survey and analysis of cooperative multi-agent robot systems: challenges and directions,” *Applications of Mobile Robots*, pp. 8–14, 2018.
- [102] L. Liu, J. Yang, Y. Zhang, J. Zhang, and Y. Ma, “An overview of opponent modeling for multi-agent competition,” in *International Conference on Machine Learning for Cyber Security*. Springer, 2022, pp. 634–648.

- [103] M. Dastani, F. Arbab, and F. De Boer, “Coordination and composition in multi-agent systems,” in *Proceedings of the fourth international joint conference on Autonomous agents and multiagent systems*, 2005, pp. 439–446.
- [104] K. Decker and V. Lesser, “A one-shot dynamic coordination algorithm for distributed sensor networks,” in *Proceedings of the Eleventh National Conference on Artificial Intelligence*, 1993, pp. 210–216.
- [105] “Towards flexible teamwork,” *Journal of artificial intelligence research*, vol. 7, pp. 83–124, 1997.
- [106] Z. Guessoum, J.-P. Briot, O. Marin, A. Hamel, and P. Sens, “Dynamic and adaptive replication for large-scale reliable multi-agent systems,” in *Software Engineering for Large-Scale Multi-Agent Systems: Research Issues and Practical Applications 1*. Springer, 2003, pp. 182–198.
- [107] M. Dignum, E. Sonenberg, F. Dignum, G. Vouros *et al.*, “Dynamic reorganization of agent societies,” in *Proceedings of workshop on coordination in emergent agent societies*, 2004.
- [108] G. Picard, J. F. Hübner, O. Boissier, and M.-P. Gleizes, “Reorganisation and self-organisation in multi-agent systems,” in *1st International Workshop on Organizational Modeling, ORGMOD*, 2009, pp. 66–80.
- [109] C. Bernon, M.-P. Gleizes, S. Peyruqueou, and G. Picard, “Adelfe: A methodology for adaptive multi-agent systems engineering,” in *Engineering Societies in the Agents World III: Third International Workshop, ESAW 2002 Madrid, Spain, September 16–17, 2002 Revised Papers 3*. Springer, 2003, pp. 156–169.
- [110] H. Karuna, P. Valckenaers, B. Saint-Germain, P. Verstraete, C. B. Zamfirescu, and H. Van Brussel, “Emergent forecasting using a stigmergy approach in manufacturing coordination and control,” in *Engineering Self-Organising Systems: Methodologies and Applications 2*. Springer, 2005, pp. 210–226.
- [111] G. D. M. Serugendo and M.-P. Irit, “Self-organisation and emergence in mas: An overview,” *Informatica*, vol. 30, no. 1, 2006.
- [112] A. Walker and M. J. Wooldridge, “Understanding the emergence of conventions in multi-agent systems,” in *ICMAS*, vol. 95, 1995, pp. 384–389.

- [113] J.-L. Dessalles and D. Phan, “Emergence in multi-agent systems: cognitive hierarchy, detection, and complexity reduction part i: methodological issues,” in *Artificial Economics: Agent-Based Methods in Finance, Game Theory and Their Applications*. Springer, 2006, pp. 147–159.
- [114] Y. M. Teo, B. L. Luong, and C. Szabo, “Formalization of emergence in multi-agent systems,” in *Proceedings of the 1st ACM SIGSIM Conference on Principles of Advanced Discrete Simulation*, 2013, pp. 231–240.
- [115] M. T. Hale, A. Nedić, and M. Egerstedt, “Cloud-based centralized/decentralized multi-agent optimization with communication delays,” in *2015 54th IEEE Conference on Decision and Control (CDC)*. IEEE, 2015, pp. 700–705.
- [116] H. T. Do, H. T. Hua, M. T. Nguyen, C. V. Nguyen, H. T. Nguyen, H. T. Nguyen, and N. T. Nguyen, “Formation control algorithms for multiple-uavs: a comprehensive survey,” *EAI Endorsed Transactions on Industrial Networks and Intelligent Systems*, vol. 8, no. 27, pp. e3–e3, 2021.
- [117] Q. Chen, Y. Wang, Y. Jin, T. Wang, X. Nie, and T. Yan, “A survey of an intelligent multi-agent formation control,” *Applied Sciences*, vol. 13, no. 10, p. 5934, 2023.
- [118] A. Esmaeili, N. Mozayani, M. R. J. Motlagh, and E. T. Matson, “The impact of diversity on performance of holonic multi-agent systems,” *Engineering Applications of Artificial Intelligence*, vol. 55, pp. 186–201, 2016.
- [119] C. Li, T. Wang, C. Wu, Q. Zhao, J. Yang, and C. Zhang, “Celebrating diversity in shared multi-agent reinforcement learning,” *Advances in Neural Information Processing Systems*, vol. 34, pp. 3991–4002, 2021.
- [120] K. R. McKee, J. Z. Leibo, C. Beattie, and R. Everett, “Quantifying the effects of environment and population diversity in multi-agent reinforcement learning,” *Autonomous Agents and Multi-Agent Systems*, vol. 36, no. 1, p. 21, 2022.
- [121] E. Argente, J. Palanca, G. Aranda, V. Julian, V. Botti, A. Garcia-Fornes, and A. Espinosa, “Supporting agent organizations,” in *Multi-Agent Systems and Applications V: 5th International Central and Eastern European Conference on Multi-Agent Systems, CEEMAS 2007, Leipzig, Germany, September 25-27, 2007. Proceedings 5*. Springer, 2007, pp. 236–245.
- [122] A. Koestler, “The ghost in the machine.” 1968.

- [123] S. Rodriguez, V. Hilaire, and A. Koukam, “Holonc modeling of environments for situated multi-agent systems,” in *Environments for Multi-Agent Systems II: Second International Workshop, E4MAS 2005, Utrecht, The Netherlands, July 25, 2005, Selected Revised and Invited Papers 2*. Springer, 2006, pp. 18–31.
- [124] G. D. M. Serugendo, M.-P. Gleizes, and A. Karageorgos, *Self-organising software: From natural to artificial adaptation*. Springer Science & Business Media, 2011.
- [125] S. Rodriguez, N. Gaud, V. Hilaire, S. Galland, and A. Koukam, “An analysis and design concept for self-organization in holonic multi-agent systems,” in *International Workshop on Engineering Self-Organising Applications*. Springer, 2006, pp. 15–27.
- [126] O. M. Shehory, K. Sycara, and S. Jha, “Multi-agent coordination through coalition formation,” in *Intelligent Agents IV Agent Theories, Architectures, and Languages: 4th International Workshop, ATAL’97 Providence, Rhode Island, USA, July 24–26, 1997 Proceedings 4*. Springer, 1998, pp. 143–154.
- [127] M. Van De Vijssel and J. Anderson, “Coalition formation in multi-agent systems under real-world conditions,” *Proceedings of association for the advancement of artificial intelligence*, 2004.
- [128] S. Ketchpel, “Forming coalitions in the face of uncertain rewards,” in *Aaai*, vol. 94, 1994, pp. 414–419.
- [129] G. Zlotkin and J. S. Rosenschein, *Coalition, cryptography, and stability: Mechanisms for coalition formation in task oriented domains*. Alfred P. Sloan School of Management, Massachusetts Institute of Technology, 1994.
- [130] L. Coviello and M. Franceschetti, “Distributed team formation in multi-agent systems: stability and approximation,” in *2012 IEEE 51st IEEE Conference on Decision and Control (CDC)*. IEEE, 2012, pp. 2755–2760.
- [131] M. E. Gaston and M. DesJardins, “Agent-organized networks for dynamic team formation,” in *Proceedings of the fourth international joint conference on Autonomous agents and multiagent systems*, 2005, pp. 230–237.
- [132] B. Dunin-Keplicz and R. Verbrugge, *Teamwork in multi-agent systems: A formal approach*. John Wiley & Sons, 2011.

- [133] C. H. Brooks, E. H. Durfee, and A. Armstrong, “An introduction to congregating in multi-agent systems,” in *Proceedings fourth international conference on multiagent systems*. IEEE, 2000, pp. 79–86.
- [134] C. H. Brooks and E. H. Durfee, “Congregating and market formation,” in *Proceedings of the first international joint conference on Autonomous agents and multiagent systems: part 1*, 2002, pp. 96–103.
- [135] —, “Congregation formation in multiagent systems,” *Autonomous Agents and Multi-Agent Systems*, vol. 7, pp. 145–170, 2003.
- [136] J. Carter, E. Bitting, and A. A. Ghorbani, “Reputation formalization for an information-sharing multi-agent system,” *Computational Intelligence*, vol. 18, no. 4, pp. 515–534, 2002.
- [137] B. Michael, “Intention, plans, and practical reason,” 1987.
- [138] A. S. Rao, M. P. Georgeff *et al.*, “Bdi agents: from theory to practice.” in *Icmas*, vol. 95, 1995, pp. 312–319.
- [139] M. Georgeff, B. Pell, M. Pollack, M. Tambe, and M. Wooldridge, “The belief-desire-intention model of agency,” in *Intelligent Agents V: Agents Theories, Architectures, and Languages: 5th International Workshop, ATAL’98 Paris, France, July 4–7, 1998 Proceedings 5*. Springer, 1999, pp. 1–10.
- [140] G. Jakobson, N. Parameswaran, J. Buford, L. Lewis, and P. Ray, “Situation-aware multi-agent system for disaster relief operations management,” *Proc. ISCRAM*, pp. 313–324, 2006.
- [141] K. Saleem, S. M. Akhtar, M. Nazir, A. S. Almadhor, Y. B. Zikria, R. Z. Ahmad, and S. W. Kim, “Situation aware intelligent reasoning during disaster situation in smart cities,” *Frontiers in Psychology*, vol. 13, p. 970789, 2022.
- [142] T. Strang and C. Linnhoff-Popien, “A context modeling survey,” in *Workshop Proceedings*, 2004.
- [143] C. Bettini, O. Brdiczka, K. Henriksen, J. Indulska, D. Nicklas, A. Ranganathan, and D. Riboni, “A survey of context modelling and reasoning techniques,” *Pervasive and mobile computing*, vol. 6, no. 2, pp. 161–180, 2010.

- [144] I. Roussaki, M. Strimpakou, N. Kalatzis, M. Anagnostou, and C. Pils, “Hybrid context modeling: A location-based scheme using ontologies,” in *Fourth Annual IEEE International Conference on Pervasive Computing and Communications Workshops (PERCOMW’06)*. IEEE, 2006, pp. 6–pp.
- [145] T. Gu, X. H. Wang, H. K. Pung, and D. Q. Zhang, “An ontology-based context model in intelligent environments,” *arXiv preprint arXiv:2003.05055*, 2020.
- [146] A. Al-Shdifat, C. Emmanouilidis, M. Khan, and A. G. Starr, “Ontology-based context modeling in physical asset integrity management,” *Frontiers in Computer Science*, vol. 2, p. 578673, 2020.
- [147] I. Horrocks, L. Li, D. Turi, and S. Bechhofer, “The instance store: DL reasoning with large numbers of individuals,” in *Proc. of the 2004 Description Logic Workshop (DL 2004)*, 2004, pp. 31–40.
- [148] P. Bellavista, A. Corradi, M. Fanelli, and L. Foschini, “A survey of context data distribution for mobile ubiquitous systems,” *ACM computing surveys (CSUR)*, vol. 44, no. 4, pp. 1–45, 2012.
- [149] B. Schilit, N. Adams, and R. Want, “Context-aware computing applications,” in *1994 first workshop on mobile computing systems and applications*. IEEE, 1994, pp. 85–90.
- [150] P. J. Brown, J. D. Bovey, and X. Chen, “Context-aware applications: from the laboratory to the marketplace,” *IEEE personal communications*, vol. 4, no. 5, pp. 58–64, 1997.
- [151] S. Qi and S.-C. Zhu, “Intent-aware multi-agent reinforcement learning,” in *2018 IEEE international conference on robotics and automation (ICRA)*. IEEE, 2018, pp. 7533–7540.
- [152] C. Tao, Q. Jiang, L. Duan, and P. Luo, “Dynamic and static context-aware lstm for multi-agent motion prediction,” in *European Conference on Computer Vision*. Springer, 2020, pp. 547–563.
- [153] X. Chen and P. Chaudhari, “Midas: Multi-agent interaction-aware decision-making with adaptive strategies for urban autonomous navigation,” in *2021 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2021, pp. 7980–7986.

- [154] K. Lee and R. Hasan Kabir, “Density-aware decentralised multi-agent exploration with energy constraint based on optimal transport theory,” *International Journal of Systems Science*, vol. 53, no. 4, pp. 851–869, 2022.
- [155] D. Fan, H. Shen, and L. Dong, “Switching-aware multi-agent deep reinforcement learning for target interception,” *Applied Intelligence*, vol. 53, no. 7, pp. 7876–7891, 2023.
- [156] N. Mahajan and Q. Zhang, “Intent-aware autonomous driving: A case study on highway merging scenarios,” *arXiv preprint arXiv:2309.13206*, 2023.
- [157] X. Wu, R. Chandra, T. Guan, A. Bedi, and D. Manocha, “Intent-aware planning in heterogeneous traffic via distributed multi-agent reinforcement learning,” in *Conference on Robot Learning*. PMLR, 2023, pp. 446–477.
- [158] A. Olaru, A. M. Florea, and A. El Fallah Seghrouchni, “A context-aware multi-agent system as a middleware for ambient intelligence,” *Mobile Networks and Applications*, vol. 18, pp. 429–443, 2013.
- [159] H. Haiouni and R. Maamri, “Context-aware multi-agent planning in intelligent environments,” *Multiagent and Grid Systems*, vol. 15, no. 3, pp. 219–236, 2019.
- [160] O. B. Kwon and N. Sadeh, “Applying case-based reasoning and multi-agent intelligent system to context-aware comparative shopping,” *Decision Support Systems*, vol. 37, no. 2, pp. 199–213, 2004.
- [161] J. Fu and Y. Fu, “An adaptive multi-agent system for cost collaborative management in supply chains,” *Engineering applications of artificial intelligence*, vol. 44, pp. 91–100, 2015.
- [162] B. Twardowski and D. Ryzko, “Iot and context-aware mobile recommendations using multi-agent systems,” in *2015 IEEE/WIC/ACM International Conference on Web Intelligence and Intelligent Agent Technology (WI-IAT)*, vol. 1. IEEE, 2015, pp. 33–40.
- [163] C. Huang, L. Yao, X. Wang, Q. Z. Sheng, S. Dustdar, Z. Wang, and X. Xu, “Intent-aware interactive internet of things for enhanced collaborative ambient intelligence,” *IEEE Internet Computing*, vol. 26, no. 5, pp. 68–75, 2022.
- [164] S. Yousaf, H. M. U. Haque, M. Atif, M. A. Hashmi, A. Khalid, and P. C. Vinh, “A context-aware multi-agent reasoning based intelligent assistive formalism,” *Internet of Things*, vol. 23, p. 100857, 2023.

- [165] A. Nadi and A. Edrisi, “Adaptive multi-agent relief assessment and emergency response,” *International journal of disaster risk reduction*, vol. 24, pp. 12–23, 2017.
- [166] S. M. Yusuf and C. Baber, “Formalizing distributed situation awareness in multi-agent networks,” *IEEE Transactions on Human-Machine Systems*, vol. 52, no. 6, pp. 1166–1175, 2022.
- [167] C. Yan, H. Fang, and H. Chao, “Energy-aware leader-follower tracking control for electric-powered multi-agent systems,” *Control Engineering Practice*, vol. 79, pp. 209–218, 2018.
- [168] G. Jelen, J. Babic, and V. Podobnik, “A multi-agent system for context-aware electric vehicle fleet routing: A step towards more sustainable urban operations,” *Journal of Cleaner Production*, vol. 374, p. 134047, 2022.
- [169] V. Riabchuk, L. Hagel, F. Germaine, and A. Zharova, “Utility-based context-aware multi-agent recommendation system for energy efficiency in residential buildings,” *arXiv preprint arXiv:2205.02704*, 2022.
- [170] V. Fuentes, N. S. Pi, J. Carbo, and J. M. Molina, “Reputation in user profiling for a context-aware multiagent system.” in *EUMAS*, 2006.
- [171] G. Castellano, M. G. Cimino, A. M. Fanelli, B. Lazzerini, F. Marcelloni, and M. A. Torsello, “A multi-agent system for enabling collaborative situation awareness via position-based stigmergy and neuro-fuzzy learning,” *Neurocomputing*, vol. 135, pp. 86–97, 2014.
- [172] M. Vladoiu and Z. Constantinescu, “Learning with a context-aware multi-agent system,” in *9th RoEduNet IEEE International Conference*. IEEE, 2010, pp. 368–373.
- [173] R. Bajcsy, “Active perception,” *Proceedings of the IEEE*, vol. 76, no. 8, pp. 966–1005, 1988.
- [174] R. Bajcsy, Y. Aloimonos, and J. K. Tsotsos, “Revisiting active perception,” *Autonomous Robots*, vol. 42, pp. 177–196, 2018.
- [175] T. Veiga and J. Renoux, “From reactive to active sensing: A survey on information gathering in decision-theoretic planning,” *ACM Computing Surveys*, vol. 55, no. 13s, pp. 1–22, 2023.
- [176] S. Franklin and A. Graesser, “Is it an agent, or just a program?: A taxonomy for autonomous agents,” in *International workshop on agent theories, architectures, and languages*. Springer, 1996, pp. 21–35.

- [177] F. Baader, A. Bauer, P. Baumgartner, A. Cregan, A. Gabaldon, K. Ji, K. Lee, D. Rajaratnam, and R. Schwitter, “A novel architecture for situation awareness systems,” in *Automated Reasoning with Analytic Tableaux and Related Methods: 18th International Conference, TABLEUX 2009, Oslo, Norway, July 6-10, 2009. Proceedings 18*. Springer, 2009, pp. 77–92.
- [178] Y. F. Chen, M. Liu, M. Everett, and J. P. How, “Decentralized non-communicating multiagent collision avoidance with deep reinforcement learning,” in *2017 IEEE international conference on robotics and automation (ICRA)*. IEEE, 2017, pp. 285–292.
- [179] N. Nezamoddini and A. Gholami, “A survey of adaptive multi-agent networks and their applications in smart cities,” *Smart Cities*, vol. 5, no. 1, pp. 318–347, 2022.
- [180] A. Bruce and G. Gordon, “Better motion prediction for people-tracking,” in *Proc. of the Int. Conf. on Robotics & Automation (ICRA), Barcelona, Spain, 2004*.
- [181] S. Dobson, S. Denazis, A. Fernández, D. Gaïti, E. Gelenbe, F. Massacci, P. Nixon, F. Saffre, N. Schmidt, and F. Zambonelli, “A survey of autonomous communications,” *ACM Transactions on Autonomous and Adaptive Systems (TAAS)*, vol. 1, no. 2, pp. 223–259, 2006.
- [182] A. Rudenko, L. Palmieri, M. Herman, K. M. Kitani, D. M. Gavrila, and K. O. Arras, “Human motion trajectory prediction: A survey,” *The International Journal of Robotics Research*, vol. 39, no. 8, pp. 895–935, 2020.
- [183] C. Julien and G.-C. Roman, “Supporting context-aware interaction in dynamic multi-agent systems,” in *International Workshop on Environments for Multi-Agent Systems*. Springer, 2004, pp. 168–189.
- [184] G.-C. Roman, C. Julien, and Q. Huang, “Network abstractions for context-aware mobile computing,” in *Proceedings of the 24th International Conference on Software Engineering*, 2002, pp. 363–373.
- [185] K. Wan and V. Alagar, “A context-aware trust model for service-oriented multi-agent systems,” in *International Conference on Service-Oriented Computing*. Springer, 2008, pp. 221–236.
- [186] M. Ghallab, D. Nau, and P. Traverso, *Automated Planning: theory and practice*. Elsevier, 2004.

- [187] J. Pearl, *Probabilistic reasoning in intelligent systems: networks of plausible inference*. Morgan kaufmann, 1988.
- [188] R. BELLMAN, *Adaptive Control Processes: A Guided Tour*. Princeton University Press, 1961. [Online]. Available: <http://www.jstor.org/stable/j.ctt183ph6v>
- [189] D. L. Donoho *et al.*, “High-dimensional data analysis: The curses and blessings of dimensionality,” *AMS math challenges lecture*, vol. 1, no. 2000, p. 32, 2000.
- [190] M. Verleysen and D. François, “The curse of dimensionality in data mining and time series prediction,” in *International work-conference on artificial neural networks*. Springer, 2005, pp. 758–770.
- [191] K. Menda, Y.-C. Chen, J. Grana, J. W. Bono, B. D. Tracey, M. J. Kochenderfer, and D. Wolpert, “Deep reinforcement learning for event-driven multi-agent decision processes,” *IEEE Transactions on Intelligent Transportation Systems*, vol. 20, no. 4, pp. 1259–1268, 2018.
- [192] J. Gao, P. Li, Z. Chen, and J. Zhang, “A survey on deep learning for multimodal data fusion,” *Neural Computation*, vol. 32, no. 5, pp. 829–864, 2020.
- [193] K. He, X. Zhang, S. Ren, and J. Sun, “Deep residual learning for image recognition,” in *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2016, pp. 770–778.
- [194] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, “You only look once: Unified, real-time object detection,” in *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2016, pp. 779–788.
- [195] E. Rohmer, S. P. Singh, and M. Freese, “V-rep: A versatile and scalable robot simulation framework,” in *2013 IEEE/RSJ international conference on intelligent robots and systems*. IEEE, 2013, pp. 1321–1326.
- [196] S. Hochreiter and J. Schmidhuber, “Long short-term memory,” *Neural computation*, vol. 9, no. 8, pp. 1735–1780, 1997.
- [197] K. D. Katyal, G. D. Hager, and C.-M. Huang, “Intent-aware pedestrian prediction for adaptive crowd navigation,” in *2020 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2020, pp. 3277–3283.

- [198] J. Lee, Y. Lee, J. Kim, A. Kosiorek, S. Choi, and Y. W. Teh, “Set transformer: A framework for attention-based permutation-invariant neural networks,” in *International conference on machine learning*. PMLR, 2019, pp. 3744–3753.
- [199] D. P. Kingma and M. Welling, “Auto-encoding variational bayes,” *CoRR*, vol. abs/1312.6114, 2013. [Online]. Available: <https://api.semanticscholar.org/CorpusID:216078090>
- [200] T. N. Kipf and M. Welling, “Semi-supervised classification with graph convolutional networks,” in *International Conference on Learning Representations*, 2016.
- [201] J. Chung, C. Gulcehre, K. Cho, and Y. Bengio, “Empirical evaluation of gated recurrent neural networks on sequence modeling,” *arXiv preprint arXiv:1412.3555*, 2014.
- [202] P. Veličković, G. Cucurull, A. Casanova, A. Romero, P. Lio, and Y. Bengio, “Graph attention networks,” *arXiv preprint arXiv:1710.10903*, 2017.
- [203] S. P. Ferrando and E. Onaindia, “Context-aware multi-agent planning in intelligent environments,” *Information Sciences*, vol. 227, pp. 22–42, 2013.
- [204] T. Murata, “Petri nets: Properties, analysis and applications,” *Proceedings of the IEEE*, vol. 77, no. 4, pp. 541–580, 1989.
- [205] I. Watson and F. Marir, “Case-based reasoning: A review,” *The knowledge engineering review*, vol. 9, no. 4, pp. 327–354, 1994.
- [206] G. Beydoun, G. Low, H. Mouratidis, and B. Henderson-Sellers, “A security-aware metamodel for multi-agent systems (mas),” *Information and software technology*, vol. 51, no. 5, pp. 832–845, 2009.
- [207] F. Feyzi, “Model-driven development of self-adaptive multi-agent systems with context-awareness,” *International Journal of Computer Aided Engineering and Technology*, vol. 12, no. 2, pp. 131–156, 2020.
- [208] Y. Shi, A. Karatzoglou, L. Baltrunas, M. Larson, and A. Hanjalic, “Cars2: Learning context-aware representations for context-aware recommendations,” in *proceedings of the 23rd ACM International Conference on Conference on Information and Knowledge Management*, 2014, pp. 291–300.
- [209] H. Robbins and S. Monro, “A stochastic approximation method,” *The annals of mathematical statistics*, pp. 400–407, 1951.

- [210] G. J. McLachlan and K. E. Basford, *Mixture models: Inference and applications to clustering*. M. Dekker New York, 1988, vol. 38.
- [211] R. Bellman, “A markovian decision process,” *Journal of mathematics and mechanics*, pp. 679–684, 1957.
- [212] A. R. Cassandra, L. P. Kaelbling, and M. L. Littman, “Acting optimally in partially observable stochastic domains,” in *Aaai*, vol. 94, 1994, pp. 1023–1028.
- [213] L. P. Kaelbling, M. L. Littman, and A. R. Cassandra, “Planning and acting in partially observable stochastic domains,” *Artificial intelligence*, vol. 101, no. 1-2, pp. 99–134, 1998.
- [214] C. J. Watkins and P. Dayan, “Q-learning,” *Machine learning*, vol. 8, pp. 279–292, 1992.
- [215] G. A. Rummery and M. Niranjan, *On-line Q-learning using connectionist systems*. University of Cambridge, Department of Engineering Cambridge, UK, 1994, vol. 37.
- [216] S. Mahadevan, “Average reward reinforcement learning: Foundations, algorithms, and empirical results,” *Machine learning*, vol. 22, no. 1, pp. 159–195, 1996.
- [217] T. Schaul, D. Horgan, K. Gregor, and D. Silver, “Universal value function approximators,” in *International conference on machine learning*. PMLR, 2015, pp. 1312–1320.
- [218] H. Sun, Z. Li, X. Liu, B. Zhou, and D. Lin, “Policy continuation with hindsight inverse dynamics,” *Advances in Neural Information Processing Systems*, vol. 32, 2019.
- [219] H. Van Hasselt, A. Guez, and D. Silver, “Deep reinforcement learning with double q-learning,” in *Proceedings of the AAAI conference on artificial intelligence*, vol. 30, no. 1, 2016.
- [220] T. P. Lillicrap, J. J. Hunt, A. Pritzel, N. M. O. Heess, T. Erez, Y. Tassa, D. Silver, and D. Wierstra, “Continuous control with deep reinforcement learning,” *CoRR*, vol. abs/1509.02971, 2015. [Online]. Available: <https://api.semanticscholar.org/CorpusID:16326763>
- [221] M. Babaeizadeh, I. Frosio, S. Tyree, J. Clemons, and J. Kautz, “Reinforcement learning through asynchronous advantage actor-critic on

- a gpu,” in *International Conference on Learning Representations*, 2016. [Online]. Available: <https://api.semanticscholar.org/CorpusID:1163588>
- [222] D. D. Corkill, D. Garant, and V. R. Lesser, “Exploring the effectiveness of agent organizations,” in *Coordination, Organizations, Institutions, and Norms in Agent Systems XI: COIN 2015 International Workshops, COIN@ AAMAS, Istanbul, Turkey, May 4, 2015, COIN@ IJCAI, Buenos Aires, Argentina, July 26, 2015, Revised Selected Papers*. Springer, 2016, pp. 78–97.
- [223] H. Kim, H. Shim, and J. H. Seo, “Output consensus of heterogeneous uncertain linear multi-agent systems,” *IEEE Transactions on Automatic Control*, vol. 56, no. 1, pp. 200–206, 2010.
- [224] J.-H. Cho, “Dynamics of uncertain and conflicting opinions in social networks,” *IEEE transactions on computational social systems*, vol. 5, no. 2, pp. 518–531, 2018.
- [225] H. Du, S. Thudumu, R. Vasa, and K. Mouzakis, “A survey on context-aware multi-agent systems: Techniques, challenges and future directions,” *arXiv preprint arXiv:2402.01968*, 2024.
- [226] J. Jiang and Z. Lu, “Learning attentional communication for multi-agent cooperation,” *Advances in neural information processing systems*, vol. 31, 2018.
- [227] Y.-C. Liu, J. Tian, C.-Y. Ma, N. Glaser, C.-W. Kuo, and Z. Kira, “Who2com: Collaborative perception via learnable handshake communication,” in *2020 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2020, pp. 6876–6883.
- [228] P. Sunehag, G. Lever, A. Gruslys, W. M. Czarnecki, V. Zambaldi, M. Jaderberg, M. Lanctot, N. Sonnerat, J. Z. Leibo, K. Tuyls, and T. Graepel, “Value-decomposition networks for cooperative multi-agent learning based on team reward,” in *Proceedings of the 17th International Conference on Autonomous Agents and MultiAgent Systems*, ser. AAMAS ’18. Richland, SC: International Foundation for Autonomous Agents and Multiagent Systems, 2018, p. 2085–2087.
- [229] Z. Xu, B. Zhang, Y. Bai, D. Li, and G. Fan, “Learning to coordinate via multiple graph neural networks,” in *Neural Information Processing: 28th International Conference, ICONIP 2021, Sanur, Bali, Indonesia, December 8–12, 2021, Proceedings, Part III 28*. Springer, 2021, pp. 52–63.

- [230] E. Pesce and G. Montana, “Learning multi-agent coordination through connectivity-driven communication,” *Machine Learning*, vol. 112, no. 2, pp. 483–514, 2023.
- [231] F. A. Oliehoek, C. Amato *et al.*, *A concise introduction to decentralized POMDPs*. Springer, 2016, vol. 1.
- [232] F. Christianos, L. Schäfer, and S. Albrecht, “Shared experience actor-critic for multi-agent reinforcement learning,” *Advances in neural information processing systems*, vol. 33, pp. 10 707–10 717, 2020.
- [233] F. Zhuang, Z. Qi, K. Duan, D. Xi, Y. Zhu, H. Zhu, H. Xiong, and Q. He, “A comprehensive survey on transfer learning,” *Proceedings of the IEEE*, vol. 109, no. 1, pp. 43–76, 2020.
- [234] C. Zhang, Y. Xie, H. Bai, B. Yu, W. Li, and Y. Gao, “A survey on federated learning,” *Knowledge-Based Systems*, vol. 216, p. 106775, 2021.
- [235] F. L. Da Silva, G. Warnell, A. H. R. Costa, and P. Stone, “Agents teaching agents: a survey on inter-agent transfer learning,” *Autonomous Agents and Multi-Agent Systems*, vol. 34, pp. 1–17, 2020.
- [236] R. J. Williams, “Simple statistical gradient-following algorithms for connectionist reinforcement learning,” *Machine learning*, vol. 8, pp. 229–256, 1992.
- [237] C. J. C. H. Watkins, “Learning from delayed rewards,” 1989.
- [238] D. P. Kingma and J. Ba, “Adam: A method for stochastic optimization,” *arXiv preprint arXiv:1412.6980*, 2014.
- [239] J. Wang, Y. Zhang, T.-K. Kim, and Y. Gu, “Shapley q-value: A local reward approach to solve global reward games,” in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 34, no. 05, 2020, pp. 7285–7292.
- [240] T. Wang, H. Dong, V. Lesser, and C. Zhang, “Roma: Multi-agent reinforcement learning with emergent roles,” in *International Conference on Machine Learning*. PMLR, 2020, pp. 9876–9886.
- [241] A. Singh, T. Jain, and S. Sukhbaatar, “Learning when to communicate at scale in multiagent cooperative and competitive tasks,” in *International Conference on Learning Representations*, 2018.

- [242] A. Das, T. Gervet, J. Romoff, D. Batra, D. Parikh, M. Rabbat, and J. Pineau, “Tarmac: Targeted multi-agent communication,” in *International Conference on machine learning*. PMLR, 2019, pp. 1538–1546.
- [243] S. Q. Zhang, Q. Zhang, and J. Lin, “Efficient communication in multi-agent reinforcement learning via variance based control,” *Advances in neural information processing systems*, vol. 32, 2019.
- [244] Y. Hu, S. Fang, Z. Lei, Y. Zhong, and S. Chen, “Where2comm: Communication-efficient collaborative perception via spatial confidence maps,” *Advances in neural information processing systems*, vol. 35, pp. 4874–4886, 2022.
- [245] L. Yuan, J. Wang, F. Zhang, C. Wang, Z. Zhang, Y. Yu, and C. Zhang, “Multi-agent incentive communication via decentralized teammate modeling,” in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 36, no. 9, 2022, pp. 9466–9474.
- [246] S. Ding, W. Du, L. Ding, L. Guo, and J. Zhang, “Learning efficient and robust multi-agent communication via graph information bottleneck,” in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 38, no. 16, 2024, pp. 17 346–17 353.
- [247] W. Du, S. Ding, L. Guo, J. Zhang, and L. Ding, “Expressive multi-agent communication via identity-aware learning,” in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 38, no. 16, 2024, pp. 17 354–17 361.
- [248] X. Meng and Y. Tan, “Pmac: Personalized multi-agent communication,” in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 38, no. 16, 2024, pp. 17 505–17 513.
- [249] F. L. Da Silva, R. Glatt, and A. H. R. Costa, “Simultaneously learning and advising in multiagent reinforcement learning,” in *Proceedings of the 16th conference on autonomous agents and multiagent systems*, 2017, pp. 1100–1108.
- [250] Y. Ba, X. Liu, X. Chen, H. Wang, Y. Xu, K. Li, and S. Zhang, “Cautiously-optimistic knowledge sharing for cooperative multi-agent reinforcement learning,” in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 38, no. 16, 2024, pp. 17 299–17 307.

- [251] Z. Ding, T. Huang, and Z. Lu, “Learning individually inferred communication for multi-agent cooperation,” *Advances in neural information processing systems*, vol. 33, pp. 22 069–22 079, 2020.
- [252] H. Mao, Z. Zhang, Z. Xiao, Z. Gong, and Y. Ni, “Learning multi-agent communication with double attentional deep reinforcement learning,” *Autonomous Agents and Multi-Agent Systems*, vol. 34, pp. 1–34, 2020.
- [253] P. Sunehag, G. Lever, A. Gruslys, W. M. Czarnecki, V. Zambaldi, M. Jaderberg, M. Lanctot, N. Sonnerat, J. Z. Leibo, K. Tuyls *et al.*, “Value-decomposition networks for cooperative multi-agent learning based on team reward,” in *Proceedings of the 17th International Conference on Autonomous Agents and MultiAgent Systems*, 2018, pp. 2085–2087.
- [254] H. Jiang, Z. Ding, and Z. Lu, “Settling decentralized multi-agent coordinated exploration by novelty sharing,” in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 38, no. 16, 2024, pp. 17 444–17 452.
- [255] C. S. De Witt, T. Gupta, D. Makoviichuk, V. Makoviychuk, P. H. Torr, M. Sun, and S. Whiteson, “Is independent learning all you need in the starcraft multi-agent challenge?” *arXiv preprint arXiv:2011.09533*, 2020.
- [256] C. Jin, Q. Liu, Y. Wang, and T. Yu, “V-learning—a simple, efficient, decentralized algorithm for multiagent rl,” in *ICLR 2022 Workshop on Gamification and Multiagent Solutions*, 2022.
- [257] C. Daskalakis, N. Golowich, and K. Zhang, “The complexity of markov equilibrium in stochastic games,” in *The Thirty Sixth Annual Conference on Learning Theory*. PMLR, 2023, pp. 4180–4234.
- [258] A. Skrynnik, A. Andreychuk, M. Nesterova, K. Yakovlev, and A. Panov, “Learn to follow: Decentralized lifelong multi-agent pathfinding via planning and learning,” in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 38, no. 16, 2024, pp. 17 541–17 549.
- [259] R. Devidze, P. Kamalaruban, and A. Singla, “Exploration-guided reward shaping for reinforcement learning under sparse rewards,” *Advances in Neural Information Processing Systems*, vol. 35, pp. 5829–5842, 2022.